

book 15

interview defense cookbook

resume claims cross referenced to the handbooks

Raeliosol

version 4.2, compiled 10 2026

Contents

1.	how to use this book	6
1.1.	the document map	6
1.2.	the rehearsal loop	6
1.3.	where the time goes	7
1.4.	ground rules for every answer	7
2.	concordance master table	9
2.1.	how to read this chapter	9
2.2.	the shape of the defense	10
2.3.	drill tables	12
2.4.	drill: specialization tag answers	12
2.5.	drill: contributions defense	13
2.6.	drill: employment defense	14
2.7.	drill: gap and chronology narratives	15
2.8.	drill: education and participations answers	15
2.9.	drill: no titles and no metrics policy answers	16
2.10.	drill: hostile cross examination playbook	16
2.11.	drill: dotfiles deep dive	16
2.12.	drill: modern-swe-library deep dive	16
2.13.	drill: llm-tournament deep dive	17
2.14.	drill: go-edge-mesh deep dive	17
2.15.	drill: order-demo deep dive	17
2.16.	drill: FlowerShop deep dive	17
2.17.	drill: SumoBot deep dive	18
2.18.	drill: dota-helper deep dive	18
2.19.	the reverse index, floor to claims	18
2.20.	claims without a handbook floor	20
3.	specialization tag answers	21
3.1.	identity line	21
3.2.	contact line	21
3.3.	Linux	22
3.4.	C/Go/Java/C#/JS/Python/Lua APIs	22
3.5.	SQLite/DuckDB	23
3.6.	FunctionalProgramming	24
3.7.	DistributedSystems	24
3.8.	TDD	25
3.9.	KISS	26
4.	contributions defense	27
4.1.	the numbers, one table	27
4.2.	dotfiles	28
4.3.	modern-swe-library	28
4.4.	llm-tournament	29
5.	employment defense	31
5.1.	Dikshatek	31
5.2.	NTQ Solution	32
5.3.	FPT Software	32
5.4.	PALTech	33
6.	gap and chronology narratives	34
6.1.	the timeline, one table	34

6.2.	the transitions	34
6.3.	the live gap	35
6.4.	the pattern, stated once	35
7.	education and participations answers	37
7.1.	the degree	37
7.2.	the 2015 to 2018 window	37
7.3.	sumo championship	38
7.4.	ICPC qualifier	38
8.	no titles and no metrics policy answers	40
8.1.	no titles	40
8.2.	no metrics	40
8.3.	verbal number rules	41
9.	hostile cross examination playbook	43
9.1.	the three modes	43
9.2.	the rules under pressure	43
9.3.	the deep probe map	43
9.4.	the hobbies lines	44
9.5.	the endgame	45
10.	rapid fire fact sheet	47
10.1.	the numbers	47
10.2.	the dates	47
10.3.	the series, one line	48
10.4.	twelve one line answers	48
11.	the repo evidence method	50
11.1.	the snapshots	50
11.2.	the two numbers the api cannot give	50
11.3.	the measurements that need a clone	51
11.4.	the vendoring rules	51
11.5.	the provenance discipline	52
11.6.	the artifact resets the chain	52
12.	the types of logic, derived and attacked	55
12.1.	the frame: what, why, how, when	55
12.2.	deduction	55
12.3.	induction	56
12.4.	abduction	56
12.5.	statistical and causal inference	57
12.6.	dialectic and definitional moves	57
12.7.	the six logics, one table	58
13.	fallacies, biases, and appeals	59
13.1.	formal fallacies	59
13.2.	fallacies of relevance	59
13.3.	fallacies of ambiguity	60
13.4.	presupposition and loaded questions	60
13.5.	cognitive biases	61
13.6.	appeals	61
13.7.	the counter playbook	61
14.	four stances, steelmanned and attacked	63
14.1.	why stances at all	63
14.2.	biological realism	63
14.3.	nominalism	64
14.4.	phenomenology	65

14.5. individualist anarchism	65
14.6. the move both ways	66
15. game theory and the evolutionary drivers	67
15.1. the primitives	67
15.2. signaling and cheap talk	67
15.3. repeated games	68
15.4. commitment and focal points	69
15.5. the evolutionary drivers	69
15.6. reading the room with the toolkit	70
16. the deception taxonomy	71
16.1. self-image maintenance	71
16.2. post-hoc rationalization	72
16.3. status negotiation	72
16.4. social-environment manipulation	72
16.5. motivated reasoning	73
16.6. appeal-to-x as a weapon	73
16.7. presupposition	73
16.8. each driver through the frame	74
17. coming out ahead in every exchange	76
17.1. the exchange as a system	76
17.2. framing and anchoring	76
17.3. question control	77
17.4. silence, timing, and concessions	77
17.5. repair	78
17.6. escalation and exits	78
17.7. the whole book in one loop	79
18. dotfiles deep dive	81
18.1. verified facts	81
18.2. the shape in thirty seconds	82
18.3. open this file first	82
18.4. drills	84
19. modern-swe-library deep dive	86
19.1. verified facts	86
19.2. the shape in thirty seconds	86
19.3. open this file first	87
19.4. drills	87
20. llm-tournament deep dive	89
20.1. verified facts	89
20.2. the shape in thirty seconds	90
20.3. open this file first	90
20.4. drills	92
21. go-edge-mesh deep dive	94
21.1. verified facts	94
21.2. the shape in thirty seconds	94
21.3. open this file first	95
21.4. drills	97
22. order-demo deep dive	99
22.1. verified facts	99
22.2. the shape in thirty seconds	99
22.3. open this file first	99
22.4. drills	102

23. FlowerShop deep dive	104
23.1. verified facts	104
23.2. the shape in thirty seconds	104
23.3. open this file first	105
23.4. drills	107
24. SumoBot deep dive	109
24.1. verified facts	109
24.2. the shape in thirty seconds	109
24.3. open this file first	110
24.4. drills	112
25. dota-helper deep dive	114
25.1. verified facts	114
25.2. the shape in thirty seconds	115
25.3. open these five files first	115
25.4. drills	119

1. how to use this book

This book is the rehearsal half of a two document defense. Volume 0, volume 0, ch 1 (resume defense analysis, method), takes resume v7.16 apart line by line and argues every severity, challenge hypothesis, and trap. This book compresses that reasoning into drills: hostile questions with spoken answers and the handbook chapter that holds the technical floor under each claim. Read volume 0 for why. Rehearse this book for what to say.

One block of this book sits on the other side of the table. Chapters 12 through 17 are the interviewer's side: the types of logic an exchange runs on, the fallacies and appeals fired at a candidate, the philosophy stances probes smuggle in, the game theory and evolutionary drivers behind deception, and the communication mechanics of the exchange itself. They carry no register rows because they defend no resume claim, they defend against the person asking.

1.1. the document map

Sixteen documents stand behind one resume page:

- volume 0, ch 10 (resume defense analysis, consolidated claim register) holds the claim register, the data this book's concordance renders.
- Book 1 is the systems manual: C23 on LLVM and clang, the operating system, the machine, and the cloud through Terraform, AWS, and GCP. It is the floor under the machine-level probes.
- Book 2 is the mathematics manual, trigonometry to monads in C23 with a deterministic physics game capstone. It is the floor under the mathematics claims.
- Books 3 through 8 are the complete language manuals, Go, Java, C# and .NET, JavaScript and TypeScript, Python, Lua. They are the floor under the language claims, and books 1 and 3 through 8 each carry a service part building the same tested rest api against frozen golden vectors, the floor under the api claim.
- Books 9 and 10 are the algorithms pair: data structures and algorithms in seven languages with a storage engine capstone, and the icpc world finals book with every problem walked through in seven languages. Books 11 through 14 are the applied handbooks: patterns, concurrency and distributed systems with a Raft capstone, infrastructure with a compose chat suite capstone, knowledge discovery in C23 with a go and duckdb pipeline capstone, and game systems with a deterministic artillery capstone. They are the floor under the depth probes.
- Book 16, the interview repertoire, is this book's sibling: it drills the frequent technical questions the same way this book drills the resume claims.

Every cross reference in this book resolves against the chapter manifests at compile time, so a chapter that moves or renames fails the build instead of silently rotting the mapping.

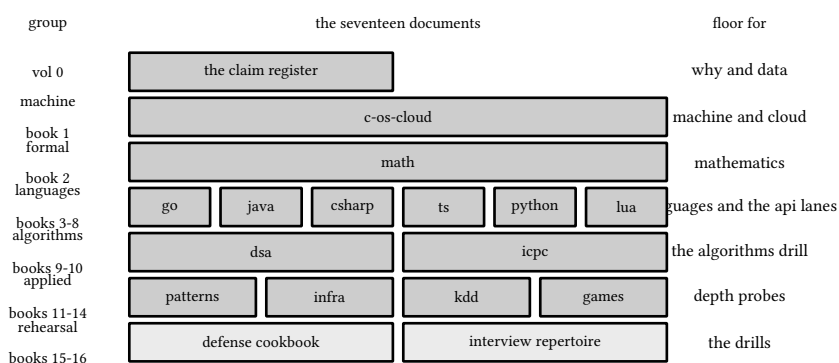
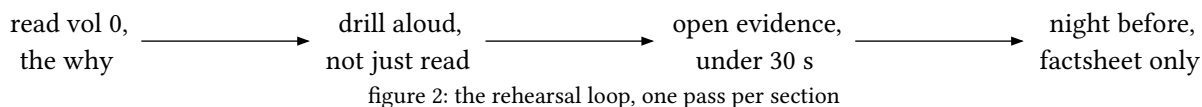


figure 1: the seventeen documents behind one resume page and what each is the floor for

1.2. the rehearsal loop

One pass per section, in this order:

1. Read the matching volume 0 chapter once, for the reasoning and the full trap lists. This book does not repeat the reasoning, it assumes it.
2. Drill the matching chapter here aloud. A drill that is only read is not drilled. The answers are scripts to internalize and then speak naturally, not lines to recite.
3. Open the evidence. Repos are public, the numbers live on their pages, the 2026-09-10 snapshots under coverage/gh pin the measured values this book drills, and the handbook chapters are the depth probes' answers. Rehearse locating each number in under thirty seconds.
4. The night before, read only chapter 10, the factsheet. Everything else is already internalized or it is not.



1.3. where the time goes

The register drives the budget. Counting it the same way volume 0 does:

48 claims: 7 safe, 21 defensible, 20 framing.

Safe rows get nothing, they are dates and public facts. Defensible rows get evidence drills, locating and defining the proof. Framing rows get scripted answers, because a framing row is true but invites a wrong conclusion if the narrative does not land first. Inside that tier sit the deliberate historical claims: numbers true of the artifact the resume line names, the 100+ gist stars, the 100k SLOC on the pre-python_service tree. Their drills state the history, which artifact the line was true of and what happened to it, and give today's measured numbers.

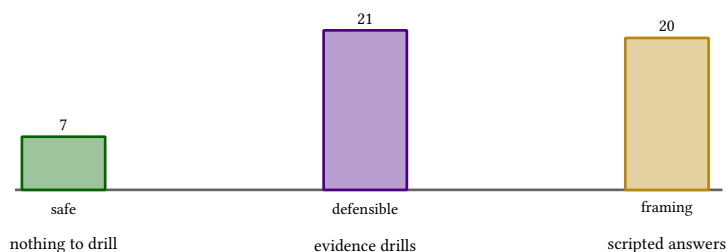
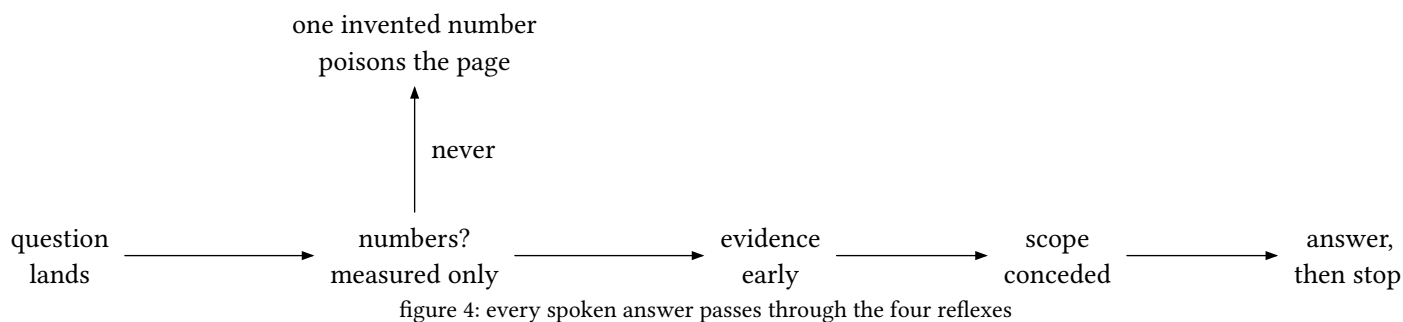


figure 3: the severity budget counted off the register, and what rehearsal effort each severity buys

1.4. ground rules for every answer

The same four rules volume 0 states, compressed to reflex length:

- Never invent a number verbally. One invented number, caught, poisons every other number on the page.
- Point at evidence early. "It is on my GitHub, the link is on the resume" is a complete sentence.
- Concede scope honestly and fast. "I used SQLite as an embedded key value store, not as a tuned OLTP server" kills the attack that was coming.
- Do not fill silence. Answer, stop, let the next question come.



NOTE: on honesty

This book only defends what the resume actually says and what the repos actually contain. Where the honest answer is “I have not done that at scale”, the drill says so, and the strategy is scope honesty, not bluster. A rehearsed overreach fails live, in front of the one audience that matters.

2. concordance master table

The concordance is the whole defense in one view: every claim from the volume 0 register, its severity, the chapter of this book that drills it, and the handbook chapters that hold its technical floor. The register lives in `books/analysis/register.typ` and this table imports it directly, so editing the register updates both volume 0 and this book in one compile. The build validates every reference against the chapter manifests, a wrong book or chapter id fails the compile, which is what makes this table trustworthy enough to rehearse from.

2.1. how to read this chapter

Three artifacts feed each other. The **register** in volume 0 holds one row per defended claim with a severity and a list of handbook references. The **drill chapters** of this book, chapters 3 through 9 plus the per repo deep dives of chapters 18 through 25, turn groups of register rows into spoken answers, one chapter per group. The **floors** are the handbook chapters a claim leans on technically, and they are what an interviewer can push into after the scripted answer lands.

Every count, table row, and arrow below is computed from the register at compile time, so the picture cannot drift from the data. The drill map itself is guarded: a register id whose prefix matches no drill chapter fails the build instead of rendering silently un drilled.

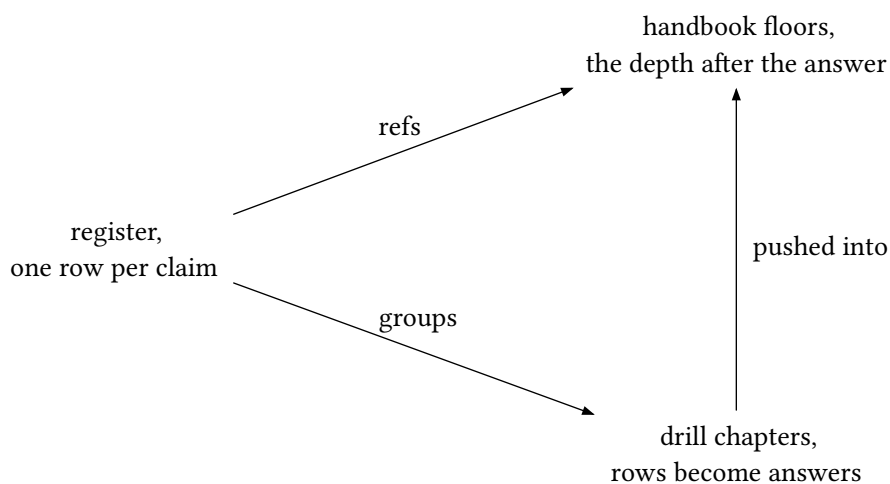


figure 5: the three artifacts, register rows, drill chapters, and handbook floors feeding each other

2.2. the shape of the defense

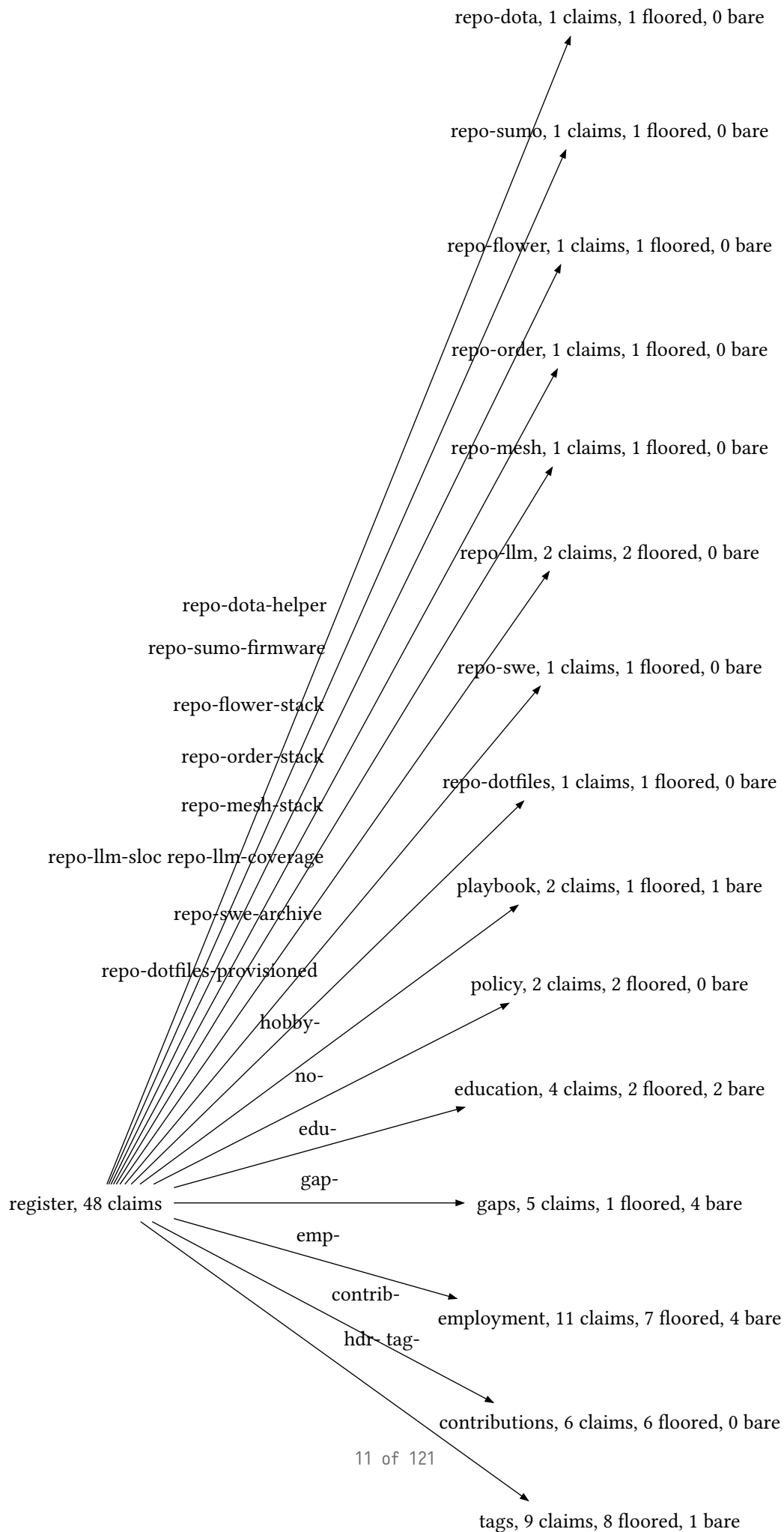


figure 6: the register routing claims into the drill chapters by id prefix

The register fans out into the drill chapters by id prefix. Each chapter node carries its own split: claims that stand on handbook floors get the depth probes of the reverse index below, bare claims get the evidence walkthrough of chapter 4 and the factsheet of chapter 10.

2.3. drill tables

One table per drill chapter, in reading order. Each row is one register claim: its id for cross reference back to volume 0, the claim in resume terms, its severity, and the handbook chapters that back it. Severity carries the rehearsal instruction, safe rows need nothing, defensible rows need evidence at hand, framing rows need the scripted answer from the drill chapter named in the heading.

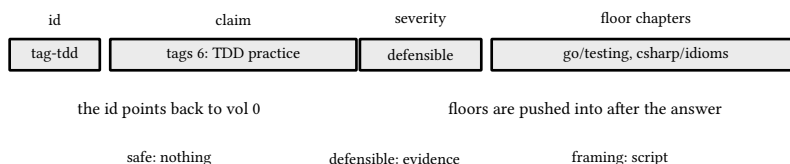


figure 7: one drill table row, the severity column read as the rehearsal instruction

2.4. drill: specialization tag answers

The header line and the seven specialization tags. Chapter 3 drills the identity answer, the contact discipline, and one spoken answer per tag. 9 claims, 1 of them with no handbook floor.

id	claim	severity	handbook floor
hdr-identity	header line 1: Raeliosol, bare name, no level word	framing	book 5, ch 17 (c# 15, f# 11, and .net 11, idioms), book 3, ch 15 (go 1.27, idioms)
hdr-contact	header line 2: messaging-apps-only phone, HCMC timezone	framing	none, evidence is the response
tag-linux	tags 1: Linux as daily driver	defensible	volume 0, ch 3 (resume defense analysis, contributions)
tag-langs	tags 2: C, Go, Java, C#, JS, Python, Lua API work	framing	book 3, ch 1 (go 1.27, toolchain and modules), book 4, ch 1 (java 27, toolchain and the jdk), book 5, ch 1 (c# 15, f# 11, and .net 11, toolchain and project model), book 6, ch 1 (javascript es2026 and typescript 7, javascript and its toolchain), book 7, ch 1 (python 3.14, toolchain: cpython 3.14 on windows), book 8, ch 3 (lua 5.5, values, metatables, metamethods)
tag-store	tags 3: SQLite/DuckDB embedded storage pair	defensible	book 12, ch 10 (infrastructure: docker, databases, and testing, sqlite in production: embedded kvs and game saves), book 12, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine), book 12, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine), volume 0, ch 4 (resume defense analysis, employment claims)
tag-fp	tags 4: functional programming style	defensible	book 5, ch 3 (c# 15, f# 11, and .net 11, the type system), book 5, ch 14 (c# 15, f# 11, and .net 11, linq), book 5, ch 18 (c# 15, f#

id	claim	severity	handbook floor
			11, and .net 11, functional programming), book 6, ch 17 (javascript es2026 and type-script 7, type-level programming)
tag-dist	tags 5: distributed systems work	framing	book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals), book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft), book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency)
tag-tdd	tags 6: TDD practice	defensible	volume 0, ch 3 (resume defense analysis, contributions), book 5, ch 17 (c# 15, f# 11, and .net 11, idioms), book 3, ch 14 (go 1.27, testing)
tag-kiss	tags 7: simplicity discipline	safe	volume 0, ch 3 (resume defense analysis, contributions), book 11, ch 1 (design patterns, concurrency, and distributed systems, pattern discipline)

2.5. drill: contributions defense

The three public repositories. Chapter 4 drills the numbers table and the per repo evidence walkthroughs, each number located in under thirty seconds. 6 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
contrib-dotfiles	contributions 1: dotfiles repo, 35 stars since 2023	safe	volume 0, ch 2 (resume defense analysis, header and specialization tags), book 15, ch 18 (interview defense cookbook, dotfiles deep dive)
contrib-dotfiles-idem	contributions 1 b1: one-click idempotent bootstrap, Linux and Windows	defensible	book 15, ch 18 (interview defense cookbook, dotfiles deep dive)
contrib-dotfiles-counts	contributions 1 b2-3: 20 LSPs, 30 tree-sitter parsers, 40+ CLI tools, 20 languages	framing	volume 0, ch 2 (resume defense analysis, header and specialization tags), book 15, ch 18 (interview defense cookbook, dotfiles deep dive)
contrib-swe	contributions 2: modern-swe-library, 100+ stars on a gist deleted at archive time, historical	framing	book 15, ch 19 (interview defense cookbook, modern-swe-library deep dive)
contrib-swe-path	contributions 2 b2-3: 2,600-hour study path, no AI slop, archived 1/2025	framing	book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)
contrib-llm	contributions 3: llm-tournament, 8 stars, 100k SLOC historical, 100% coverage live again	framing	volume 0, ch 2 (resume defense analysis, header and specialization tags), book 3, ch 14 (go 1.27, testing), book 5, ch 17 (c# 15, f# 11, and .net 11, idioms), book 15, ch 20 (interview defense cookbook, llm-tournament deep dive)

2.6. drill: employment defense

The four employers and their scope claims. Chapter 5 drills per employer stories, honest scope lines, and the demo repos as evidence. 11 claims, 4 of them with no handbook floor.

id	claim	severity	handbook floor
emp-dik-header	Dikshatek header: three stints, two modes, 2020 to 2025	framing	none, evidence is the response
emp-dik-sqlite	Dikshatek b1: C#/Go/Docker/Postgres/SQLite-as-embedded-KVS/DuckDB APIs	defensible	book 9, ch 44 (practical data structures and algorithms, capstone: storage engine toolkit), book 5, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics), book 3, ch 12 (go 1.27, stdlib tour 1: fmt, errors, slices, maps, cmp, time, slog), book 11, ch 16 (design patterns, concurrency, and distributed systems, capstone: raft replicated configuration service), book 12, ch 10 (infrastructure: docker, databases, and testing, sqlite in production: embedded kvs and game saves), book 12, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine)
emp-dik-craft	Dikshatek b2: tests, handlers, middlewares, business logic	framing	book 3, ch 14 (go 1.27, testing), book 5, ch 17 (c# 15, f# 11, and .net 11, idioms)
emp-dik-migration	Dikshatek b3: legacy Java/.NET to Go/NodeJS migration, behavioral equivalence	defensible	book 11, ch 1 (design patterns, concurrency, and distributed systems, pattern discipline), book 3, ch 14 (go 1.27, testing)
emp-dik-mesh	Dikshatek b4: southbound edge mesh, Go, Grafana, CNCF, construction sites	framing	book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals), book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft), book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), book 3, ch 6 (go 1.27, concurrency)
emp-ntq-header	NTQ header: 10 months, order-demo link	defensible	none, evidence is the response
emp-ntq-flutter	NTQ b1 b4: Flutter features, 3rd party sandbox integrations	defensible	none, evidence is the response
emp-ntq-orders	NTQ b2 b3: Go, Protobuf/PostgreSQL, gRPC orders, MongoDB pipelines	defensible	book 3, ch 13 (go 1.27, stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io, testing), book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), book 5, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics)
emp-fpt-header	FPT header b1: 5 months, E470 toll system slice	framing	none, evidence is the response

id	claim	severity	handbook floor
emp-fpt-avro	FPT b2 b3: Spring Boot, Avro/PostgreSQL, Kafka on GCP, ISO-matched tests	defensible	book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), book 11, ch 14 (design patterns, concurrency, and distributed systems, observability)
emp-pal	PALTech all: whitelabel exchange platforms, Angular frontends, FlowerShop	defensible	book 5, ch 3 (c# 15, f# 11, and .net 11, the type system), book 5, ch 17 (c# 15, f# 11, and .net 11, idioms), book 11, ch 15 (design patterns, concurrency, and distributed systems, resilience)

2.7. drill: gap and chronology narratives

The chronology claims between and after the jobs. Chapter 6 drills the timeline narrative and the current gap as a built period with inspectable output. 5 claims, 4 of them with no handbook floor.

id	claim	severity	handbook floor
gap-2020a	3/2020 to 7/2020: four month gap, pandemic quarter	framing	none, evidence is the response
gap-2020b	11/2020 to 12/2020: one month transition	safe	none, evidence is the response
gap-2021	12/2021 to 1/2022: one month transition	safe	none, evidence is the response
gap-2022	10/2022 to 12/2022: two months, rehired by previous client	safe	none, evidence is the response
gap-now	12/2025 to present: about nine months, degree plus build period	framing	volume 0, ch 6 (resume defense analysis, education timeline), volume 0, ch 3 (resume defense analysis, contributions)

2.8. drill: education and participations answers

The degree line and the participations. Chapter 7 drills the degree framing, the sumobot technicals, and the ICPC one liner. 4 claims, 2 of them with no handbook floor.

id	claim	severity	handbook floor
edu-degree	education: CITD-UIT accredited online BSc 2024 to 2027, in progress	framing	none, evidence is the response
edu-window	2015 to 2018: competitions then self study before first job	framing	none, evidence is the response
edu-sumo	participations 1: 2017 sumo champion zero loss, custom PCB, C/asm firmware, fuzzy logic, bit masking, PID	defensible	book 14, ch 5 (game systems and architecture, finite state machines), book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking), book 15, ch 24 (interview defense cookbook, SumoBot deep dive)
edu-icpc	participations 2: 2015 ICPC qualifier participant	safe	book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking), book 9, ch 10 (practical data structures and algorithms, graphs), book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic), book 9, ch 24 (practical data

id	claim	severity	handbook floor
			structures and algorithms, lattice geometry and grid algorithms)

2.9. drill: no titles and no metrics policy answers

The two policy positions, no titles and no metrics. Chapter 8 drills the policy answers including the informal title variants and the verbal number rules. 2 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
no-titles	all employment headers: no job titles anywhere	framing	volume 0, ch 2 (resume defense analysis, header and specialization tags), volume 0, ch 4 (resume defense analysis, employment claims)
no-metrics	all employment bullets: no impact metrics anywhere, policy	framing	volume 0, ch 3 (resume defense analysis, contributions), book 11, ch 14 (design patterns, concurrency, and distributed systems, observability)

2.10. drill: hostile cross examination playbook

The hobbies line claims. Chapter 9 carries their drills inside the hostile playbook, game programming answers that survive cross examination. 2 claims, 1 of them with no handbook floor.

id	claim	severity	handbook floor
hobby-games	hobbies 1: systems games, science, ancient texts	safe	none, evidence is the response
hobby-ecs	hobbies 2: game programming with ECS and FSM, WC3 map making	framing	book 14, ch 2 (game systems and architecture, ecs theory), book 14, ch 3 (game systems and architecture, an ecs implementation in c#), book 14, ch 5 (game systems and architecture, finite state machines), book 14, ch 19 (game systems and architecture, capstone part 1: the battle engine), book 8, ch 32 (lua 5.5, capstone: auto chess, human versus bot)

2.11. drill: dotfiles deep dive

The dotfiles provisioning counts. Chapter 18 drills the measured lists, 20 LSPs, 30 parsers, 46 installables, each located live. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-dotfiles-provisioned	contributions 1 b2-3: provisioning measured, 20 LSPs, 30 parsers, 46 installables, cli_tools alone 18	defensible	book 15, ch 18 (interview defense cookbook, dotfiles deep dive)

2.12. drill: modern-swe-library deep dive

The archived library and its stars story. Chapter 19 drills the history-plus-measurement for the one number the evidence pass cannot verify today. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-swe-archive	contributions 2: single-file readme.md repo, archived 1/2025, stars measured repo 9 gist 0, the 100+ star gist was deleted at archive time so no version of the artifact exists to check today	framing	book 15, ch 19 (interview defense cookbook, modern-swe-library deep dive)

2.13. drill: llm-tournament deep dive

The tournament measurements. Chapter 20 drills the verbatim numbers, 42,059 of 49,780 and 100.0 percent, and the CI gate that enforces it. 2 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-llm-sloc	contributions 3 b1: size measured 42,059 code of 49,780 total, tokei 15.0.0, main @ f046fb2	defensible	book 15, ch 20 (interview defense cookbook, llm-tournament deep dive)
repo-llm-coverage	contributions 3 b2: coverage measured 100.0% go statements atomic mode go1.27.1, badge 100.0, ci gate enforces it, the 100% figure reads true again since python_service was removed	defensible	book 3, ch 14 (go 1.27, testing), book 15, ch 20 (interview defense cookbook, llm-tournament deep dive)

2.14. drill: go-edge-mesh deep dive

The edge mesh demo stack. Chapter 21 drills the topology walk from fake sensor to grafana panel. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-mesh-stack	Dikshatek b4 demo: go-edge-mesh demo stack, edgex compose, mosquitto, telegraf, influxdb, grafana	defensible	book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals), book 15, ch 21 (interview defense cookbook, go-edge-mesh deep dive)

2.15. drill: order-demo deep dive

The order demo stack. Chapter 22 drills the sqlc and paseto choices and the production scoping. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-order-stack	NTQ demo: order-demo stack, gin, sqlc, postgres migrations, paseto auth	defensible	book 3, ch 13 (go 1.27, stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io, testing), book 15, ch 22 (interview defense cookbook, order-demo deep dive)

2.16. drill: FlowerShop deep dive

The FlowerShop stack. Chapter 23 drills the oldest repo, its chronology value, and its dated habits. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-flower-stack	PALTech demo: FlowerShop stack, spring boot JPA, MySQL, Angular storefront	defensible	book 11, ch 15 (design patterns, concurrency, and distributed systems, re-

id	claim	severity	handbook floor
			silience), book 15, ch 23 (interview defense cookbook, FlowerShop deep dive)

2.17. drill: SumoBot deep dive

The sumo firmware. Chapter 24 drills the reflex ladder and concedes what the chip does not run. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-sumo-firmware	participations 1: sumobot.c on AT89S52, timer PWM, sonar threshold, reflex ladder, no fuzzy or pid on chip	defensible	book 14, ch 5 (game systems and architecture, finite state machines), book 15, ch 24 (interview defense cookbook, SumoBot deep dive)

2.18. drill: dota-helper deep dive

The pick-simulator engine. Chapter 25 drills the measured repo, its exact-statistics discipline, and the twin contract. 1 claims, 0 of them with no handbook floor.

id	claim	severity	handbook floor
repo-dota-helper	hobbies 1: dota-helper, offline dota 2 pick-simulator and hero-pool guide over duckdb, exact hand-rolled statistics with a bit-identical js twin	defensible	book 3, ch 31 (go 1.27, numerical statistics in go), book 3, ch 33 (go 1.27, the offline engine and its js twin), book 15, ch 25 (interview defense cookbook, dota-helper deep dive)

2.19. the reverse index, floor to claims

Read the drill tables claim first. The reverse index reads floor first: which handbook chapters are load bearing, and for which claims. Before an interview pitched at a specific stack, drill the rows under the relevant floor chapters, those are the depth probes most likely to fire. It is the handbook floors of the drill tables inverted, the same edges walked from the other end. The floors the index leaves out sit outside the handbooks: the analysis refs point into volume 0, whose own chapters hold the why for tag-linux, tag-store, tag-tdd, tag-kiss, contrib-dotfiles, contrib-dotfiles-counts, contrib-llm, gap-now, no-titles, no-metrics, and the cookbook refs are this book's repo deep dives, which drill the claim instead of flooring it.

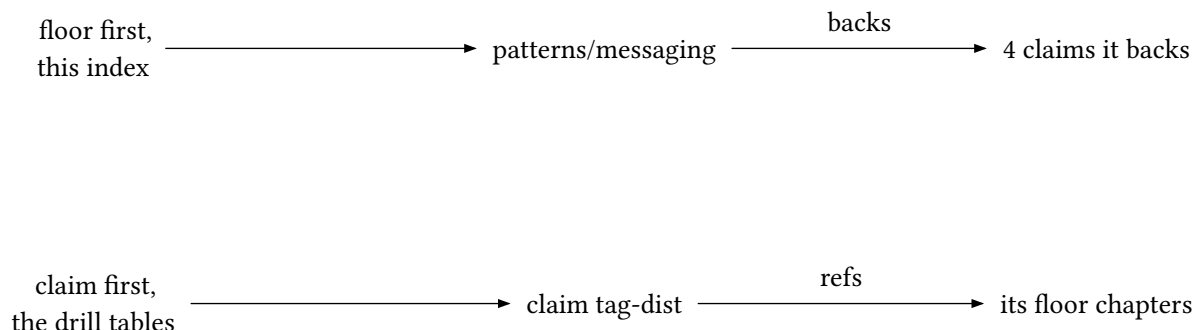


figure 8: the reverse index, the same edges as the drill tables walked from the floor end

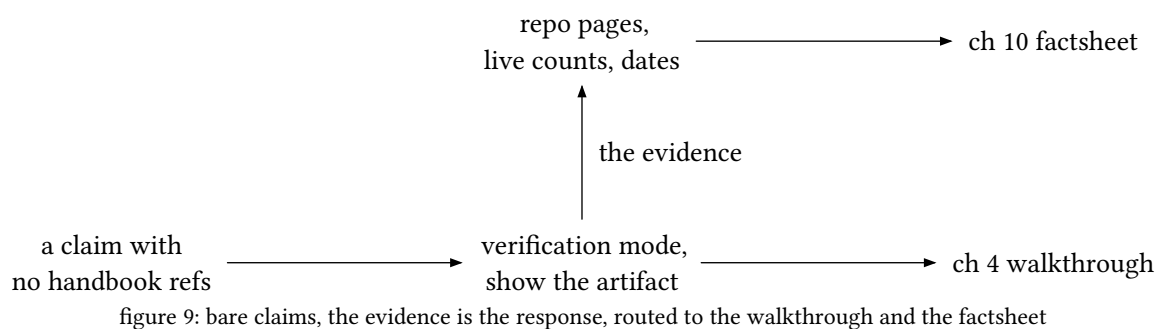
floor chapter	claims it backs
book 5, ch 17 (c# 15, f# 11, and .net 11, idioms)	hdr-identity, tag-tdd, contrib-llm, emp-dik-craft, emp-pal

floor chapter	claims it backs
book 5, ch 1 (c# 15, f# 11, and .net 11, toolchain and project model)	tag-langs
book 5, ch 3 (c# 15, f# 11, and .net 11, the type system)	tag-fp, emp-pal
book 5, ch 14 (c# 15, f# 11, and .net 11, linq)	tag-fp
book 5, ch 18 (c# 15, f# 11, and .net 11, functional programming)	tag-fp
book 5, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics)	emp-dik-sqlite, emp-ntq-orders
book 3, ch 15 (go 1.27, idioms)	hdr-identity
book 3, ch 1 (go 1.27, toolchain and modules)	tag-langs
book 3, ch 14 (go 1.27, testing)	tag-tdd, contrib-llm, emp-dik-craft, emp-dik-migration, repo-llm-coverage
book 3, ch 12 (go 1.27, stdlib tour 1: fmt, errors, slices, maps, cmp, time, slog)	emp-dik-sqlite
book 3, ch 6 (go 1.27, concurrency)	emp-dik-mesh
book 3, ch 13 (go 1.27, stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io, testing)	emp-ntq-orders, repo-order-stack
book 3, ch 31 (go 1.27, numerical statistics in go)	repo-dota-helper
book 3, ch 33 (go 1.27, the offline engine and its js twin)	repo-dota-helper
book 6, ch 1 (javascript es2026 and typescript 7, javascript and its toolchain)	tag-langs
book 6, ch 17 (javascript es2026 and typescript 7, type-level programming)	tag-fp
book 7, ch 1 (python 3.14, toolchain: cpython 3.14 on windows)	tag-langs
book 8, ch 3 (lua 5.5, values, metatables, metamethods)	tag-langs
book 8, ch 32 (lua 5.5, capstone: auto chess, human versus bot)	hobby-ecs
book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)	contrib-swe-path, edu-sumo, edu-icpc
book 9, ch 44 (practical data structures and algorithms, capstone: storage engine toolkit)	emp-dik-sqlite
book 9, ch 10 (practical data structures and algorithms, graphs)	edu-icpc
book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic)	edu-icpc
book 9, ch 24 (practical data structures and algorithms, lattice geometry and grid algorithms)	edu-icpc
book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals)	tag-dist, emp-dik-mesh, repo-mesh-stack
book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft)	tag-dist, emp-dik-mesh
book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency)	tag-dist, emp-dik-mesh, emp-ntq-orders, emp-fpt-avro
book 11, ch 1 (design patterns, concurrency, and distributed systems, pattern discipline)	tag-kiss, emp-dik-migration

floor chapter	claims it backs
book 11, ch 16 (design patterns, concurrency, and distributed systems, capstone: raft replicated configuration service)	emp-dik-sqlite
book 11, ch 14 (design patterns, concurrency, and distributed systems, observability)	emp-fpt-avro, no-metrics
book 11, ch 15 (design patterns, concurrency, and distributed systems, resilience)	emp-pal, repo-flower-stack
book 12, ch 10 (infrastructure: docker, databases, and testing, sqlite in production: embedded kvs and game saves)	tag-store, emp-dik-sqlite
book 12, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine)	tag-store, emp-dik-sqlite
book 12, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine)	tag-store
book 14, ch 5 (game systems and architecture, finite state machines)	edu-sumo, hobby-ecs, repo-sumo-firmware
book 14, ch 2 (game systems and architecture, ecs theory)	hobby-ecs
book 14, ch 3 (game systems and architecture, an ecs implementation in c#)	hobby-ecs
book 14, ch 19 (game systems and architecture, capstone part 1: the battle engine)	hobby-ecs

2.20. claims without a handbook floor

Some claims carry no handbook reference because the response itself is the evidence: repo pages, live counts, calendar facts. These are the verification mode questions, verification being one of the three questioning modes chapter 9 names, and the drill for them is the live evidence walkthrough in chapter 4 and the factsheet in chapter 10, not a book.



hdr-contact	emp-dik-header	emp-ntq-header	emp-ntq-flutter
emp-fpt-header	gap-2020a	gap-2020b	gap-2021
gap-2022	edu-degree	edu-window	hobby-games

NOTE: one source of truth

The register is edited in one place, books/analysis/register.typ, and both documents re-render on the next compile. If a claim's severity or floor changes after a resume revision, fix the register first, then let these tables and volume 0's register chapter pick it up. Never hand edit a row here.

3. specialization tag answers

The header row is the resume's thesis and the first thing probed. This chapter drills the identity line, the contact line, and each of the seven tags. The full reasoning per claim lives in volume 0, ch 2 (resume defense analysis, header and specialization tags), and the register rows `hdr-*` and `tag-*` map here. Every drill answer routes from claim to evidence in two moves or fewer.

3.1. identity line

The header opens with “Raeliosol”, a bare name, no title and no level word. The drill answer refuses the level word and the defensive upgrade.

How would you level yourself? Junior, mid, senior?

The header carries no title because a self-assigned level word is the weakest claim on the page. I write software for a living in several languages and I prefer being judged on the work over a level word I assigned myself. I have shipped client work as the whole backend on C# and Go for five years, the Dikshatek and NTQ stints, 50 plus 10 months, and the repos on the resume are the calibration. Level me off those.

floor: book 5, ch 17 (c# 15, f# 11, and .net 11, idioms), book 3, ch 15 (go 1.27, idioms)

No title at all, not engineer, not architect?

The header carries no title by design. Inside the work I have designed an edge mesh and owned client APIs end to end, and I would rather describe that scope than coin a noun for my own header.

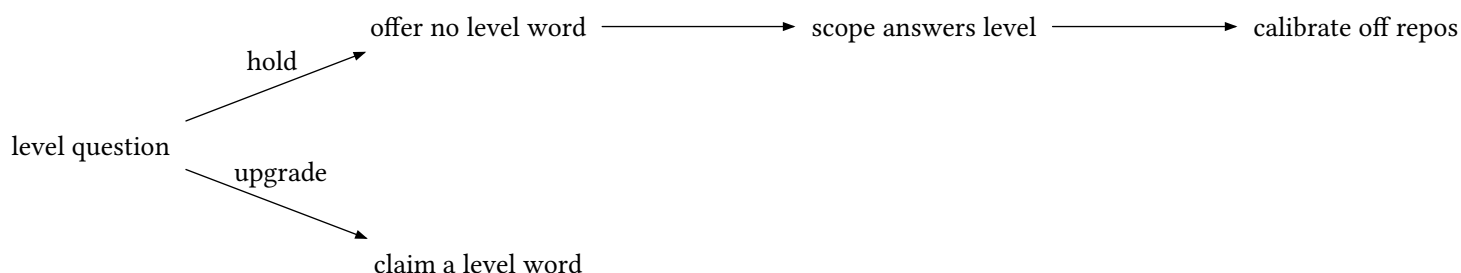


figure 10: the identity line, the level word refused, scope and repos answering it instead

3.2. contact line

HCMC, UTC+7, messaging apps for the personal phone number. One trap, one answer.

If we cannot call you, how do you do standups?

The restriction is on inbound cold calls to a personal Vietnamese mobile that gets spam called. Every scheduled call, interview, or standup happens normally on whatever platform the team uses. I am in Ho Chi Minh City, UTC+7, and I have overlapped European mornings and Asia Pacific hours for client work.

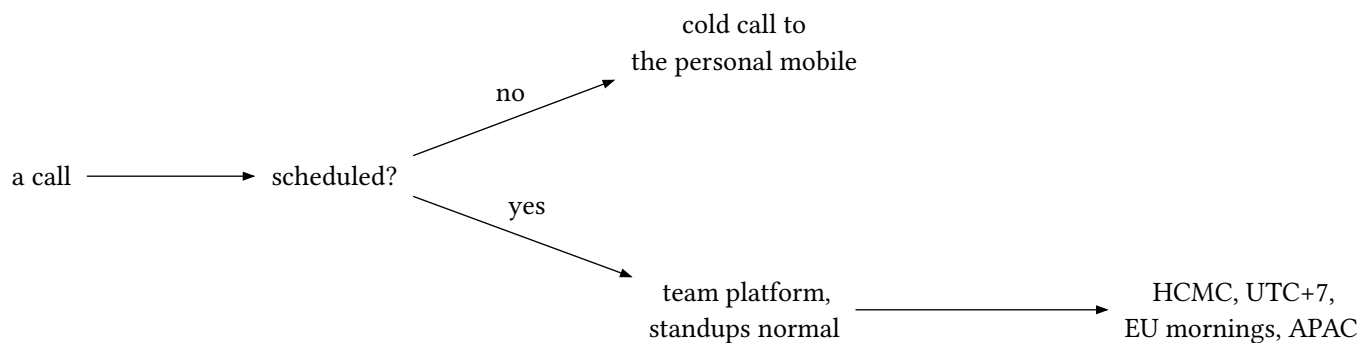


figure 11: the contact line, cold calls restricted, scheduled calls normal, UTC+7 overlap held

3.3. Linux

Daily driver, provable from the dotfiles repo in under a minute. Expect one system level question.

You put Linux first. What does that mean concretely?

Linux is my daily driver, bare metal and WSL2, and the dotfiles repo automates the whole environment, one idempotent command. Concretely: shell, ssh, systemd units, permissions, and reading strace or journalctl when something misbehaves. The provisioning lists are public, you can count the tools during the call.

floor: volume 0, ch 3 (resume defense analysis, contributions)

What happens when you type ls and press enter?

The shell resolves ls through PATH, forks, execs it, the dynamic loader maps libc, and the kernel returns directory entries that ls formats. I can go deeper on any leg of that on request, that is the difference between using Linux and listing it.

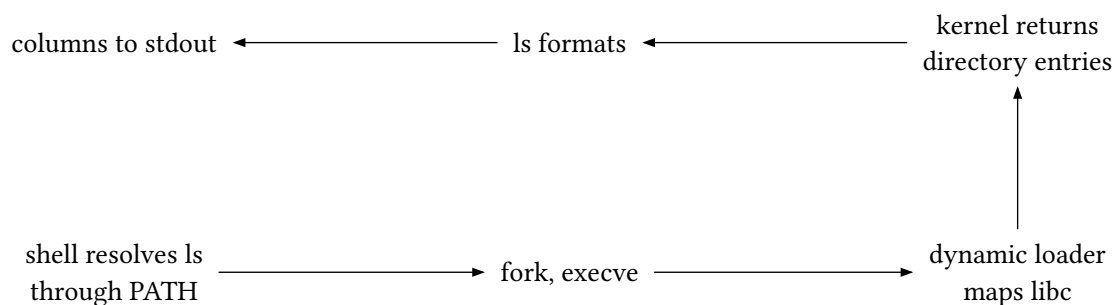


figure 12: what runs when ls runs, each leg of the answer one node deep

3.4. C/Go/Java/C#/JS/Python/Lua APIs

The loudest claim on the resume. The scoping word is APIs, and the honest split arrives before the probe does.

Seven languages. Which one are you actually best at?

The honest split: C# and Go are my production API languages, five years of client work, the Dikshatek and NTQ bullets. JavaScript is the API surface I write when the product needs it, TypeScript is its type layer. C is the systems end that goes with the C23 manual, Python is my scripting and tooling language, and Lua is embedded scripting from game programming. Java is the newest depth lane, the java 27 manual carries its own tested http service and the line clone capstone, so its proof is the book lane, not client work. For a greenfield API I would pick Go for a lean service surface and C# where the team leans .NET, and I can defend either on operational grounds.

floor: book 3, ch 1 (go 1.27, toolchain and modules), book 4, ch 1 (java 27, toolchain and the jdk), book 5, ch 1 (c# 15, f# 11, and .net 11, toolchain and project model), book 6, ch 1 (javascript es2026 and typescript 7, javascript and its toolchain), book 7, ch 1 (python 3.14, toolchain: cpython 3.14 on windows), book 8, ch 3 (lua 5.5, values, metatables, metamethods)

Seven languages. Where is the proof you can ship an api in them?

The service corpus. Books 1 and 3 through 8 each carry a service part building the same user rest api from first principles, one lane per language, every lane taught against the same frozen golden vectors: the http kernel, the middleware chain, authn with a password kdf and a hand rolled jwt, the store, etags and idempotency, caching, rate limiting, and the byte exact vector replays. Each lane runs under its own make gate, verify-capi through the lua lanes, with the counts recorded in the audit docs. The split above is how I talk about it, this is what I can open.

floor: book 1, ch 28 (c23: the language, the machine, and the cloud, http kernel), book 3, ch 23 (go 1.27, store), book 4, ch 19 (java 27, http kernel), book 5, ch 22 (c# 15, f# 11, and .net 11, api kernel), book 6, ch 23 (javascript es2026 and typescript 7, api kernel), book 7, ch 27 (python 3.14, http kernel), book 8, ch 19 (lua 5.5, the http kernel)

Where does Lua fit in real work for you?

Embedded scripting and tooling. It entered through WC3 custom map making, and I went deep enough to write the 5.5 manual including the C API outline, so metatables and coroutines are fair game.

language	honest role	the evidence
C	systems depth	the c23 systems manual
Go	production API pair	client APIs owned end to end
Java	newest full manual	book 4, service and line clone
C#	production API pair	Dikshatek and NTQ bullets
JS	api surface on demand	typescript as its type layer
Python	scripting and tooling	tooling to the 3.14 manual
Lua	embedded scripting	WC3 to the 5.5 manual

greenfield: Go for a lean service, C# where the team leans .NET

figure 13: seven languages, seven honest roles, one greenfield rule

3.5. SQLite/DuckDB

The storage pair tag. Two embedded engines, and the drilled answer is which deployment shape each one fits.

Why do two databases share one tag?

They answer different questions. SQLite is the embedded OLTP engine, single writer process, WAL and busy timeout, no network hop and no DBA, the right shape for client state on customer boxes and game saves. DuckDB is the embedded analytical engine, columnar and vectorized, the right shape when the question is aggregation over stored data instead of row by row transactions. Choosing the engine per deployment shape is what the tag claims, and the choosing chapter is the floor.

floor: book 12, ch 10 (infrastructure: docker, databases, and testing, sqlite in production: embedded kvs and game saves), book 12, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine)

Convince me the storage choice is engineered, not brand loyalty.

Constraints first, then the choice falls out. SQLite as an embedded key value store: the constraint set was single writer process, no network hop, no DBA on customer boxes, so a database server is an anti answer. Same habit as re-deriving amortized $O(1)$ array append from the doubling argument instead of memorizing the table, know the why, then borrow proven answers freely.

floor: book 9, ch 3 (practical data structures and algorithms, dynamic arrays and amortization)

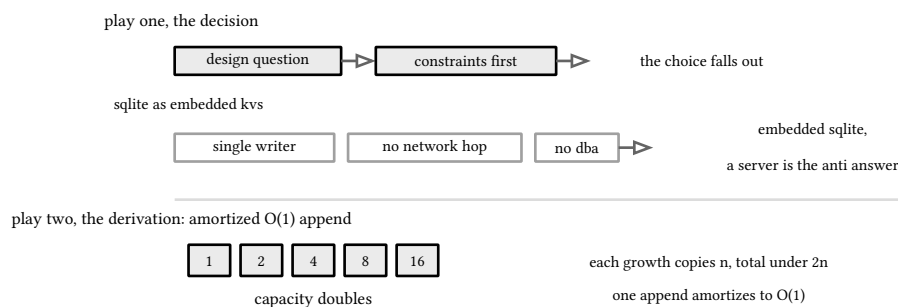


figure 14: the storage tag as two plays, the sqlite constraint set and the doubling derivation

3.6. FunctionalProgramming

On a resume with OOP heavy bullets, the FP tag needs a concrete anchor.

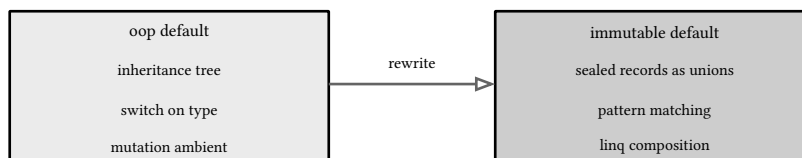
C# is not an FP language. Why is this tag here?

Functionally I write means immutable data by default, records and readonly structs, C# 15 union types with exhaustive pattern matching over them, LINQ for composition, and pure functions wherever state is not needed. In Go it is simpler, functions as values and no hidden mutation. It is a default, not a religion, which is also why KISS sits next to it.

floor: book 5, ch 3 (c# 15, f# 11, and .net 11, the type system), book 5, ch 18 (c# 15, f# 11, and .net 11, functional programming), book 6, ch 17 (javascript es2026 and typescript 7, type-level programming)

Show me a place where FP made the code worse.

Heavy higher order abstraction over two call sites. Readability died and I inlined it back. That answer exists because the question is a trap the other way, an FP advocate who cannot name a failure is advocating a religion.



the failure conceded: abstraction over two call sites, inlined back

figure 15: the same routine, oop default against immutable default

3.7. DistributedSystems

The strongest tag and the most attackable. The scope statement comes first, the Raft concession comes before it is asked.

You have one distributed system on this resume. Why the tag?

The evidence is the edge mesh: Go agents on construction site boxes, Grafana telemetry, CNCF components where they fit, handling intermittent uplinks. When the uplink dies the agents keep collecting, buffer locally, and reconcile on reconnect. That design problem, partial ordering, idempotent writes, backpressure, is what the tag claims.

floor: book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals), book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft), book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency)

Have you run real consensus, Raft or Paxos, in production?

No, and I will say so. The edge mesh did not earn Raft, local persistence and last writer wins on versioned config was the right cost trade. Then I can walk you through Raft elections and log replication at whiteboard depth, I implemented it for the book 11 capstone. The tag claims the problem class I work in, not a title I have held.

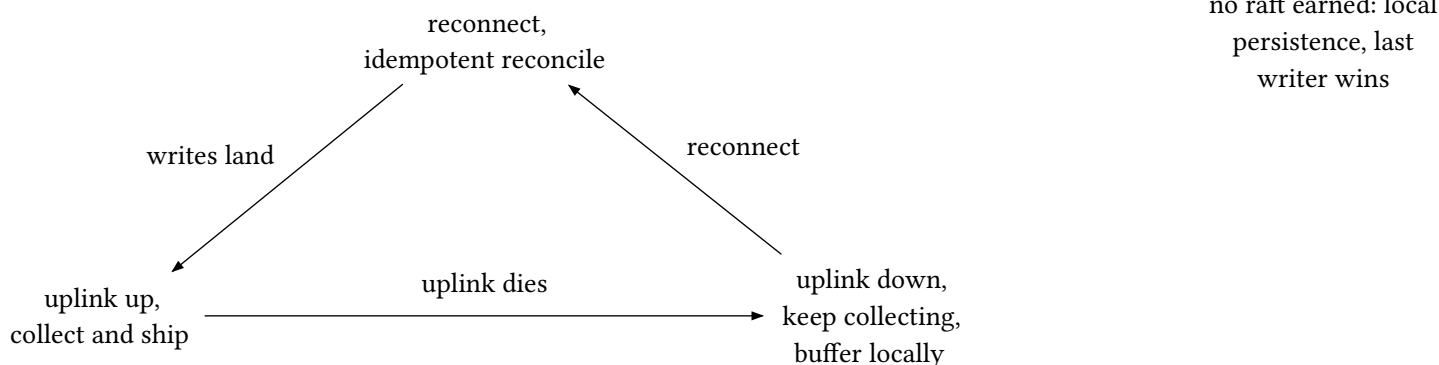


figure 16: the partition cycle, collect, buffer, reconcile, with the no-raft concession parked beside it

3.8. TDD

Backed by the llm-tournament measurement, not by the resume's headline: the measured number today is 100.0% total Go statement coverage on 42k code lines, every package with statements at exactly 100.0, with a CI gate holding the floor, the "100% global" wording was true of the tree the line was written against, dipped to 99.6 while python_service sat untested, and reads true again since the 2026-09-30 head removed it, and the tag's defense gives that arc and stands on the measurement.

headline 100% reads true again at the 2026-09-30 head, measured today: 100.0 total

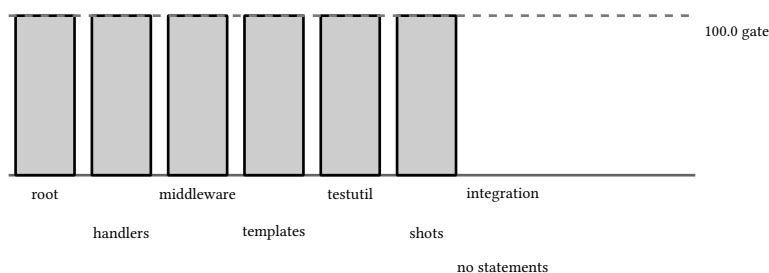


figure 17: the measured coverage shape, every package with statements at exactly 100.0, the gate line at the total

What does 100% global test coverage mean exactly?

On my resume that wording names the tree it was written against, so I will give you the arc first: 100% global held before the Python service landed, dipped to 99.6 while it sat untested, and reads true again at the current head, 100.0% total Go statement coverage, an atomic covermode run on go1.27.1, every package with statements at exactly 100.0, and a CI gate that fails below 100. The measurement command is the repo's own and runnable during the call. Red, green, refactor on new logic, and the coverage is where the discipline held.

floor: book 3, ch 14 (go 1.27, testing), book 5, ch 17 (c# 15, f# 11, and .net 11, idioms)

100% coverage is a vanity metric.

Agree in general, coverage is an input and assertions prove things. The claim is the discipline of a CI gate that fails below 100.0 on 42k code lines, every package with statements at the ceiling, not bug freedom. If the bar were cheap it would not be evidence of anything.

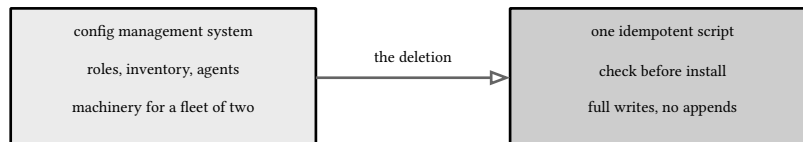
3.9. KISS

Nobody attacks KISS directly. The risk is hypocrisy, and the drill is a real deletion.

Tell me about a time you simplified existing code.

The dotfiles bootstrap is one idempotent script, not Ansible, because the constraint set, two operating systems and no fleet, does not earn a config management system. Too simple versus too general: generalize for the second real use case, never for an imagined third.

floor: book 11, ch 1 (design patterns, concurrency, and distributed systems, pattern discipline)



generalize at the second real use case, never the imagined third

figure 18: kiss as a real deletion, one script where a system would go

NOTE: the tag row as a whole

A hostile reader can call the row a buzzword strip. The single response: each tag is backed by either a public repo or a full handbook in this series, name the floor and offer the depth probe. The tags are ordered systems first, then languages scoped to API work, then the storage pair, then thinking styles, domain claims, and practice rules, and that order is the thesis, not decoration.

4. contributions defense

Three entries, thirteen numbers, every one self reported and every one checkable, and checked: the 2026-09-10 gh evidence pass measured every row. Most verify exact, the 40+ tools number holds only under the all-installables reading, and the gist stars plus the size and coverage pair came back false against today's live artifacts (coverage/gh/gist-stars.md, coverage/gh/llm-tournament-measurements.md). This is the most verifiable part of the resume and where a hostile interviewer spends the most time, because numbers on personal projects are exactly where inflation lives. The pattern for the rows that hold: define the measurement, show the live command, offer the walkthrough. The pattern for the two historical rows: state which artifact the line was true of and what happened to it, give today's measured numbers, offer the repo. Full reasoning in volume 0, ch 3 (resume defense analysis, contributions), register rows contrib-* map here.

4.1. the numbers, one table

The complete list of self reported numbers with what the 2026-09-10 measurement returned. An interviewer who checks one and finds it exact stops checking, which is why the two false rows get conceded out loud before anyone else runs the count.

number	where	how it is checked
35 stars	dotfiles	repo page, measured exact 2026-10-05
20 language servers	dotfiles	init.lua list, measured exact 2026-10-05
30 Tree-sitter parsers	dotfiles	init.lua list, measured exact 2026-10-05
40+ CLI tools	dotfiles	all 46 packages.yaml installables, cli_tools alone 18
20 languages	dotfiles	the LSP count restated, concede if probed
100+ stars	modern-swe-library	historical: gist deleted at archive, today repo 9 gist 0, coverage/gh/gist-stars.md
2,600-hour path	modern-swe-library	sum of per item estimates
8 stars	llm-tournament	repo page, measured exact 2026-10-05
100k SLOC	llm-tournament	historical: measured 42,059 code of 49,780 total, tokei 15.0.0
100% global coverage	llm-tournament	measured 100.0% Go, CI gate enforces it, coverage/gh/llm-tournament-measurements.md
since 2023	dotfiles	repo created 2023-06-02, commit history, measured exact
since 2023	library	historical: names the pre-gist era, no surviving artifact predates 2024-08, coverage/gh/gist-stars.md
archived 1/2025	library	repo timestamps

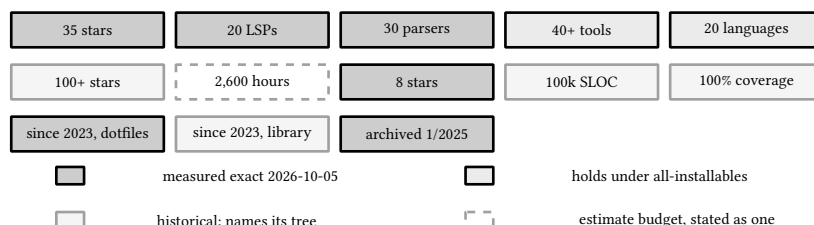


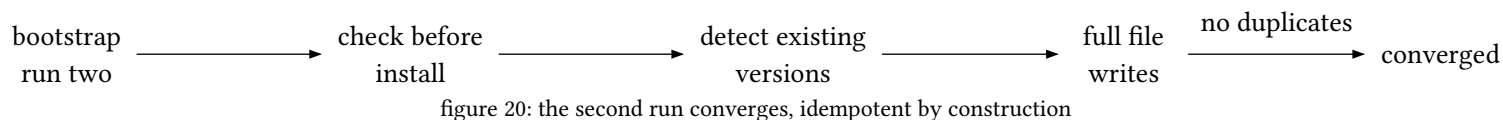
figure 19: the thirteen self reported numbers by verdict, every one checkable

4.2. dotfiles

The play is never to defend 35 stars as impressive. It is to defend the number as real and let the content carry weight: most dotfiles repos are a pile of symlinks, this one is a bootstrap you can actually run.

What does your bootstrap do on the second run?

It converges instead of duplicating: package managers check before install, tool installs detect existing versions, config writes full file writes, not appends. A half configured machine is the normal case, the script forces state toward the target. One entry point, platform branches per side, because the package managers differ.



Name ten of the language servers you provision.

rust-analyzer, gopls, csharp-ls, typescript-language-server, lua-language-server, pyright, clangd, then the markup and config side, bash-language-server, toml-ls, yaml-ls. The counts come straight from the provisioning lists, each one countable during the call.

What is the Claude Code integration, concretely?

Project rules and hooks wired into the repo so the assistant respects the same lint and test gates I do. It does not write my config, the boundary is that tooling enforces standards on humans and assistants alike.

4.3. modern-swe-library

Register row contrib-swe sits at framing as a historical claim: the 100+ stars were on the gist the library lived on, and that gist was deleted when the project was archived to GitHub in 1/2025. The 2026-09-10 pass and the 2026-10-05 re-check are consistent with that: both enumerated all 32 public gists under lavantien and found no library gist, and the old-version gist the repo readme links, under still-lake, measures 0 stars and 2 forks, a different, older artifact. The drill states the history and gives today's numbers.

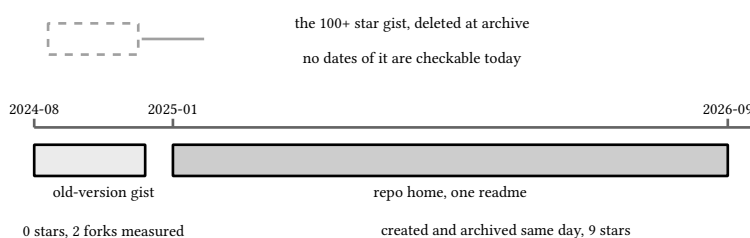


figure 21: the two homes of the library, the gist era then the repo, stars measured on both

100+ stars where, exactly? Your repo shows fewer.

On the gist the library lived on before the archive, and that gist was deleted in January 2025 when the project moved to GitHub, so there is nothing to check today and I will give you today's numbers instead: measured on October 5, the repo has 9 stars and 4 forks, and the old-version gist it links has zero stars and two forks. I enumerated every gist under both accounts to be sure. The library itself is real and public, one curated markdown document with a 2,600 hour path.

Where does 2,600 hours come from?

It is the sum of per item estimates across the whole path, ordered so dependencies come first. An estimate budget, not a stopwatch log, and I say exactly that. If you have 500 hours: keep the dev environment section, the data structures core, and interview questions for your target stack, cut breadth.

floor: book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)

You wrote “no AI slop”. How is a reader supposed to verify that?

Manually composed and curated, every entry read and placed by me, one document with navigation that holds it together. The claim is about composition method, and the document predates the slop era in style.

4.4. llm-tournament

Three numbers, zero bullets on the resume, deliberately: it is a conversation starter, and the 30 second description comes first. Register row contrib-llm sits at framing as a historical claim: the stars measured exact, the size number names the tree the line was written against, before python_service and its untested 2,328 lines landed and were later removed, and the coverage number reads true again at the current head, so those drills state the arc and give today’s measurements.

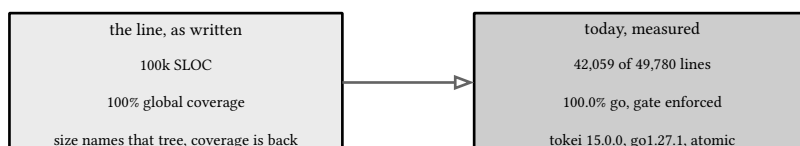


figure 22: the line names the tree it was written against, today’s measurements beside it

What is llm-tournament and why does it exist?

A local-first benchmarking arena where LLM models run against prompt suites and I score them by hand, results grid, profiles, and analytics, fully offline with no api keys. It exists because I wanted a large codebase where the discipline never slips, every feature test first, coverage held at the ceiling behind a CI gate. Its job was to make the TDD tag on my resume a fact instead of an adjective.

floor: book 3, ch 14 (go 1.27, testing), book 5, ch 17 (c# 15, f# 11, and .net 11, idioms)

100% coverage on 100k lines, how is that measured and who audits it?

The line names the tree it was written against, so here is the audit of the arc. Then: 100% global held before the Python service landed, and the 2026-09-10 pass measured the dip, 99.6 with python untested. Now, measured on main with token: 42,059 code lines, not 100k. The repo’s own test command under atomic covermode on go1.27.1: 100.0% total Go statement coverage, every package with statements at exactly 100.0, the badge says 100.0, and make check ends with coverage-enforce, so CI fails below 100, the Python service that moved it off 100 was removed at the 2026-09-30 head. You can run any of this during the call. Coverage does not prove correctness, assertions do, and the project holds the bar on assertions too.

Eight stars. Was it worth 100k lines?

Worth 42k code lines, the measured count, and yes. It is a personal discipline project, I never promoted it, and its audience is me and any interviewer. Popularity was never the goal, the number that matters is held by the repo, not the star count.

VERIFY: live evidence commands

The strongest move is the same for all three repos: open the repo during the interview and run the count. Star counts are on the page, coverage is one command, the provisioning lists are plain files. Rehearse locating each number from the table above in under thirty seconds.

5. employment defense

Four employers, thirteen bullets, and one standing rule for all of them: client and outsourcing work means NDAs cover the interesting specifics. Say “client confidential” once, calmly, then pivot to what is public, the problem class, the tech decisions and why, and the demo repo where one exists. Full reasoning in volume 0, ch 4 (resume defense analysis, employment claims), register rows emp-* map here.

5.1. Dikshatek

The frame lands first: three engagements across two modes is a referral pattern, not a retention problem.

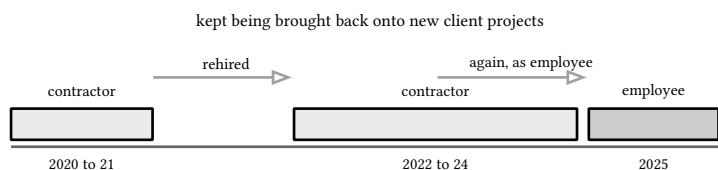


figure 23: three engagements across two modes, read as repeat clients

You have three separate periods at Dikshatek. Explain that.

Three engagements: contractor 2020 to 2021, contractor 2022 to 2024, employee 2025. They kept bringing me back onto new client projects, which I read as the useful signal, repeat clients. In 2025 I joined as an employee for backend work on client systems and left at the end of that engagement as planned.

Walk me through your SQLite setup: journal mode, busy timeout, backup strategy.

WAL mode for reader and writer concurrency, busy timeout for write contention, atomic file copy backups. The access pattern was key value with a small indexed surface on customer boxes that do not earn a database server process: no network hop, no separate ops. It becomes wrong when concurrent writers multiply or multiple processes need the same base, and the ones that outgrew it moved to Postgres, I did exactly that where growth demanded it.

floor: book 9, ch 44 (practical data structures and algorithms, capstone: storage engine toolkit), book 5, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics), book 12, ch 10 (infrastructure: docker, databases, and testing, sqlite in production: embedded kvs and game saves)

How do you prove a rewrite is behaviorally identical?

Behavior first, four moves. One, capture the legacy service’s outputs on recorded requests as golden masters. Two, port. Three, run both and diff. Four, decide each diff explicitly with the client, keep or fix, then cut over with the old service kept warm for rollback. The diffs were usually undocumented behavior, which is what the capture step exists to surface. What tests missed was mostly performance shape and timing dependent behavior, which is why the shadow period mattered.

floor: book 11, ch 1 (design patterns, concurrency, and distributed systems, pattern discipline), book 3, ch 14 (go 1.27, testing)

What fails first when a construction site loses uplink for a day?

Nothing downstream of the site. Agents keep collecting, buffer locally, and reconcile on reconnect with idempotent writes, so a day of outage costs latency, not data. Sites with no direct uplink relay through peers, that is the mesh part, and southbound is the leg toward the devices versus northbound toward the cloud. The demo repo, go-edge-mesh, is public, you can read the transport and reconciliation code during the call.

floor: book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals), book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), book 3, ch 6 (go 1.27, concurrency)

5.2. NTQ Solution

Ten months, one mobile bullet, one substantive backend. The mobile bullet is scoped before it is attacked.

What did you actually build on the Flutter side?

Features across the app, screens and flows wired to the backend, not platform or rendering work, and I am explicit about that. The substantive backend work was the order processing services. The provider integration work taught the real lesson: sandboxes drift from production behavior, so we pinned contracts with tests and treated sandbox pass as necessary, not sufficient.

Your aggregation pipeline got slow. Walk me through your diagnosis.

Explain first: check index coverage of match and sort, keep documents small, watch memory limits on large stages, and push match before lookup everywhere it can move. The pipelines were account statements for account services, and that diagnosis is the optimization story.

floor: book 3, ch 13 (go 1.27, stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io, testing), book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), book 5, ch 16 (c# 15, f# 11, and .net 11, stdlib tour 2: io, text, net, channels, diagnostics)

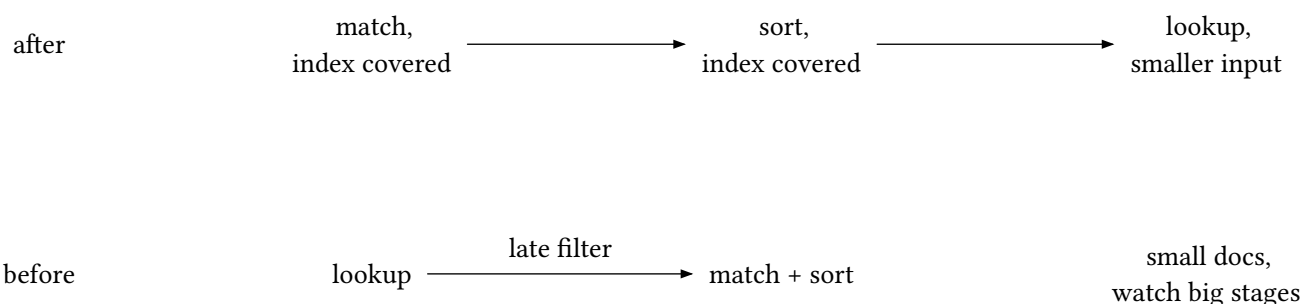


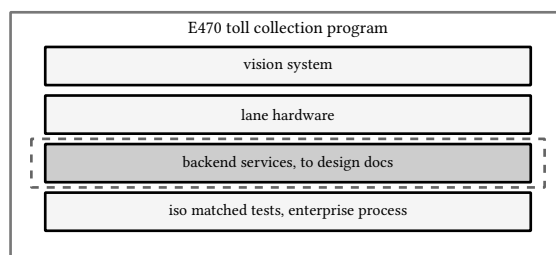
figure 24: the aggregation diagnosis, match pushed before lookup, indexes covering match and sort

Protobuf: you removed a field and broke production. What went wrong?

Field numbers are the contract, not names. Removed fields must be reserved, never renumbered, never reused, and new fields go in optional with defaults for forward compatibility. On order creation specifically, retries are made safe by client generated idempotency keys with a uniqueness constraint, the state machine enforces transitions in one place.

5.3. FPT Software

A named public system, five months, one module. Scope honesty arrives before the basking temptation does.



dashed box: the module owned, five months, to spec

figure 25: one module inside e470, the scope box around what was owned

What exactly was yours on E470?

One developer inside a large FPT team on the toll collection program, five months, backend services to detailed design documents, to spec. Not the vision system, not the lane hardware. What five months buys is a real system and enterprise process, and I say exactly that.

Avro schema evolution: walk me through a backward incompatible change and what happens to consumers.

With a schema registry enforcing compatibility, an incompatible producer cannot ship, that is the point of the registry. Compatible changes only: new optional fields with defaults, never rename. Avro over JSON for schema enforcement and payload size on high volume topics, because the pipeline consumed typed events across teams.

floor: book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency), book 11, ch 14 (design patterns, concurrency, and distributed systems, observability)

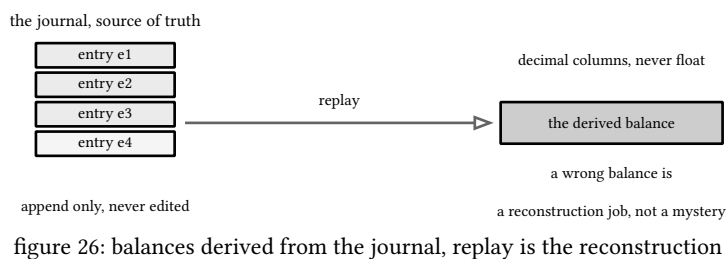
5.4. PALTech

First job, fintech, fixed more than designed. The money math answer is the whole game, get it right first.

Exchange platform: where did you store prices and balances and why not floating point?

Decimal columns, never float. Balances derived from journaled entries, so a wrong balance is a reconstruction job, not a mystery: replay the journal. A client reports a wrong balance, you investigate by reconstructing from entries and finding where the divergence entered, which is also why audit logs are not optional in that domain.

floor: book 5, ch 3 (c# 15, f# 11, and .net 11, the type system), book 11, ch 15 (design patterns, concurrency, and distributed systems, resilience)



Show me FlowerShop. What is it and what does it prove?

An Angular shop game, the public demo of the frontend period, it proves the frontend bullet is real. Small claim, small answer. The first job was fixing more than designing, that is what a first job is for.

NOTE: the discipline bullet

The flattest Dikshatek bullet, tests, handlers, middlewares, business logic, gets one prepared line: it is the discipline bullet that pairs with the TDD tag, the ordinary craft stated under the interesting systems. Middleware order on a typical Go API: logging, recovery, auth, then handler, the chain explicit and testable end to end.

6. gap and chronology narratives

Five gaps separate six employment runs, and one runs to the present. Interviewers see the same timeline, so every gap gets one or two sentences prepared, without apology and without invention. Register rows gap-* map here, full reasoning in volume 0, ch 5 (resume defense analysis, chronology and gaps).

WARNING: narratives must be trued up

These narratives are drafted from public facts and calendar logic, contract cycles, the degree start date, the COVID period. Replace any sentence that misstates the actual circumstances before rehearsing. A defensible narrative is true, short, and moves the conversation forward.

6.1. the timeline, one table

period	length	what it is
3/2020 to 7/2020	4 months	pandemic quarter, PALTech to FPT
11/2020 to 12/2020	1 month	transition, FPT to Dikshatek
12/2021 to 1/2022	1 month	transition, Dikshatek to NTQ
10/2022 to 12/2022	2 months	client return, NTQ to Dikshatek
12/2025 to present	9 months	degree plus build period, by design

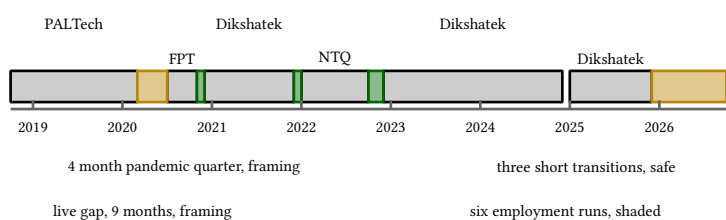


figure 27: six runs and five gaps on one strip, the gaps severity colored

The two one month stretches are transitions measured in weeks, and the point is their size. Each gets one sentence, delivered flat.

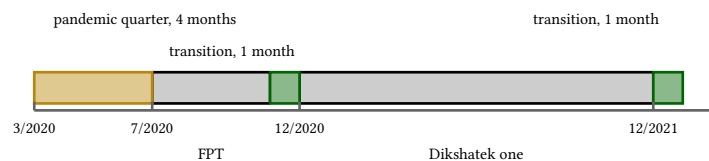
6.2. the transitions

Why four months between PALTech and FPT?

PALTech ended as the pandemic hit. Vietnam locked down in the spring of 2020 and hiring froze everywhere, so I used the quarter to study and build, and came back to the market when openings resumed, landing at FPT in July.

The NTQ role ended after ten months. Why?

The engagement ran its course in October 2022, and Dikshatek, where I had already contracted in 2020 and 2021, brought me back in December for a longer two year run. The pattern to read is that previous clients rehired me, twice.



the two one month bands are the point: transitions measured in weeks

figure 28: the transitions zoomed, two one month bands and the pandemic quarter

6.3. the live gap

The only gap an interviewer will press hard on, because it has no end date on the resume. The narrative is a plan, not a gap, and the output of the period is specific and nameable.

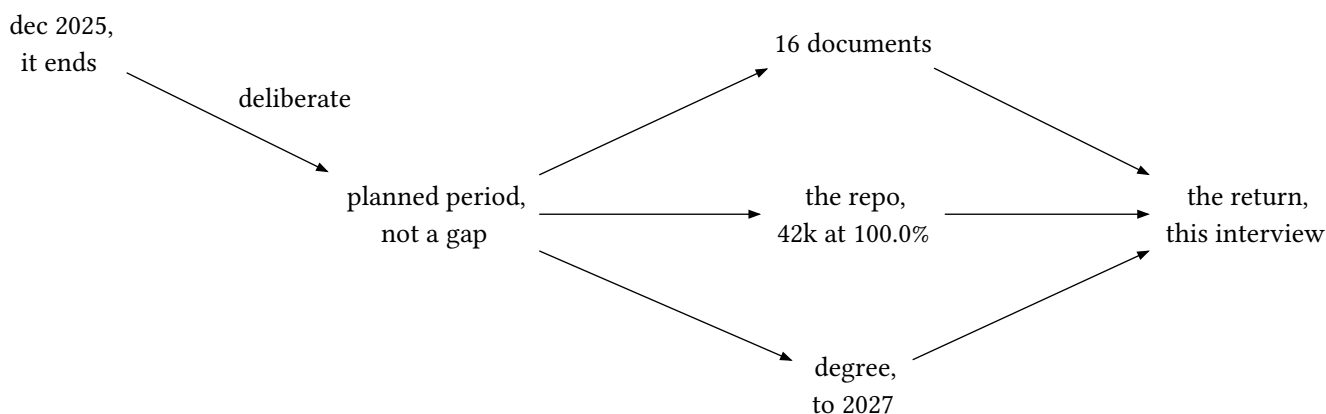


figure 29: the live gap as a plan, named outputs, a deliberate return

You have been off since December 2025. That is nine months. Why?

When the Dikshatek engagement ended in December 2025 I took a planned period to finish a stretch of my part time degree, which runs to 2027, and to build. The output is inspectable: the LLM tournament repo, 42k code lines held at 100.0% Go coverage by the 2026-10-05 measurement, dates from this period, and a seventeen document series stands behind the resume, the volume 0 claim analysis, a systems manual, six language manuals and a mathematics manual, six engineering handbooks, five with tested, runnable capstones, including the six language data structures volume and the icpc world finals book, and two rehearsal books. I am returning to work deliberately, this interview is the start of that.

floor: volume 0, ch 6 (resume defense analysis, education timeline), volume 0, ch 3 (resume defense analysis, contributions)

Why should I hire someone who has been out for nine months?

Because the period was production, not absence. The degree advanced, the repos are public, and the handbooks mean every depth probe you fire today lands on something I can teach back at chapter length. Ask me anything from them right now, that is the return to work demo.

How are you supporting yourself?

Savings and contract savings from the Dikshatek years. It is not a technical question, and then, happily, back to work.

6.4. the pattern, stated once

Map your whole timeline for me.

Eight years since the first job, four employers, one of them bringing me back twice, every transition measured in weeks, one pandemic quarter, and one deliberate build year aligned with the degree. The timeline is a record of repeat clients, that is what the resume actually shows.

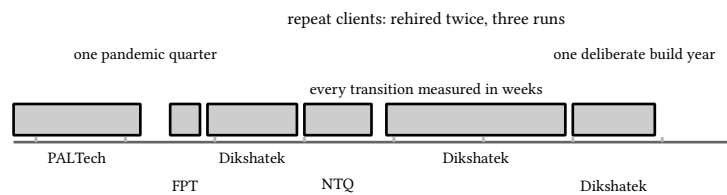


figure 30: the eight year pattern, repeat clients, week-scale transitions, one build year

NOTE: the overthinking trap

Long explanations of gaps smell like insecurity. Three sentences maximum, then stop and let the next question come. The gap answers are the shortest drills in this book on purpose.

7. education and participations answers

One education line, two competition entries, and the self taught then formalized arc delivered as an arc, in that order, before the arithmetic question lands. Register rows edu-* map here, full reasoning in volume 0, ch 6 (resume defense analysis, education timeline).

7.1. the degree

You have been working since 2018 and you are only in a BSc now? Explain the timeline.

I entered the industry in 2018 self taught, through competitions and then client work, competitions first, ICPC qualifier 2015, the sumo championship 2017, work followed. In 2024 I started the accredited online BSc with UIT deliberately, part time, concurrent with full time work, because formal credentials open doors that demonstrated skill does not, visa processes and enterprise HR filters among them. It runs to 2027.

Online degree, how rigorous is it really?

It is a University of Information Technology affiliated program delivered online and accredited, and the coursework is real and current, I can tell you exactly what I am taking this term and one thing from it that changed my practice. The degree is a formalization pass over six years of shipped work, not an entry ticket.

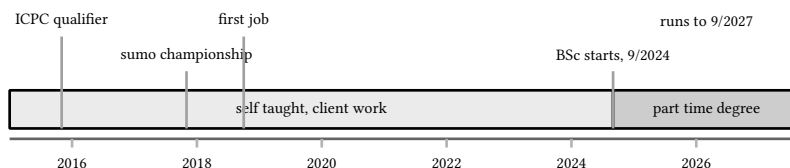
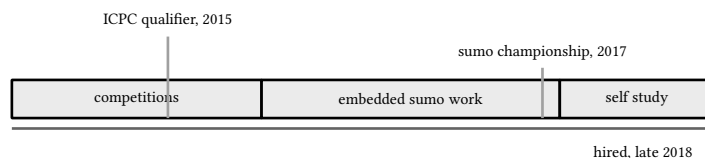


figure 31: self taught then formalized, competitions, work, then the 2024 degree

7.2. the 2015 to 2018 window

WARNING: true this up before rehearsing

The window between the 2015 qualifier and the 2018 first job must be filled with what actually happened, education fragments, self study, early freelancing or support work. The drill below is the shape, not the content. A scattered truth said with shape beats a polished invention.



the drill fills this shape with what actually happened

figure 32: the pre career window as a shape, competitions, embedded work, self study

What did you do between 2015 and 2018 before PALTech?

The student era was competitions and self study: the ICPC qualifier in 2015, then embedded work for the sumo robot championship in 2017, and self taught backend development until PALTech hired me in late 2018.

7.3. sumo championship

Fully auditable, public firmware, and one whiteboard almost guaranteed: PID. The layering answer comes first, why fuzzy and PID together.

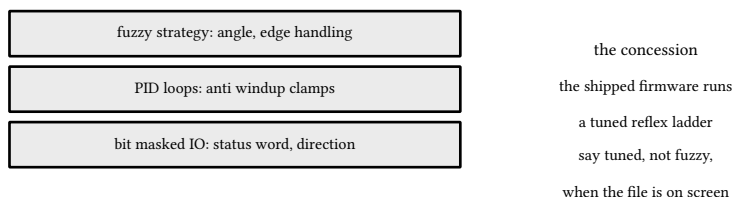


figure 33: the layered control story and its concession, tuned not fuzzy on the chip

Explain your sumo robot's control architecture. Why fuzzy and PID together?

Different control layers. A fuzzy layer for strategy, attack angle and edge handling with membership functions over distance and position, and PID loops closing the loop on motor output with anti windup clamps. Bit masking ran the sensor status word and direction control, checking and setting bits in one register rather than a pile of booleans. The shipped file runs a tuned reflex ladder, the fuzzy and PID language names the tuning work around it, and the firmware is open source, the link is on the resume.

floor: book 14, ch 5 (game systems and architecture, finite state machines), book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)

What is integral windup and how did you handle it?

The integral term accumulates while the actuator is saturated, so the response overshoots long after the error changes sign. Handling is clamping the integral and resetting it on saturation. Derivative on measurement, not on error, avoids the kick on setpoint change, same era, same lessons.

That was 2017. What have you done in embedded since?

Nothing professional, the career went backend, and the habits carried: bitwise state and control loops are the same discipline in a game loop or a data pipeline. The honest pivot is that the current strengths are on the rest of the resume.

7.4. ICPC qualifier

Participant, not medalist, stated plainly, never upgraded verbally.

How far did you get in ICPC?

Participant, the 2015 VNU Ho Chi Minh qualifier round. I did not advance, and the resume says participant because that is what it was. What stuck was the training habit, that period is why data structures and algorithms stayed sharp, and that foundation is now a full handbook in this series.

floor: book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking), book 9, ch 10 (practical data structures and algorithms, graphs)

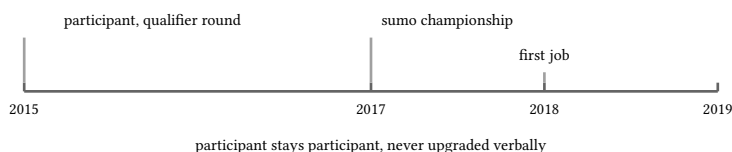


figure 34: the qualifier as origin evidence, participant, stated plainly

NOTE: participations strategy

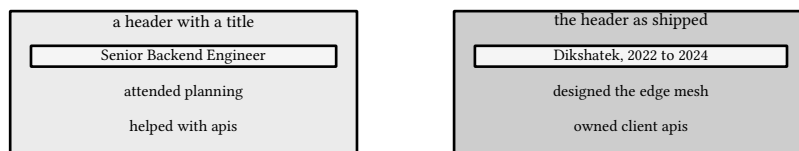
Both entries are old, 2015 and 2017. Their job is origin evidence, proof of competitive and systems thinking before the career started. Defend them with specifics when asked, never lead with them. The recent work and the repos are the headline, the competitions are the prologue.

8. no titles and no metrics policy answers

Two deliberate absences run through the whole resume: no job titles on any employment header, and no impact metrics in any employment bullet. Both will be noticed, both are policy, and both defenses are delivered as a professional standard rather than an evasion. Register rows no-* map here, full reasoning in volume 0, ch 7 (resume defense analysis, missing job titles) and volume 0, ch 8 (resume defense analysis, the no metrics policy).

8.1. no titles

The reason is ownable in one breath, and the follow up has a concrete answer waiting.



the defense holds only because the verbs carry scope

figure 35: no titles holds because the bullets carry scope verbs

Why does your resume have no job titles?

The titles genuinely varied and outsourcing titles are not comparable across companies, a senior at one outsourcing house is a mid somewhere else, so I let the work speak. The bullets carry scope verbs, designed, built, wrote, owned, and the level question answers itself from scope of responsibility.

floor: volume 0, ch 2 (resume defense analysis, header and specialization tags), volume 0, ch 4 (resume defense analysis, employment claims)

What was your title at each of these companies?

Informally: at PALTech the junior backend fixer, at FPT a developer on the toll system services, at NTQ a Go backend developer on order processing, and across the Dikshatek engagements a backend developer who owned client APIs end to end and owned the edge mesh design in the later run. Whether that counts as senior is your call, the scope I just described is the input.

Were you ever a lead?

On the edge mesh design, yes, in the sense that matters: I owned the architecture and the client conversations behind it. The title word that went with it internally is not on the resume because it carries no signal across companies.

NOTE: why this defense holds

Removing titles only works because the bullets carry scope. If the bullets were passive participation verbs, missing titles would be fatal. The defense is the verbs.

8.2. no metrics

The absence is stated policy from the repo README, “no made-up numbers or hard-to-defend claims”, and the numbers that do appear all live in the contributions section where they are public and checkable.

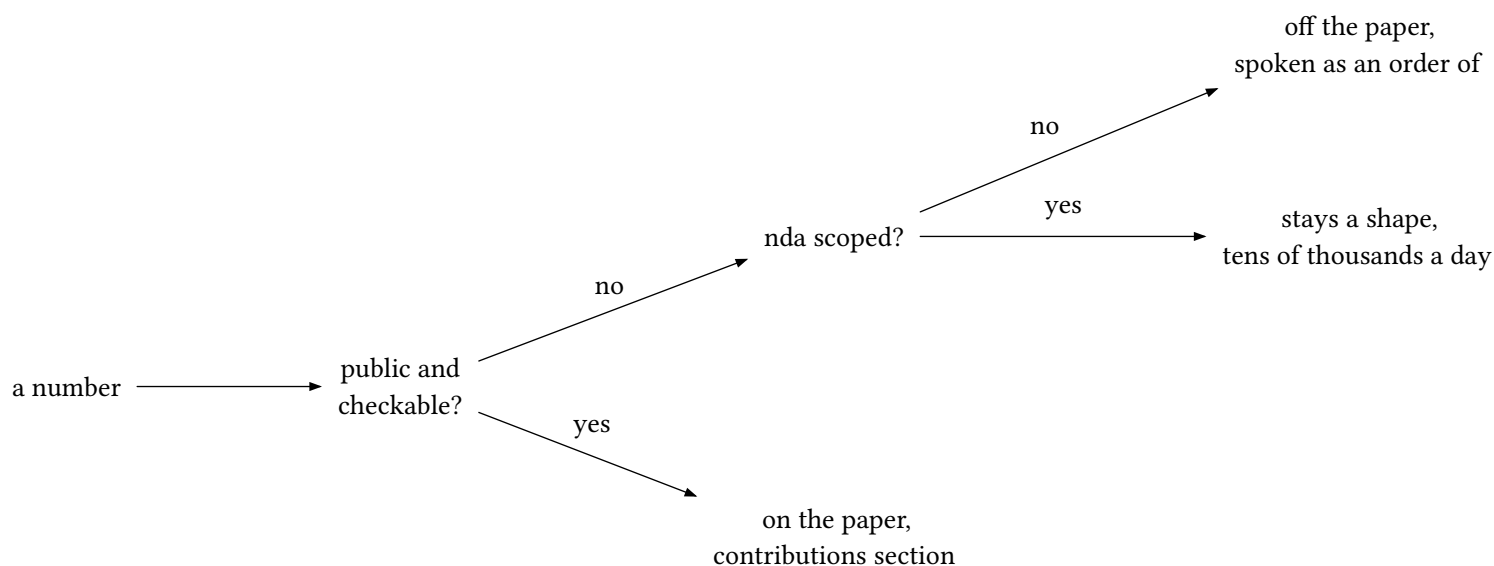


figure 36: where a number may live, the policy as one tree

Your resume has no metrics. How do I know you made an impact?

It is policy, not omission. Client work lives under NDAs, and a number I cannot source in an interview is a liability, so the resume carries only numbers that are public and checkable, the repos in the contributions section. For impact, here is what I can source: I owned client APIs end to end and designed the edge mesh, and I can walk the design decisions and trade offs in as much depth as you want.

floor: volume 0, ch 3 (resume defense analysis, contributions), book 11, ch 14 (design patterns, concurrency, and distributed systems, observability)

Give me one measurable result from your last job.

From my own systems, take the edge mesh design constraint that mattered: a day of site uplink loss must cost latency, not data, and the buffered reconcile design held that. If you want numbers with digits, my repos have them, star counts, line counts, coverage, all verifiable right now.

8.3. verbal number rules

The policy bans unsourced numbers on paper, not all numbers in speech. When a real measured number exists it can be spoken under three rules:

- Only numbers you can reconstruct the measurement for. Asked how you know, the answer must exist.
- Round down when uncertain, and say “on the order of”.
- NDA sensitive figures stay as shapes: “tens of thousands of transactions a day”, not the exact internal dashboard figure.

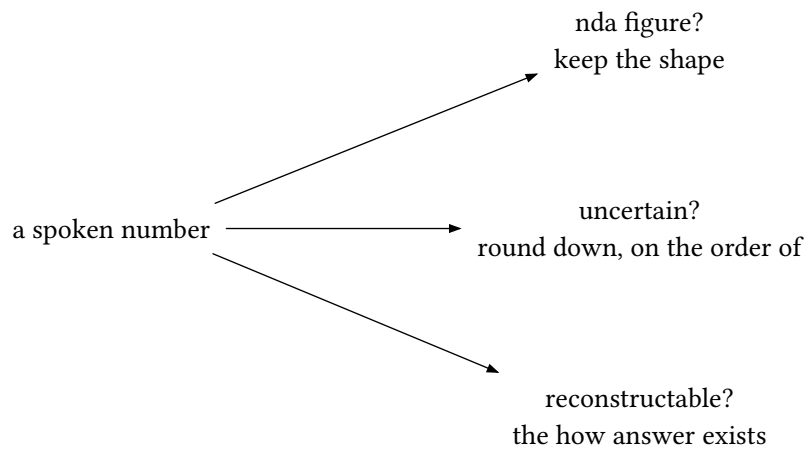


figure 37: the three speech rules as leaves off one root

VERIFY: the prepared line

Rehearse until reflex: “I do not put numbers on my resume I cannot prove in this room, so let me show you the ones I can.” Then open a repo. That pivot, from absence to evidence, is the whole chapter in one move. Do not fill the awkward beat after “no numbers?” with an invented one, the whole defense collapses there.

9. hostile cross examination playbook

Everything before this chapter defends one section of the resume. This chapter is the interview itself as a system: the modes of questioning, the rules that hold under pressure, the map from claim to the whiteboard most likely to follow it, and the two hobby lines that close the resume. volume 0, ch 1 (resume defense analysis, method) states the full threat model. The exchange theory block at chapters 12 through 17 sits under this one: the modes name what is firing, the logic and counter chapters say what to do about it.

9.1. the three modes

Verification checks that a claim is real: show the repo, run the count. Depth probes accept the claim and test the floor under it: explain metatables, walk the mesh partition. Hostility implies inflation or fabrication: 8 stars next to a size number the 2026-09-10 evidence pass cut in half, the gap, the missing titles. The drills in this book are tagged implicitly by their chapters, evidence drills for verification, floor drills for depth, scripted answers for hostility. Know which mode is firing before answering, the same question in three modes gets three different answers.

row	verification	depth	hostility
checks	checks it is real	tests the floor	implies inflation
move	show the repo	teach it at length	concede, correct
drill	ch 4, ch 18 to 24	handbook floors	scripted answers

figure 38: one question, three modes, three different answers

9.2. the rules under pressure

- Concede scope first. “I have not run Raft in production” converts the attack into a demonstration before it lands.
- Evidence early. “It is on my GitHub, the link is on the resume” is a complete sentence.
- Never invent a number. One caught invention poisons every other number on the page.
- Correct a false number before the interviewer finishes checking it. State the measured value, name the measurement, offer the repo. Two resume lines from the 2026-09-10 pass turned out to be historical rather than present-tense, and their drills state the history and give today’s measured value in the same breath.
- Do not fill silence. Answer, stop, let the next question come. Pauses are the interrogator’s tool, not a gap you owe.
- NDA discipline: say “client confidential” once, calmly, pivot to the problem class and the public demo. Never break an NDA to win a point.
- Never upgrade a claim verbally. Participant stays participant, the informal titles stay informal, the scope stays scoped.

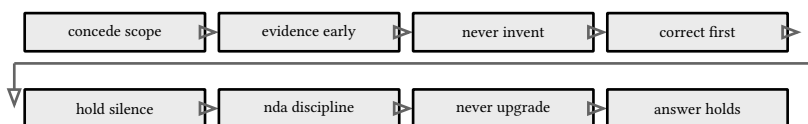


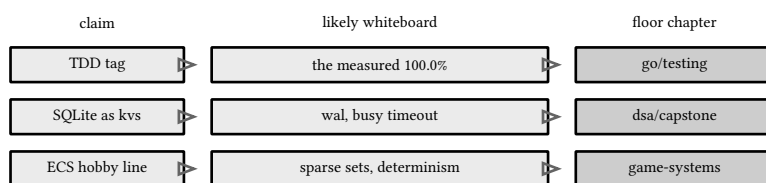
figure 39: the seven rules as one guard chain under fire

9.3. the deep probe map

Each row is the whiteboard most likely to follow the claim. The floor column is the chapter that holds the answer, and the answer should be taught, not recalled.

claim	likely whiteboard	floor
SQLite as embedded KVS	WAL, busy timeout, single writer, the Postgres pivot	book 9, ch 44 (practical data structures and algorithms, capstone: storage engine toolkit)

claim	likely whiteboard	floor
DistributedSystems tag	CAP, the partition-or-consistency tradeoff, mapped to the mesh, delivery semantics, idempotent reconcile	book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals)
No Raft in production	elections, log replication, why the mesh did not need it	book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft)
gRPC order processing	protobuf evolution rules, idempotency keys, the order state machine	book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency)
Kafka and Avro	schema registry compatibility modes, what breaks consumers	book 11, ch 13 (design patterns, concurrency, and distributed systems, messaging semantics and idempotency)
Fintech backend	decimal money, journal reconstruction of balances	book 5, ch 3 (c# 15, f# 11, and .net 11, the type system)
Sumo firmware	PID terms, windup, fuzzy layers, the status word	book 14, ch 5 (game systems and architecture, finite state machines)
ECS hobby line	composition over inheritance, sparse set storage, determinism	book 14, ch 2 (game systems and architecture, ecs theory)
FunctionalProgramming tag	immutability by default, records as unions, LINQ as embedded DSL	book 5, ch 3 (c# 15, f# 11, and .net 11, the type system)
SQLite/DuckDB tag	why two engines, WAL versus duckdb's model, the deployment shape each fits	book 12, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine)
TDD tag	the measured 100.0% behind a CI gate, what it covers and omits, red green refactor on a bug fix	book 3, ch 14 (go 1.27, testing)
Linux tag	what happens when ls runs: PATH, execve, dynamic loader	volume 0, ch 3 (resume defense analysis, contributions)



12 rows in the table, drill the three matching the job description

figure 40: claim to whiteboard to floor, the routing pattern of the probe map

9.4. the hobbies lines

The register rows hobby-* map here because the second hobby line is a technical claim in disguise, and it is why book 14 exists.

You list four competitive games. Does that affect your availability?

They are systems games, that is the common thread, and I play like anyone watches a series. Availability is normal. The religious texts on the same line are the counterweight, older systems of thought than any tech stack, they keep me from mistaking the new for the important.

Explain ECS like I have only done OOP. Why not inheritance trees?

ECS is composition taken literally: entities are ids, components are plain data, systems are functions over queries, and behavior emerges from the pipeline instead of living in class trees. You give up vtable dispatch and get cache friendly batched iteration, which is why the same idea shows up in data oriented backend work. My implementation uses sparse set storage per component type, structural changes batched at frame boundaries, FSMs drive the game flow, and the artillery capstone runs deterministic simulation with seeded random and replay from an input log. The WC3 editor started it, triggers then JASS, which is also where Lua entered my life.

floor: book 14, ch 2 (game systems and architecture, ecs theory), book 14, ch 3 (game systems and architecture, an ecs implementation in c#), book 14, ch 5 (game systems and architecture, finite state machines), book 14, ch 19 (game systems and architecture, capstone part 1: the battle engine), book 8, ch 32 (lua 5.5, capstone: auto chess, human versus bot)

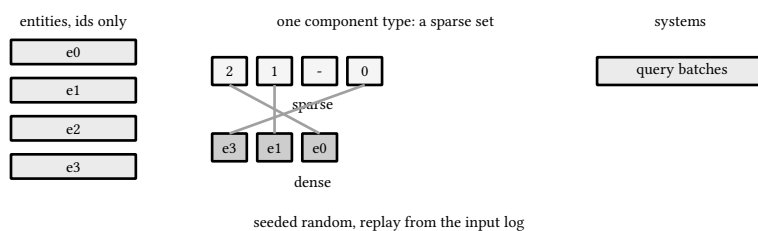


figure 41: ecs under the hobby line, ids, data, queries, sparse set storage

PITFALL: the “nice hobby” trap

“Why is game programming on a resume?” Answer: data oriented design is a backend skill wearing a game costume. Cache friendly layouts and batched transforms are the same discipline in a game loop and in a hot API path. A hobby line that opens into a real codebase is a strength, one that opens into nothing is a liability, and this one opens into a tested capstone.

9.5. the endgame

When the interrogation runs out of ammunition it turns general. Three closers, prepared:

Why should we not hire you?

If you need a specialist, one stack at scale, one domain for years, I am the wrong shape. I am broad by design, four languages, the systems underneath them, and I prove depth where it counts with the handbooks and the repos. If breadth with provable floors is the job, the fit is exact.

What is your weakness?

Distribution over promotion. I build things that are testable and documented, including for myself, and I underinvest in making noise about them. Eight stars on 42k code lines is that trait rendered as a number.

Any questions for us?

Ask one that uses the preparation: “What does the team’s test discipline look like, and where does it hurt?” The answer tells you the engineering culture, and the question tells them yours.

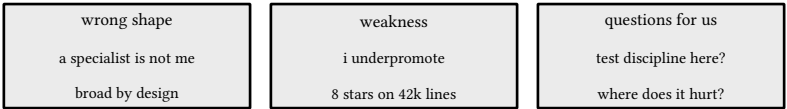


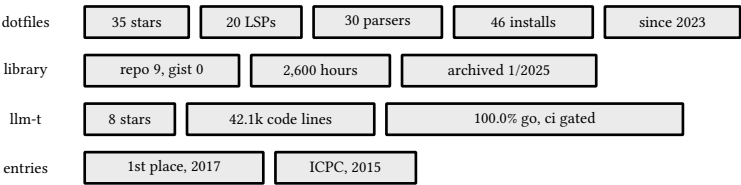
figure 42: three closers, prepared cards for the general turn

10. rapid fire fact sheet

The night before, read only this chapter. Every number and date the resume commits to, in one place, with where each one comes from, measured on 2026-09-10. Where the resume’s wording and the measurement differ, the value column is the measurement and the resume line carries a pending correction. If a live count has drifted since, say the live number.

10.1. the numbers

fact	value	source
dotfiles stars	35	repo page, live
dotfiles provisioning	20 LSPs, 30 parsers, 46 installables	provisioning lists, measured 2026-10-05
dotfiles start	2023	commit history
modern-swe-library stars	repo 9, gist 0	measured 2026-10-05, coverage/gh/gist-stars.md
study path length	2,600 hours	sum of per item estimates
library archive	1/2025	repo timestamps
llm-tournament stars	8	repo page
llm-tournament size	42.1k code lines	tokei, coverage/gh/llm-tournament-measurements.md
llm-tournament coverage	100.0% go, ci gated	measured 2026-10-05, coverage/gh/llm-tournament-measurements.md
sumobot result	1st place, zero loss, 2017	public firmware repo
ICPC	participant, 2015 VNU-HCM qualifier	resume, say no more



pins: repo pages live, coverage/gh snapshots and measurements, firmware repo

figure 43: the eleven facts as a card strip, grouped by source

10.2. the dates

period	value	one line
PALTech	10/2018 to 3/2020	first job, whitelabel exchanges
gap	4 months	pandemic quarter
FPT Software	7/2020 to 11/2020	E470 toll, one module, to spec
Dikshatek one	12/2020 to 12/2021	contractor
NTQ Solution	1/2022 to 10/2022	orders, gRPC, pipelines
gap	2 months	client return
Dikshatek two	12/2022 to 12/2024	contractor, edge mesh
Dikshatek three	1/2025 to 12/2025	employee
gap	9 months	degree plus build period, by design

period	value	one line
BSc	9/2024 to expected 9/2027	CITD-UIT, online, accredited, part time
career length	8 years	2018 to present, four employers

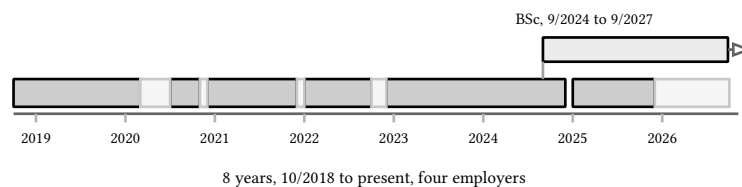


figure 44: the date strip, runs and gaps alternating, the degree overlaid

10.3. the series, one line

Sixteen documents: the volume 0 analysis, the systems manual for C23 and the cloud, the mathematics manual from trigonometry to monads in C23 with a deterministic physics game capstone, six complete language manuals for Go 1.27, Java 27, C# 15, F# 11, and .NET 11 with the F# chapters as a tour, JavaScript ES2026 with TypeScript 7 as its type layer, Python 3.14, and Lua 5.5, the algorithms pair with data structures and algorithms in seven languages and the icpc world finals book built from first principles in all seven languages, four applied handbooks with tested, runnable capstones, a storage engine toolkit, a Raft implementation, an infrastructure suite, a data mining pipeline over duckdb in go, and a deterministic artillery game, and two rehearsal books, this one and the interview repertoire. The systems manual and all six language manuals carry a service part building the same user rest api against the same frozen golden vectors, one lane per language under its own make gate. Book 6 stands at 6.1 with the react and svelte framework layer, books 3 and 8 at 6.0 with the htmx and templ chapters on book 3, books 5 and 8 at 5.0 with c interop and the allegro c host on book 8, books 1, 7, and 10 at 4.0, books 11 and 14 at 3.0, volume 0 and books 2, 12, and 13 at 2.0, the interview repertoire at 3.0 with the handbook floors part, and the cookbook itself at 4.0 with the reading order map, the service corpus floors, the exchange theory chapters, and the repo deep dives. Every sample and capstone suite runs green under one make verify with zero skipped tests, and every external claim in every chapter carries a pinned, dated source. When the gap question fires, this is the inspectable output of the period.

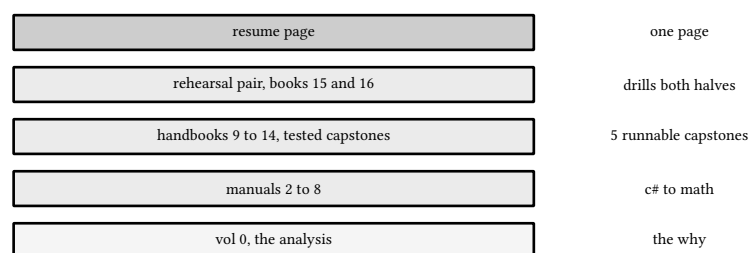


figure 45: one resume page standing on a seventeen document stack

10.4. twelve one line answers

1. Level? No title word, calibrate me off the repos.
2. Strongest language? C# and Go in production, TypeScript when the product needs it, Lua for embedded and tooling.
3. Why no titles? Outsourcing titles are not comparable, the bullets carry the scope.
4. Why no metrics? Policy, no number I cannot prove in this room, the provable ones are in the repos.
5. The gap? Degree stretch plus the handbook build, output is inspectable, this interview is the return.
6. Raft in production? No, and here is the whiteboard anyway.
7. 100% coverage? Held pre-python_service, dipped while the python service sat untested, 100.0% again at the current head with a CI gate, all numbers stated on request.
8. Eight stars? Personal discipline project, the number that matters is held by the repo.

9. SQLite as KVS? Deployment shape: no network hop, no DBA, WAL plus busy timeout, Postgres where it outgrew.
10. Degree in 2024? Formalization pass over six years of shipped work, concurrent, part time.
11. Why Lua? WC3 map making to the 5.5 manual, metatables are fair game.
12. What is llm-tournament? Models over prompt suites, manually scored, results grid and analytics, TDD made into a fact.

1 level: off the repos	4 no metrics: policy
7 coverage: 100.0 gated	10 degree: 2024 pass
2 c#, go in production	5 gap: inspectable output
8 stars: 8, held by repo	11 lua: wc3 to the 5.5 manual
3 no titles: scope	6 raft: no, whiteboard yes
9 sqlite: deployment shape	12 llm-t: tdd as fact

figure 46: the twelve line deck, the whole book compressed

VERIFY: the night before checklist

Open each repo once and locate its headline number. Re-read the gap answers aloud once, they are the shortest and must stay shortest. Skim the deep probe map and pick the three rows matching the job description. Sleep. The preparation is done, the drills are internalized or they are not, and no amount of last minute reading moves that.

11. the repo evidence method

Chapters 18 through 24 defend the seven public repositories the resume names, and every number in them comes from somewhere reproducible. This chapter is that somewhere: how the evidence was captured on 2026-09-10, where it is stored, and the rules that keep the deep dives honest. When an interviewer asks “how do you know that”, the answer is always a command that was run, a date it was run on, and a file in this repo holding the raw response.

11.1. the snapshots

Each repo has one snapshot file, `coverage/gh/<repo>.json`, holding three raw gh api response bodies verbatim, delimited by `// == <request> ==` header lines:

```
gh api repos/lavantien/<repo>
gh api repos/lavantien/<repo>/languages
gh api "repos/lavantien/<repo>/commits?per_page=5"
```

fragment, illustrative, not executed

Captured with gh 2.100.0 authenticated as lavantien, method and headline table recorded in `coverage/gh/capture.md`. The stars, forks, created and pushed dates, license, and archived flag in every fact table of chapters 18 through 24 are read off these files, not off memory of the repo pages. Live counts drift, that is what live means, so each table row carries measured 2026-09-10, snapshot `coverage/gh/<repo>.json` and the drill is to say the live number if it has moved since.

The languages endpoint gives bytes per language, which is why the fact tables quote bytes, not percentages. One trap the capture notes: some repos return null for topics and license, and modern-swe-library returns an empty object for languages because it contains a single markdown file.

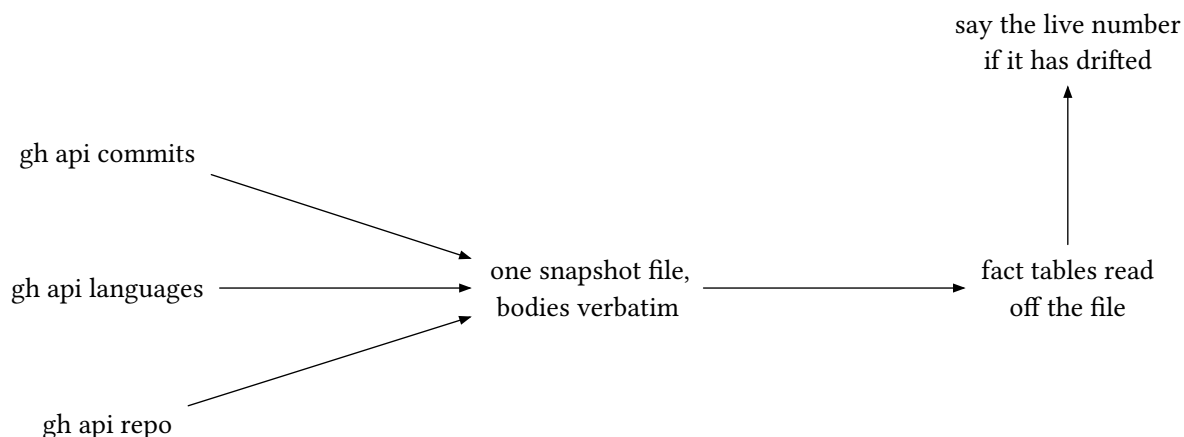


figure 47: three calls, one verbatim snapshot, the facts read off the file

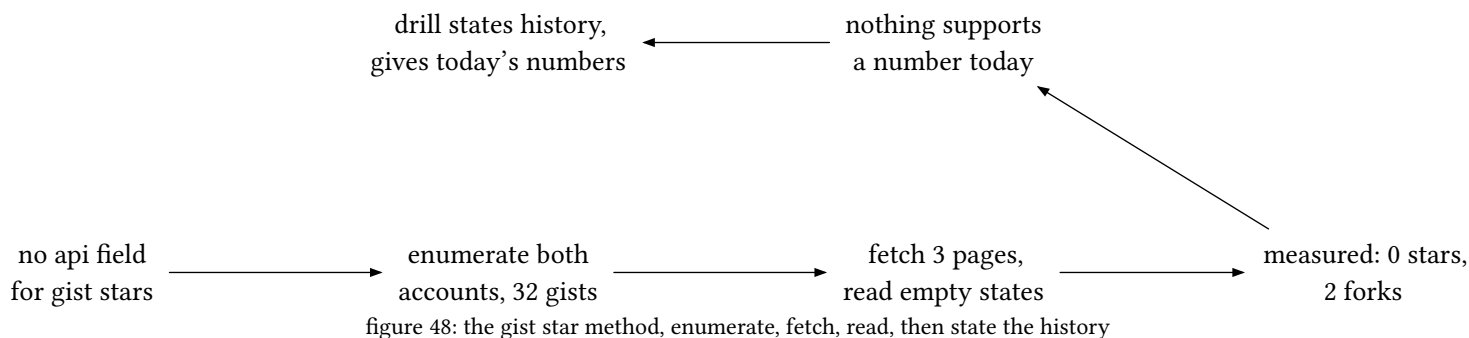
11.2. the two numbers the api cannot give

Star counts for repos come straight from the snapshot. Star counts for gists do not, because the gist REST API exposes no star field at all, unlike `stargazers_count` for repositories. The modern-swe-library claim says “100+ stars on gist”, so verifying it needed a different method, recorded in `coverage/gh/gist-stars.md`:

1. Enumerate every gist under both accounts, `gh api "users/lavantien/gists?per_page=100"` and the same for still-lake, to prove no other candidate exists. The readme’s own line 9 points at the old version gist.

2. Fetch the three gist pages over HTTP and read them: the gist page, where the star control renders bare with no count, the /stargazers page, which shows the empty state “Be the first” and zero user links, and the /forks page, one fork at the 2026-09-10 scrape, two at the 2026-10-05 re-check, the second confirmed against the gists forks api.

Measured result: 0 stars, 1 fork on 2026-09-10, 2 forks on 2026-10-05. A star count that has vanished since cannot be checked by any available method, and the owner states it vanished because the 100+ star gist was deleted at archive time, so the drill states that history and the measured present rather than defending a number nothing today supports.



11.3. the measurements that need a clone

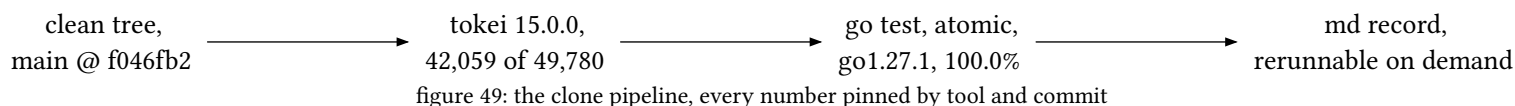
Two resume numbers describe the inside of llm-tournament, size and coverage, and neither can be read off an api. coverage/gh/llm-tournament-measurements.md records what ran, verbatim:

```

git clone --depth 1 https://github.com/lavantien/llm-tournament
tokei <clone>
CGO_ENABLED=1 go test ./... -coverprofile=cover.out -covermode=atomic -count=1
go tool cover -func
  
```

fragment, illustrative, not executed

tokei 15.0.0 on main at f046fb2, the repo’s own documented test invocation under atomic covermode on go1.27.1 windows/amd64, first measured 2026-09-10 on a deleted shallow clone at 01904fc, re-measured 2026-10-05 on the local clone verified clean at origin/main. The January clone lived outside this repo and was deleted after measuring, so the numbers are pinned by the markdown record, not by a checkable working copy. That is a scope limit stated out loud in chapter 14: anyone can rerun it, nobody can diff this repo against a tree it does not hold.



11.4. the vendoring rules

A listing in this series is a verbatim slice of a real file, enforced at two levels. tools/check-listings.sh fails the build if a listing() path points at nothing, and the listing() helper itself panics on a bad line range. External repo code cannot be listing material unless it lives under books/, so each deep dive vendors a small set of representative files under vendored/<repo>/, fetched by gh api repos/lavantien/<repo>/contents/<path>, relative paths preserved:

- one to three files per repo, an entry point, a config, or the firmware core, chosen to back the chapter’s walkthrough
- the set is evidence, not a mirror, whole repos stay on GitHub

- listings quote slices of the vendored file only, never retyped code, and the caption names the source path and line range
- syntax tags must resolve in typst 0.15.1, C# is cs never csharp, and PowerShell renders as plain text because this typst ships no grammar for it

Fragments that illustrate rather than quote use `snippet()` and are labeled illustrative, not executed. The distinction matters under hostility: a listing is a claim about someone's actual code, a snippet is a claim about an idea.

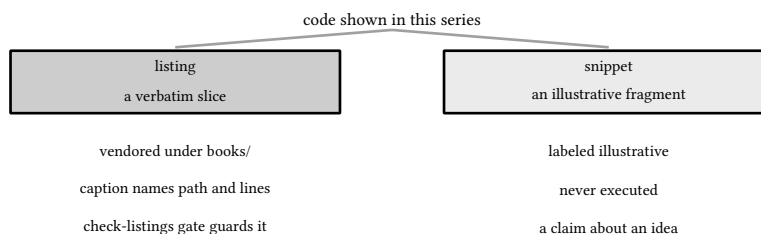


figure 50: listing against snippet, verbatim evidence against illustration

11.5. the provenance discipline

Every number in chapters 18 through 24 cites its snapshot file or its measurement record, in the table row or in the drill answer itself. The discipline has three parts:

- Numbers that came from an api response cite the snapshot, and carry the capture date, because they are the one kind of number guaranteed to drift.
- Numbers that came from a command cite the command, the tool version, and the commit measured, because "42,059 lines" without tokei 15.0.0 and main @ f046fb2 is a different, weaker claim.
- Numbers the evidence contradicted as present-tense readings are drilled as history plus measurement. The two regraded rows in the register, contrib-swe and contrib-llm, were caught by this method and regraded to framing once the owner stated the history.

The six service lanes ride the command-number discipline: each count cites its make target and its recorded date, 1130 checks under `make verify-capi` in the book 1 record, 314 in the python record, and the lua plain, jit, and host lanes at 188, 18, and 162 in the lua record.

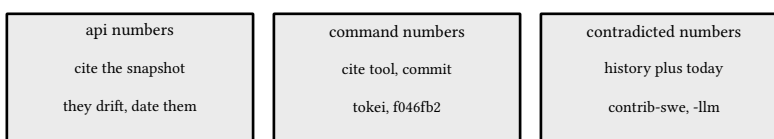


figure 51: three number kinds, three citation duties

11.6. the artifact resets the chain

A self-graded pass rate is a claim produced inside the same process that produced the work, so the chain of trust restarts at the physical artifact, never at the report about it. The artifact is the binary, the rendered page, the log, the extracted file on disk. When a tool or an agent reports work done, first ask which artifact on disk carries the claim, then open the artifact itself and read it against the claim. The same move already runs through this book, the measured-versus-claimed verdicts of book 15, ch 4 (interview defense cookbook, contributions defense) and the falsifier drill of book 15, ch 16 (interview defense cookbook, the deception taxonomy), and this section states it as a method of its own.

The exemplar is the launcher postmortem recorded in `ref/source-review-openai-math-gunny-mvp.md`. A reimplement-sprint reported verification pass rates above 90 percent, and physical inspection of the artifacts refuted the narrative the rates sat on: the screenshots were placeholder renders unrelated to the recovered assets, the asset loader and the 6,731 extracted pngs both existed but were never wired together, and the claimed atlas chain had zero output on disk. The files were real, the narrative was false, and only opening the artifacts separated the two.

The audit that catches this is 4 moves:

1. Open the artifact, not the report, because the report is written by the same process whose work is in question.
2. Trace one claim end to end, from the bullet that asserts it to the bytes that carry it, because a pipeline whose output is never consumed is a story.
3. Date the evidence, because an artifact inspected today says nothing about tomorrow's tree.
4. Regrade the claim, not the test, because a suite that mirrors the implementation passes while the requirement is unmet.

That last failure is the mirror-suite law: a suite written after the implementation to match it certifies self-consistency, and its perfect scores are overfitting with a certificate.

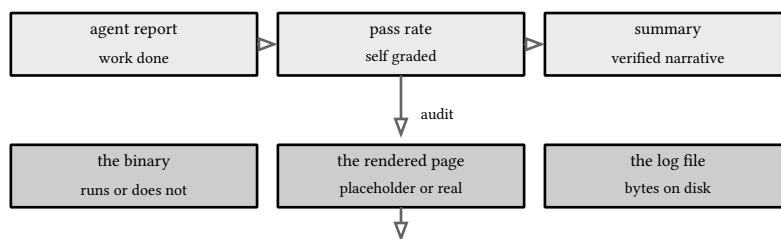


figure 52: the claims chain goes dashed, the audit resets to the artifact row

An agent run reports 95 percent verification passing on work you did not watch. Is that good enough to ship?

No, a pass rate is a claim from inside the same process whose work is in question, so I restart at the artifact: which binary runs, what the rendered page shows, which log holds the bytes, then one claim traced from the bullet that asserts it to the file that carries it. The launcher postmortem at [ref/source-review-openai-math-gunny-mvp.md](#) is the case I keep, 90-plus percent reported while the screenshots were placeholders, 6,731 extracted pngs sat unwired, and the atlas chain had zero output on disk. I regrade the claim against the artifact, not the suite, because a suite written after the code to match it only certifies itself.

floor: book 15, ch 4 (interview defense cookbook, contributions defense), book 15, ch 16 (interview defense cookbook, the deception taxonomy)

You quote star counts from last week. What if they changed?

They drift, that is what live counts do. My numbers are pinned to the 2026-09-10 snapshots under [coverage/gh](#), each one says so, and the drill is to say the live number when it has moved and name both. What does not drift is the method: three `gh api` calls, saved verbatim, anyone can rerun them during the call.

Why should I trust your tokei count over the resume's 100k?

You should not trust either, you should run one. The measurement is documented: tokei 15.0.0 over main at `f046fb2`, 42,059 code lines of 49,780 total, Go alone 22,695. The resume number names the earlier tree before the Python service landed, and giving you today's measurement with the command is exactly why the rest of my numbers deserve the benefit of the doubt.

floor: book 15, ch 20 (interview defense cookbook, llm-tournament deep dive)

These vendored files, how do I know they are real?

Fetch them yourself, the paths are the repo paths and the repos are public. The listing captions name the source file and line range, the check-listings gate fails the build if a path points at nothing, and I kept the set small precisely so it stays evidence instead of a fork I would have to keep current.

VERIFY: the one command to remember

Every deep dive in chapters 18 through 24 reduces to the same move: open `coverage/gh/<repo>.json`, read the raw response, compare against the live repo page. Rehearse that move once per repo and the whole evidence half of this book is covered.

12. the types of logic, derived and attacked

Every drill in this book is an inference wearing interview clothes. book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) names the modes of questioning, this chapter names the machinery underneath the answers. Six shapes cover the ground: deduction, induction, abduction, statistical and causal inference, and dialectic. For each one the chapter states what it is, how it derives its conclusion, how it defends under attack, and where it breaks.

12.1. the frame: what, why, how, when

Any inference someone hands you survives four questions. What is the claim, and what do its words mean. Why does it follow from the ground offered. How was that ground produced. When does the claim stop holding. Each question invites exactly one attack: what contests the definition, why contests the link, how contests the method, when contests the boundary. Naming the logic an argument runs on names the legal attacks against it, and naming the logic your own answer runs on matters more, because the interviewer is about to choose the attack for you.

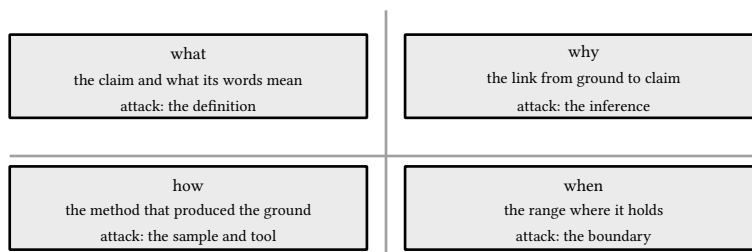


figure 53: what, why, how, when, four questions and the attack each invites

Which logic are you using when you defend the coverage number?

Induction, with a verification step behind it, and I name both before I answer. The number is a generalization from a measured run, so the defense is the measurement: how the sample was drawn, when it ran, and what it leaves out. If your question is really about evidence, that is verification, the repo answers it and the number stops mattering. Naming the mode first tells you which attacks are legal and tells me which floor to stand on.

floor: book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking), book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

NOTE: name the mode before the answer

Naming the logic first does three things at once. It tells the room which attacks are legal, it tells you which floor to stand on, and it converts an ambush into chosen ground. An answer whose mode stays unnamed can be attacked from all four questions at once, and defending every side at the same time sounds like dodging.

12.2. deduction

Deduction derives the conclusion from the premises by form alone. If every writer serializes through one log and this service has one log, its writes serialize: the form carries the conclusion regardless of the content. Validity is a property of that form, a valid argument cannot have true premises and a false conclusion. Soundness is validity plus true premises, and the gap between them is where interview fights live. Defend a deduction by showing the form: strip the argument to its shape, and the only thing left to attack is a premise. Attack one by contesting a premise you can show false, or contest the form itself when you hold a counterexample, a world where the premises hold and the conclusion fails. book 2, ch 13 (mathematics for programmers, logic, sets, and proof) teaches the forms at full length.

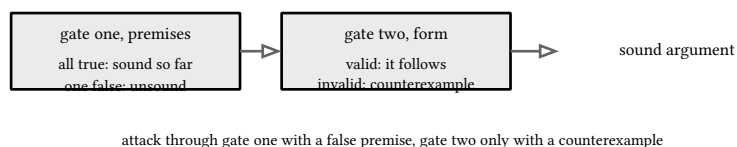


figure 54: validity and soundness as two gates in series

Your logic checks out, you said so yourself. So if the conclusion turns out wrong you were just arguing from bad premises. Isn't that guessing with extra steps?

No, and the distinction is the whole defense. Validity says the conclusion follows from the premises, soundness says the premises are also true, and I defend in that order. I show the form first, so the only thing left to attack is a premise, and every premise I rest on is one you already hold. If you show a premise false, the argument is unsound and I concede it on the spot, that is the system working. Guessing has no premises for you to inspect, my arguments do.

floor: book 2, ch 13 (mathematics for programmers, logic, sets, and proof)

12.3. induction

Induction generalizes from enumerated cases: these runs behaved so, therefore the system behaves so. The conclusion is never certain, it is weighted by the sample, which makes the sample the entire battlefield. Sample bias is the quiet attack, the selection rule that made the sample lie before anyone spoke. Hasty generalization is the loud one, a handful of cases promoted to a law. The defense is procedural: state how the sample was drawn, state the measured number, state when it was measured, and scope the claim to exactly that sample. A number quoted without its measurement date is a rumor. book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) owns the honest measurement discipline this defense borrows.

Every example you show me works. You are showing me the survivors and calling it a pattern, aren't you?

That attack is legal, so I answer it before you finish asking. Every generalization I claim names its sample: what was measured, how it was selected, and when the measurement ran. The scope of my claim is the scope of the sample, and I do not extend past it. If the selection rule favors the survivors, the number is biased and I say so out loud, because a generalization stated with its selection rule is one nobody can take apart in this room.

floor: book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)

12.4. abduction

Abduction infers the best explanation for an observation, and it is the logic the employment gap runs on. The interviewer observes months without an employer and generates explanations, some flattering, some not. Denying the observation loses. The win goes to the explanation that covers more of the evidence than any rival: a planned period with named, inspectable outputs explains the timeline and the artifacts together. The attack is surgical, leave a competing explanation standing and the abduction fails even when yours is better. So the rehearsal retires the rivals in advance, each one named and out-explained. book 15, ch 6 (interview defense cookbook, gap and chronology narratives) carries the narratives themselves.

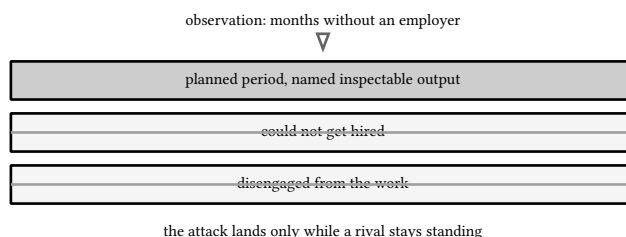


figure 55: candidate explanations for one observation, the best retires its rivals

That gap explains itself, doesn't it? I can read it three ways and two of them do not flatter you.

Then we race the readings instead of trading impressions, that is abduction and it has rules. A planned build period with a named, inspectable output explains the timeline and the artifacts at once. Disengagement explains the timeline but contradicts the artifacts, and failing to get hired explains neither the outputs nor the clients who rehired me. I rehearsed the rivals before this room, the best explanation wins on scope, and you only defeat me by keeping one rival standing.

floor: book 15, ch 6 (interview defense cookbook, gap and chronology narratives), volume 0, ch 1 (resume defense analysis, method)

12.5. statistical and causal inference

Statistical inference derives a rate from a counted population, and the population is where it dies. The base rate comes first: a large share of a small pool can sit below a small share of a large pool, and any conditional quoted without its base is a trap you set for yourself. One number reads three ways: as chance inside the noise, as correlation traveling with a change, as cause when a mechanism connects them. Correlation is the default reading and causation is earned, and the currency that earns it is the mechanism or the intervention, the change run both ways. The attack swaps the denominator, hides the base rate, or promotes a correlation to a cause without a mechanism. book 2, ch 18 (mathematics for programmers, probability and distributions) and book 2, ch 19 (mathematics for programmers, statistics and inference) own the machinery end to end.

The number moved after your change. You say improvement, I say coincidence. Prove it to me.

I read the number three ways before claiming anything, and I do it out loud. As chance: whether the move sits inside the noise the base rate already produces. As correlation: the move travels with my change and with nothing else that moved. As cause: there is a mechanism from the change to the effect, and the intervention run both ways settles it. I state the base rate first, because a big share of a small pool can sit below a small share of a big pool. If I cannot show the mechanism, I downgrade my own claim to correlation before you do it for me.

floor: book 2, ch 18 (mathematics for programmers, probability and distributions), book 2, ch 19 (mathematics for programmers, statistics and inference)

12.6. dialectic and definitional moves

Dialectic is the logic of the definition and the concession. Definition fights decide arguments before evidence arrives: what counts as production, what counts as a rewrite, what counts as senior. Fix the definition first, derive its consequences second, and the fight moves onto ground you chose. The Socratic concession ladder is the defensive form: concede the smallest true thing first, then trade each rung for scope. Informal titles stay informal, participant stays participant, and every concession buys back the boundary of the claim. The attack is the late definition shift, waiting until the claim lands, then redefining its central word. The defense is having fixed the words before use. volume 0, ch 1 (resume defense analysis, method) frames the threat model this ladder lives inside.

What counts as production? You keep using the word and I do not think you have earned it.

Then we fix the definition before it does damage, that is the whole move. Production means real users, real consequences for failure, and no undo. Under that definition my client systems count and my capstones do not, and I say that part plainly. If your definition differs, state it and I re-scope my claim against yours on the spot. I concede the smallest true thing first, informal stays informal, and every rung I trade buys the scope of the claim back.

floor: volume 0, ch 1 (resume defense analysis, method), book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

12.7. the six logics, one table

One row per logic: how it derives, how it defends, how it is attacked, and the conditions where it wins the room.

type	derive	defend	attack	when it wins
deduction	conclusion from premises by form	show the form, rest on shared premises	contest a premise, or the form via counterexample	the premises are already shared
induction	generalization from enumerated cases	sample rule, measured number, measurement date	sample bias, hasty generalization	the sample is honest and named
abduction	best explanation for an observation	explanatory scope, rivals retired in advance	leave a competing explanation standing	the rivals are known and weaker
statistical	a rate from a counted population	base rate stated before the conditional	swap the denominator, hide the base	the number and its base ride together
causal	a mechanism from a correlation	show the mechanism or run the intervention	common cause, reverse cause, chance	the mechanism is on the table
dialectic	a definition, then its consequences	the concession ladder, rungs traded for scope	contest the definition, shift it late	the words were fixed before use

PITFALL: the validity trap

A valid argument over a false premise still loses the room. Validity feels like winning because the form cannot be broken, yet soundness is what the listener actually grades. Show the form, then volunteer which premises do the work and how you know they are true. The candidate who defends an unsound syllogism because it is valid has already conceded without noticing.

13. fallacies, biases, and appeals

book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) names the modes of questioning, this chapter names the moves inside them. A hostile question is rarely a lie, it is an argument with a broken shape, and every broken shape has a name, a marker, and a counter that drills like any other answer, on one habit from volume 0, ch 1 (resume defense analysis, method): a claim backed by a repo cannot be argued away, only ignored, and ignoring is visible.

13.1. formal fallacies

A formal fallacy is an inference that breaks regardless of content, so write the argument as letters and the break survives every topic swap. Affirming the consequent runs if p then q, q, therefore p: if the code were correct the tests would pass, the tests pass, therefore the code is correct, yet both premises hold and the conclusion fails, other roads reach a passing suite. Denying the antecedent runs if p then q, not p, therefore not q, engineers who ran Raft reason about consensus, you never did, therefore you cannot, and the conditional never promised production was the only road in. The undistributed middle runs all a are c, all b are c, therefore all a are b, the hires who struggled were generalists, you are one too, therefore you will struggle, and the shared predicate points both terms at c, never at each other.

Spot any of them by restoring the form, then counter by writing the form back at the speaker in one calm sentence and handing over a true instance of the claim. The counter is a restoration, not a refutation, the tone stays curious while the logic moves.

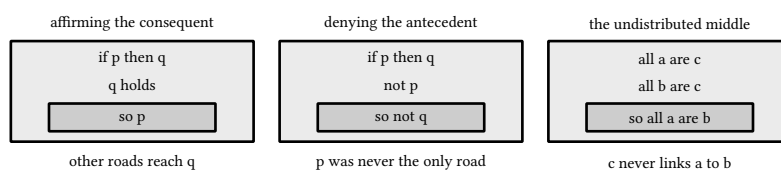


figure 56: the three formal breaks, each form restored, the broken step shaded

You have never run Raft in production, so you cannot reason about consensus. Why does the mesh line on your resume prove anything?

The form is if p then q, not p, therefore not q, denying the antecedent, and the premise never said production is the only road to the reasoning. The claim I actually make is checkable: the mesh runs membership gossip with delivery semantics and idempotent reconcile, I can teach why it never needed an election, and the consensus floor behind it I can walk from memory. Ask the mesh question at any depth.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

13.2. fallacies of relevance

The premises may all be true and still do no work, because they aim somewhere other than the claim. The ad hominem attacks the source, the gap history, the school, the star count. The tu quoque answers a claim with the claimant's hypocrisy. The straw man swaps your claim for a weaker neighbor and beats the neighbor, you said breadth, the reply attacks dabbling. The red herring drags the exchange onto friendlier ground, and the claim fades the longer you stay.

One counter covers all four: restate your original claim verbatim, then answer it. Verbatim is the trick, the restatement shows the room exactly what was dropped and does the naming without one label spoken aloud. Answer and stop, chasing the herring answers a question nobody asked, and it reads as guilt.

Your gap history proves you'll leave us too. Why should we bet a head count on someone who will bolt?

Two moves are firing and neither touches the claim: a few exits turned into a law, hasty generalization, argued against the person instead of the record, ad hominem. The claim on the table is whether I will do the work, so here it is restated: across those years the repos kept landing, employment and shipped projects fill the gaps, and the story on either side is the same story. I answer the record all day, I will not defend a prediction from other exits.

floor: book 15, ch 6 (interview defense cookbook, gap and chronology narratives)

13.3. fallacies of ambiguity

The words hold still and the meanings slide. Equivocation is one word doing two jobs, experience as years elapsed against experience as systems survived. Composition infers the whole from the parts, the team shipped so every member carries the credit, and division runs back the other way, every function tested so the system is correct. The slippery slope argues the last step first, hire a generalist and soon nobody owns the database, with none of the middle steps defended, and each step is checkable, so press it there.

The counter is to pin the definition once, publicly: state the sense you are using, freeze it, answer under it. The pin turns fog into a checkable claim, home ground for a resume with repos behind it. On the slope, argue the next step only and let the undefended steps stay visible.

You call yourself a generalist, so you're a jack of all trades and master of none. Why should we pay for breadth?

The word changed meaning mid-sentence, and that swap was the whole argument. You read generalist as master of none, the resume writes it as breadth with a floor under each column, so let me pin it: four languages, provable depth under each, and the depth is checkable, pick any line and I will teach the system underneath it at whatever depth you want. Master of none was never the claim, and now the word cannot slide again.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

13.4. presupposition and loaded questions

A loaded question is two assertions wearing one question mark. Have you stopped padding your resume numbers carries the presupposition that you padded, and the yes and the no both convict. The question does its work before any answer, so open the box instead of picking a door.

The counter: split the presupposition, answer only the true half. Name the smuggled claim in one sentence, deny it if it is false, then answer the legitimate remainder. I never padded, so there is nothing to stop, and the numbers are measured, checkable, said without heat.

Have you stopped padding your resume numbers?

There are two questions in there and they come apart. The smuggled one: I never padded, so there is nothing to stop, every number on that page is measured and the repos hold all of them. The fair remainder is whether a number survives checking, and the answer is yes, pick one and open the repo with me. I answer the fair half, I decline the smuggled half.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

NOTE: this section feeds the taxonomy

Here the loaded question is a shape, a form with a marker and a counter. A later chapter owns the driver view, who loads a question on purpose, what the loading buys, and where the move sits in the deception taxonomy. Learn the split here, the intent when that one lands.

13.5. cognitive biases

A bias is a tilt in the processing, not an argument, and the first diagnostic question is whose head it lives in, because their biases and yours take different counters.

Confirmation is usually yours: rehearsing only the questions that flatter the resume leaves the hostile ones unrehearsed, so hunt your own disconfirming evidence first and concede the weak points before they get read aloud. It is theirs when they read the gap line and stop reading, and the fix is the same, volunteer the record they skipped.

Anchoring is theirs most of the time: the first number spoken frames every number after it, so restate the number from your own record before negotiating inside theirs, measurement is the anchor you control, book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking) owns that discipline. Availability is theirs too, one vivid failure, the last generalist who washed out, outweighs a quiet record of shipping, so name the sample, one story is one draw, and offer the record as the base.

Sunk cost is yours: defending a dead project because years went into it reads as attachment, so answer what the project proves, never what it cost. The dunning-kruger gap is theirs when it arrives as the confidently wrong question, consensus needs Raft, everyone knows that, and the counter is to teach rather than match the confidence.

13.6. appeals

An appeal borrows its force from somewhere other than the claim, and every source has a one line counter: point back at the checkable claim and let the borrowed force dissipate.

- Authority: a staff engineer would call this overengineered. Rank is not a reason, ask for the reason under the rank and answer that.
- Popularity: everyone does it this way. Adoption says how many, never whether, and the evidence per claim does not move with the crowd.
- Novelty: nobody builds it like that anymore. New is not better, ask what the new way fixes and compare on the tradeoff.
- Tradition: we have always run it this way. Age is not correctness, agree on the criterion and run both ways against it.
- Fear: if you are wrong the outage is on you. Price the fear honestly, then bound it, show the guardrails and the blast radius.
- Pity: the team is drowning, you would be the savior. Need is not fit, state what you can carry and decline the savior frame.
- Nature: that stack is not natural. Natural names nothing, ask for the mechanism and answer with the mechanism.

Everyone uses AI to write resumes now. Why should we treat yours as evidence of anything?

Popularity measures adoption, and adoption cannot testify about truth, so the appeal does no work here. Whoever wrote the words, each claim on that page points at a repo or a measurement, and the offer is unchanged: pick a line and I will defend it at the whiteboard for as long as you like. If a model drafted the sentence, the floor under it did not move, and the floor is what checks.

floor: volume 0, ch 1 (resume defense analysis, method)

13.7. the counter playbook

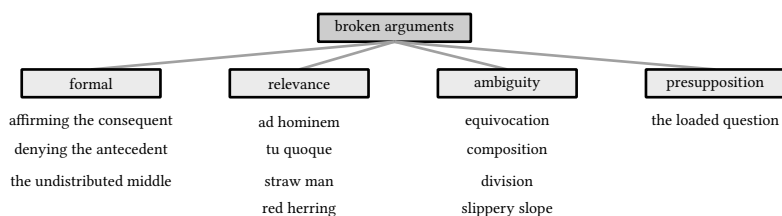
The table is the drill sheet: the marker is what you hear, the counter is what you say, the trap column is what happens when the counter misfires.

fallacy	marker	counter	trap if you misfire
affirming the consequent	tests pass, so it is correct	restore the form, offer a true instance	misfire on a valid form, hand back the point
denying the antecedent	never ran it, so cannot know it	show another road reaches the conclusion	concede production is the only teacher
undistributed middle	that group failed, you are that group	separate the shared predicate from the person	defend the group instead of yourself
ad hominem	the attack lands on you, not the claim	restate the claim verbatim, answer it	name the person, lose the room
tu quoque	you did it too, so quiet	the hypocrisy and the claim are separate bills	answer the hypocrisy, drop your claim
straw man	the paraphrase is weaker than your claim	restate the original claim verbatim	defend the paraphrase as if it were yours
red herring	the topic moved, the claim did not	name the swap, return to the claim	chase the herring to look responsive
equivocation	one word doing two jobs	pin the definition once, publicly	argue both meanings, feed the fog
slippery slope	the last step argued first	argue the next step only	defend the slope's end from its start
loaded question	both answers convict	split the presupposition, answer the true half	answer the compound, lose either way
confirmation bias	the record read only for guilt	volunteer the weak point first	rehearse only the friendly questions
anchoring	their number frames the exchange	restate the number from your own record	negotiate inside their frame
appeal to popularity	everyone does it	point at the evidence per claim	out-appeal the appeal with a bigger crowd

One discipline governs the whole table: name the move, never the person. The label aims your counter, it is not the counter, and said aloud with an edge it converts a won point into a lost room, because the room scores you under fire, not the logic. The interviewer can be wrong quietly or be humiliated loudly, and only one of those ends with an offer.

PITFALL: naming the person

The fastest way to lose a won exchange is to attach the label to the speaker, you are just using an ad hominem. The move deserves the name, the person does not, and the room cannot tell a correct accusation from a counterattack. Keep the label private and speak the restoration.



one counter per family: restore the form, restate the claim, pin the word, split the question

figure 57: the broken shapes sorted into four families, one counter pattern per family

14. four stances, steelmanned and attacked

Interviews smuggle philosophy inside small phrases. “Natural talent”, “a real engineer”, “how did you notice”, each phrase carries a stance about what categories are, what counts as evidence, and what authority deserves. This chapter arms four stances that recur in technical interviews, each steelmanned in one breath, attacked in the next, then routed back to the question on the table.

14.1. why stances at all

Nobody asks about metaphysics directly. They ask whether some people are just born for this, whether a title is even a real thing, and the stance arrives wearing a job question as a costume. Four recur: biological realism behind talent talk, nominalism behind title talk, phenomenology behind every noticing question, individualist anarchism behind every question about managers. Under the modes of book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) these are depth probes in costume, and this chapter is the floor. Each stance gets the same three moves.

- Steelman it in one breath, its strongest form, never the straw version the internet prefers to argue with.
- Attack it in the next breath, harder than the interviewer would, so nobody suspects you are hiding inside it.
- Answer the question actually asked, because the stance was the frame and the work is the point.

The concession rule prices the exchange: a frame point is a way of putting the question, a fact is something that happened, and they are not the same currency. Conceding that categories are fuzzy costs nothing and buys the room. Conceding that maybe you never owned that API goes on the record and reads like a fact discovered under pressure. Frames trade freely, facts move one direction only.

the phrase	stance smuggled	the question actually asked
some people are just born for this	biological realism	what did you practice, and can we test it
is a “senior engineer” even real	nominalism	what scope did you own and ship
walk me through how you noticed it	phenomenology	what signal did you act on, what checked it
engineers do not really need managers	individualist anarchism	how do you work inside this team’s process

You steelman a position, then you attack it yourself. Do you believe anything, or is hedging just your personality?

Hedging refuses to commit. I commit twice, once to the strongest version of the claim and once to its best counterattack, so you can inspect both before I tell you what I think. Ask me a live one and I will pick a side and name the evidence that would flip me. That is testing the floor before standing on it.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

14.2. biological realism

The claim: categories with an evolutionary history track real structure. Organisms come in kinds, minds differ, and the differences are pattern rather than noise. Every “born for this” probe stands on this stance, usually without saying so.

Steelmanned in one breath: living kinds are homeostatic property clusters, traits covary and hold each other in range because a shared history of selection paid for the bundle. A species is a population lineage, ancestry leaves real seams, and selection is what holds the category together. Temperament loads what a person will enjoy practicing for a decade.

Attacked in one breath: every edge is vague, ring species walk the boundary, and any cluster dissolves into a continuum under enough zoom. The stance slides from description to license, from “this cluster exists” to “this

cluster should sort the pipeline”, and no observation carries it across that gap. The cut that predicts milk yield is not the cut that predicts herding, the world supplies the variation and we supply the knife.

When it matters: talent talk and “born for this” probes.

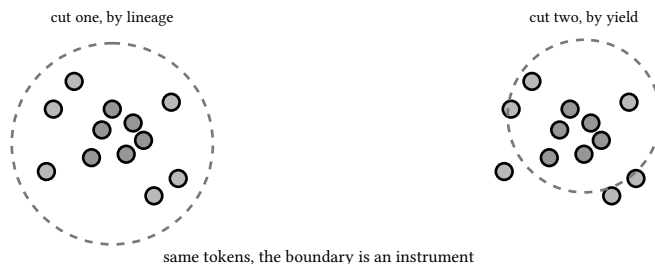


figure 58: the same twelve tokens under two defensible cuts

Some people are just born engineers and some are not. You believe that, yes? Defend it in one breath, then attack it before I do.

Steelman: kinds are property clusters held together by selection, minds differ, and temperament loads what a person will practice for a decade, so “born for it” points at something real. Attack: every edge is vague, the slide to “talent should decide the hire” is smuggling no observation licenses, and nobody here can measure the talent claimed, so test the practice instead: I got here through thousands of hours I enjoyed.

floor: book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)

14.3. nominalism

The claim: only tokens exist. This keyboard, this build, this particular Tuesday, and every kind is a name flung over a pile of tokens for convenience. “Senior engineer” is a sound companies make, there is no form of Seniority casting shadows onto org charts.

Steelmanned in one breath: no two tokens are identical, every actual thing is this thing with a serial number, and the world’s only population is individuals. Kinds earn their keep anyway as predictive shorthand, the name compresses what the next token will probably do, and laws are descriptions of the pattern rather than legislation.

Attacked in one breath: structures are real and they push back. The same theorem covers every stack ever instantiated, and the theorem is not one of the tokens. The abstract data types every handbook in this series leans on are defined by laws that outrun their instances, book 2, ch 13 (mathematics for programmers, logic, sets, and proof) holds that floor. A name that is merely convenient cannot explain why it is inconvenient to disagree with the algebra.

When it matters: the “is X real” probes and title essentialism. The no titles policy is this stance operationalized, the title word is a name whose reference drifts while scope verbs point at tokens anyone can inspect.

“Senior engineer” is just a label somebody invented. Is anything on your resume real, or is nominalism your excuse for everything?

Steelman: only tokens exist, my resume is tokens, repos, commits, shipped services, and “senior” is a sound that means different things at different companies, which is why titles are off. Attack: some structures are real regardless of naming, a stack is its laws, and the laws are not a label. So I drop the names that drift and keep the scope that holds still.

floor: book 2, ch 13 (mathematics for programmers, logic, sets, and proof), book 15, ch 8 (interview defense cookbook, no titles and no metrics policy answers)

14.4. phenomenology

The claim: experience is the datum. Whatever else is real reaches you through a first-person channel, consciousness is always of something, and the honest method is to bracket the existence question and describe how the thing is given.

Steelmanned in one breath: there is no view from nowhere. Every instrument reading becomes an experience before it becomes a fact, and the interview itself is a first-person report the whole way down. Debugging lives in this channel, the anomaly is noticed before it is measured, and bracketing is how you describe the noticing without swearing prematurely that it was the cache.

Attacked in one breath: the channel is lossy and it edits. Introspection confabulates, hindsight rewrites the noticing into a story with a hero, and a feeling of slowness is a hypothesis, never a result. The repair is the third-person check, the report ends in a claim someone else can run, and intersubjective verification is why code review beats introspection, the reviewer sees the diff and not the feeling.

When it matters: the “walk me through how you noticed X” probes. Answer as channel, signal, check: what I was watching, what stood out, what I ran to confirm it.

You keep saying you “noticed” things. Vibes are not engineering. Defend phenomenology in one breath, then attack it yourself.

Steelman: every fact you will ever hold arrived through a first-person channel, noticing is where debugging starts, and this interview is a stack of first-person reports, including your read on my resume. Attack: the channel confabulates and hindsight edits the story, so a hunch is worth exactly the hypothesis it generates, and I run the check, the profile, the replay, the second pair of eyes. The noticing starts the pipeline, the measurement ends it.

floor: book 9, ch 1 (practical data structures and algorithms, complexity analysis and honest benchmarking)

14.5. individualist anarchism

The claim: you own yourself, association should be voluntary, and a hierarchy that issues orders owes a reason every time. Self-ownership, mutual aid, and a standing suspicion of coercive authority travel together in this stance.

Steelmanned in one breath: self-organization demonstrably ships. Open source runs critical infrastructure with nobody commanding the roadmap, maintainers and contributors coordinate through reputation and patches, and mutual aid is older than the firm. The employment relation reads as repeated voluntary exchange, both parties can decline the next round, and reputation disciplines employers almost as hard as it disciplines workers.

Attacked in one breath: free riders eat uncommanded goods. The roadmap nobody commands still rides on documentation, security response, and boring maintenance, and public goods want institutions. The interview is the refutation in miniature, you are asking to join this hierarchy, and the honest version distinguishes functional hierarchy, which coordinates, from coercive hierarchy, which merely commands. Exit cost asymmetry finishes it: one side’s walk is a Tuesday and the other side’s is rent.

When it matters: questions about managers, process, and authority.

WARNING: never volunteer this stance

Unsolicited, this stance reads as a culture flag. The interviewer hears “will not take direction” and files it before the sentence ends, and no steelman repairs a first impression. Steelman and attack it only when the question demands it, then return to how you work inside teams. When positions may be stated at all is owned by the policy chapter, book 15, ch 8 (interview defense cookbook, no titles and no metrics policy answers), and the rule there is the rule here: as professional standard, never as identity.

You seem allergic to authority. Would you even accept a manager? Defend individualist anarchism in one breath, then attack it.

Steelman: I own myself, association should be voluntary, and open source ships infrastructure nobody commanded, so coordination without coercion is real. Attack: free riders eat uncommanded goods, public goods need institutions, and the exit is asymmetric, my “no thanks” costs me rent while the employer’s costs a Tuesday. The job I am applying for is a hierarchy, which tells you I read this one as functional rather than coercive, and I work well inside a structure that explains itself.

floor: book 15, ch 8 (interview defense cookbook, no titles and no metrics policy answers)

14.6. the move both ways

Both directions run the same move: concede the frame, answer the question asked, never lecture. The steelman is one breath because it exists to earn credibility, the attack is one breath because it exists to prove independence, and the answer gets the rest of the air in the room. Two failure modes do most of the damage: the steelman that becomes a lecture, and the stance volunteered when nobody asked. The interviewer asked about the work, and the work is the only thing on the clock.

stance	steelman in one breath	attack in one breath	when it matters
bio realism	selection bundles	vague edges, slide	talent talk
nominalism	kinds compress	structures push back	titles, is x real
phenomenology	all is first person	third person check	how you noticed
anarchism	open source ships	free riders, exit	hierarchy probes

figure 59: four stances, three moves, the chapter as one rehearsal card

15. game theory and the evolutionary drivers

An interview is a game in the technical sense: two players, moves in sequence, payoffs that overlap only partly. book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) names the modes of questioning and book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked) names the machinery under the answers. This chapter names the machinery underneath both: what a claim costs, why the room prices it that way, and why a species that talks all day evolved to lie at a steady rate. Payoffs explain more of the room than personalities do.

15.1. the primitives

A game is four things. Players who choose. Payoffs, what each player actually values measured in their own currency, never yours. Strategies, complete plans covering every branch. Equilibria, strategy pairs where neither player gains by switching alone. The discipline is reading behavior as the output of payoffs: when a move looks irrational, the payoff you assumed is wrong, and somebody is optimizing something you have not named yet.

The split running this chapter is one-shot against repeated. In a one-shot game defection pays, the stranger you will never meet again owes you nothing. In a repeated game cooperation pays, because the shadow of the future discounts every tempting move. The prisoner's dilemma compresses that into four cells: both cooperating beats both defecting, yet each player alone gains by defecting whatever the other does, so rational one-shot play lands both on the worse cell.

	they cooperate	they defect
you cooperate	3, 3 joint best, and unstable	0, 5 the sucker cell
you defect	5, 0 the temptation	1, 1 the one-shot landing

figure 60: the prisoner's dilemma, payoffs written you then them

Each side's temptation beats its own cooperation cell, so the rational one-shot player defects and both land on mutual defection. Nothing here requires bad people, only players pricing the exchange as a single move. Almost nothing in a career is a single move.

You keep saying the word equilibrium. What does it actually buy you in this room?

It tells me what each side gains by its own move, so I stop reading promises and start reading payoffs. If inflating a claim pays until it is checked and checking is cheap, the equilibrium is that the room checks, and I build answers that survive the cheap check first. The same read runs my side of the table: an employer who rewards verification gets verified claims out of me, and one who rewards performance trains every candidate to perform. I would rather be checked, so I open with checkable things.

floor: book 9, ch 40 (practical data structures and algorithms, game theory and scheduling), book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

15.2. signaling and cheap talk

Talk is free and priced accordingly. A claim that costs the same to make whether or not it is true carries no information: the honest sender and the liar can both afford it, so the market clears at zero. Interviews are full of such claims, passion, ownership, team player, and interviewers price them at nothing, correctly. A signal earns belief only when it costs a dishonest sender more than an honest one.

That is the handicap principle: the waste in a costly signal is not a bug, it is the filter. The peacock's tail is expensive precisely because only a fit animal can afford to carry it, so the cost does the sorting that the display alone never could. Costly signals look irrational to an accountant and exactly rational to a game theorist.

cheap talk	handicap signal
honest cost: nothing	honest cost: years, sunk
liar's cost: nothing	liar's cost: the same years, unpaid
market price: zero	market price: belief

the room prices the left column at zero daily, the public repos live on the right

figure 61: what a claim costs to send honestly against dishonestly

The public repositories are handicap signals, and that is the whole reason they carry weight. They cost years, the cost is already sunk, and a fabricator would have had to pay it too. book 15, ch 11 (interview defense cookbook, the repo evidence method) holds the check: every number the resume claims about a repo resolves to a snapshot or a recorded command, so belief never has to be asked for. Cheap talk asks to be trusted, a sunk cost invites inspection.

Anyone can say they write tested code. Why should your saying it count for anything?

It should not, and I price it that way myself. A sentence costs the same from an honest mouth and a dishonest one, so its market price is zero and the room is right to pay nothing for it. What carries weight is cost a liar cannot skip: years of public history, tests that run, claims pinned to a command anyone can rerun. I hand you the artifact and let the adjective go, and if a claim of mine ever sounds like it is asking for trust, treat that as my mistake, the check belongs on the table first.

floor: book 15, ch 11 (interview defense cookbook, the repo evidence method)

15.3. repeated games

Put the same players together again and again and the shadow of the future rewrites the matrix. Axelrod's tournaments settled the folklore: the winning strategies were nice, they never defected first, retaliatory, they punished promptly, forgiving, they returned to cooperation after the punishment, and clear, they were easy to read. Tit for tat is the minimal version, cooperate first, then mirror the last move. Generosity forgives a share of defections, because real interaction is noisy and one punished mistake can echo into a feud. Grim trigger cooperates until betrayed and defects forever after, the maximal threat and the most fragile one.

The interview is move one of a repeated game, and that is what lying costs. Not the caught moment, which is cheap. Every later move gets repriced: a colleague who remembers, a reference call, a background check, the renegotiation where your word now discounts. Even rejection is not exit from the game, this industry is a village, panels forward resumes and remember names.

If we don't hire you, you will never see us again. Why not just tell us what we want to hear?

Because that prices the exchange wrong in both directions. If the inflation works, I am hired into a job I cannot hold, and the gap between my claims and my output collects the difference with interest. If it fails, the village still remembers, the panel talks to the next panel and the discount follows me there. I play the opening move the way the winning strategies did, cooperate first and stay readable, which is exactly why my claims are checkable ones.

floor: book 9, ch 40 (practical data structures and algorithms, game theory and scheduling)

PITFALL: the one-shot frame

Most interview failure is a pricing error. The candidate prices the exchange as one-shot, where inflation pays until the check lands, and answers accordingly. Every part of that price is wrong: the check is cheap, the checkers talk to each other, and the hire itself opens the longest repeated game in the relationship. Price the room as repeated and the honest answer stops looking like idealism and starts looking like arithmetic.

15.4. commitment and focal points

A commitment device works by removing your own options in advance. The bridge burner is credible because retreat is gone: the payoff matrix changed before the argument started, not during it. That is why volunteering your own weakness works in an interview, the concession cannot be taken back and the attack it retires was about to arrive free. You trade a small option for credibility across every later move, which is a good trade whenever the game continues.

Schelling on focal points solves the other half, coordination. When two parties must land on the same choice without being able to negotiate, they converge on whatever stands out, and the convergence holds because nobody gains by deviating alone. A named artifact, a stated number, a specific date all work as focal points in a room: once stated, the discussion organizes around them.

The engineering analog is exact. A write-ahead log is a commitment device, the entry the majority accepted cannot be un-accepted, and the node that cannot retract is believed without further conversation. book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft) owns that floor end to end, quorum, log, and the leader bound by both.

Why would anyone burn their own bridges? You are sitting here arguing that having fewer options is a strategy.

Fewer options, different game, and I choose the game before you choose the move. A commitment only binds me going forward, it turns a promise into a fact on the ground, and the other side prices facts differently from promises. "I have not run Raft in production" retires an attack you were about to run for free and buys the room's attention for the parts I can defend. I burn small bridges on purpose, scope concessions and stated numbers, because they cost one option and return credibility on every later move.

floor: book 11, ch 11 (design patterns, concurrency, and distributed systems, replication and consensus: raft), book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

15.5. the evolutionary drivers

None of this is learned in business school, it is installed. Trivers on self-deception: the way to fool others reliably is to first fool yourself, because a lie its own teller believes leaks nothing. The cost is that you now hold the false belief, the payoff is a performance with no tells to read. Status hierarchies make the payoffs positional, rank is scored against peers rather than against a fixed bar, so the marginal payoff of flattering yourself is highest exactly where the check is hardest. Coalition politics and gossip are the ledger: reputation is a shared store and gossip is its replication protocol, so a defection in one room surfaces in rooms you have never entered. Machiavellian intelligence is the hypothesis that primate brains grew for exactly this chess, tracking who owes whom and who is lying about it.

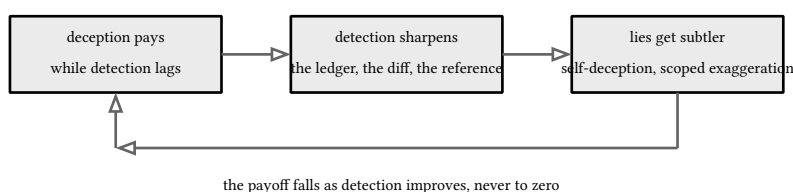


figure 62: deception and detection in the loop that never closes

Deception is therefore a stable equilibrium rather than a defect. As long as detection lags, deceptive strategies earn. Detection improves, the payoff drops, the surviving liars grow subtler, and the spiral never resolves to zero and never runs away entirely. book 9, ch 40 (practical data structures and algorithms, game theory and scheduling) carries the formal treatment, the equilibrium and minimax machinery this argument borrows.

Why would someone lie when the repo is public?

Because the payoff never lived in the checkable part. The star count is not the lie, that dies the moment anyone opens the page. The lie is the flattering read of what the repo means, the architecture was mine, the rewrite was my idea, the team followed my design, and no diff settles any of it. The reliable carrier of that lie is self-deception: believe the read first and the telling leaks nothing, which is Trivers' whole point. The payoff is positional, status in this room and rank against peers, which is why my own reads are scoped by drill, mine stays mine, participant stays participant, and the artifact carries rank so my adjectives do not have to.

floor: book 9, ch 40 (practical data structures and algorithms, game theory and scheduling), book 15, ch 11 (interview defense cookbook, the repo evidence method)

NOTE: an equilibrium is a description, not an endorsement

Calling deception stable is not calling it good, and it is not a claim that everyone runs it. A strategy earns its payoff in a specific environment, and the environment that pays deception is one where detection lags. Every chapter in this book is an attempt to live where detection does not lag, checkable claims, scoped reads, sunk costs on display. Theory prices the options, you choose which equilibrium to occupy.

15.6. reading the room with the toolkit

Each primitive names something to look for in an exchange. The table is a readout, read it the night before rather than during, then let the moves surface on their own.

primitive	how it shows up in an interview	the counter-move
payoffs	every question serves the asker, screening, calibration, politics	name the payoff in play, then serve it and your own
equilibrium	where checking is cheap nobody inflates, filler dies under follow-up	build answers that survive the cheap check first
cheap talk	passion, ownership, team player, priced at zero on both sides	drop the adjective, hand over the artifact
costly signal	years of public work, a cost the liar cannot skip	point at sunk cost and keep it checkable
repetition	the panel talks to the next panel, the reference call is move two	price every claim as one the room will re-ask
commitment	the volunteered weakness, the scoped claim, the stated number	concede scope first and retire the attack
focal point	the salient artifact or number the room converges on	put your focal point on the table before theirs
self-deception	the flattering read the candidate believes, the tellless delivery	scope every read to what the diff shows

The chapter that follows owns the marker taxonomy: the tells that mark a fabricated claim, classified, rehearsed, and attacked one by one. This chapter stops at the machinery that makes those markers worth studying, and book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) turns the whole toolkit back into answers under fire.

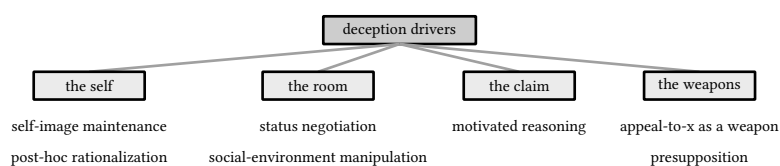
16. the deception taxonomy

book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals) named the broken shapes and drilled the counters, this chapter names the engines that run them. A fallacy is a move, a driver is the payoff the move chases, and one move serves several masters: the loaded question can guard a self-image, bid for status, or end an inquiry, and the counter changes with the master.

driver	what it is	why it pays	how it is spotted	when it fires
self-image maintenance	protecting the story one tells about oneself	the story stays intact while facts bend	credit absorbed, blame externalized, pivots land fluent	a mistake surfaces, a number gets checked
post-hoc rationalization	reasons assembled after the choice	the choice wears deliberation it never had	reasons shift across retellings, no record behind the fluency	the outcome is known and the listener is new
status negotiation	pricing relative standing in the exchange	higher standing passes claims unexamined	credential drops, territory marking, gatekeeping moves	a claim threatens their ground or invites yours
social-environment manipulation	shaping who hears what	unrebutted claims harden into facts	isolation moves, manufactured urgency, hallway re-openings	a settled point starts losing its holder
motivated reasoning	scrutiny assigned by the conclusion	the conclusion survives uneven tests	hard tests for hostile facts, soft tests for friendly ones	the evidence turns inconvenient
appeal-to-x as a weapon	borrowed force aimed at the inquiry	a dead question needs no evidence	the answer replaces the question it was asked	the point is about to be lost on the merits
presupposition	a fact smuggled inside a question	the fact enters the exchange unexamined	when-did-you-stop frames, assumptions as common knowledge	early, before the frame becomes the baseline

NOTE: the move is public, the driver is not

A fallacy can be shown to the room, a driver can only be inferred, and the inference never reaches proof, because one move serves several engines. The taxonomy aims the counter, it is not the counter: name the driver silently, speak the move's counter, and let the payoff drain on its own. An accusation of motive cannot be checked, and it trades a won point for a personal fight.



one counter per family: cheap truth, free standing, even scrutiny, the split question

figure 63: the seven drivers, four families by payoff

16.1. self-image maintenance

The first driver protects the story the speaker tells about themselves, competent, consistent, never wrong, and every incoming fact is graded on one question, does this threaten the story. Credit absorption pulls successes inward, the plan was mine, blame externalization pushes failures away, the tool was slow, the teammate skipped review, and the face-saving pivot lands fluent, that was the plan all along. It pays because admissions are priced socially. The counter makes truth cheap to tell: hand the face-saving exit before it is needed, let the correction ride a process instead of a character, and take the correction without demanding the confession.

Three projects on this resume failed and every failure belongs to someone else. The tool was slow, the teammate skipped review, the client changed course. Is anything ever your fault?

A record where all the blame points outward is the marker this chapter warns about, so I grade mine out loud. The delivery bug on the mesh was mine, I wrote the path and I own the writeup, and the two external calls are checkable the same way, the timeline and the tickets carry the blame. Pick any failure on the list and I will show you whose it was.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

16.2. post-hoc rationalization

Reasons assembled after the choice arrive fluent and shift with the audience, because they are reconstructions, built from the outcome backward. The reason told to the recruiter differs from the reason told to the panel, and behind the fluency sits no decision record, no note written before anyone knew how the choice would land. The driver pays because retrofitted reasoning is free. The counter probes the record, never the reason: ask what the design note said at the time, artifacts with a history cannot be backfilled, and the record settles what the retelling cannot. volume 0, ch 1 (resume defense analysis, method) states the rule, a claim backed by a record cannot be argued away, only ignored.

The recruiter says you picked that stack for the ecosystem. You just told me it was for the numbers. Which reason is the real one?

Neither telling is the reason, both are reconstructions, memory rebuilds the why for whoever is asking, and that drift is the marker. The reason that counts predates the outcome and exists on paper: the design note states the problem the choice had to solve, written before the results came in. Read the note, that is the decision talking.

floor: volume 0, ch 1 (resume defense analysis, method)

16.3. status negotiation

Every exchange prices standing, and much of what reads as deception is a bid. The credential drop names a school, a title, a platform, I built this. Territory marking stakes the ground, my system, my area. The gatekeeping move charges admission, that is a specialist question, let me translate. It pays because standing decides which claims pass unexamined. The counter gives status cheaply and never bids for it: acknowledge the ground plainly, you know this stack deeper than I ever will, then ask the question anyway, and a person given standing for free stops taking it by force. The claim and the rank travel on separate bills, concede the rank and answer the claim.

I built the platform you are interviewing to work on. What could a generalist possibly teach my team about it?

You know that platform deeper than I ever will, and saying so costs me nothing, so take the standing, it is yours. The claim sits beside it on a separate bill: the systems underneath platforms like yours are the ones I carry, the storage engine, the coordination, the failure paths, and I can teach any of them at whatever depth you test.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

16.4. social-environment manipulation

The fourth driver shapes who hears what. Isolation moves thin the audience, take this offline. Manufactured urgency compresses scrutiny, decide today, the window closes tonight. Selective channels route praise where it is public and doubt where it is not, and the hallway follow-up reopens a settled point in the one venue with no witnesses and no record, where a claim nobody can rebut hardens into a fact by repetition. The counter is architectural: open channels, written trails, agreements restated in the room, let us put that in the notes is the whole move, and a point reopened in the hallway gets walked back to the room, on the record.

We settled your numbers in the room, but off the record, between us, they were flattered for the audience, right? Just tell me what you say privately.

Off the record is the venue where records go to die, so I decline the venue and keep the claim. The room settled what those numbers are and the room reopens them, I will put it back on the agenda myself if they need re-examining. What I say privately and publicly is one sentence, a claim that needs a private audience is a claim I do not hold.

floor: volume 0, ch 1 (resume defense analysis, method)

16.5. motivated reasoning

Here the conclusion is chosen first and scrutiny is assigned second. Inconvenient facts get the hard tests, what is the sample, where is the base rate, who measured it. Convenient facts get the soft ones, a story, one vivid case, everyone knows. The marker is not the doubt, it is the doubt's direction, scrutiny that always points away from where the speaker needs to arrive.

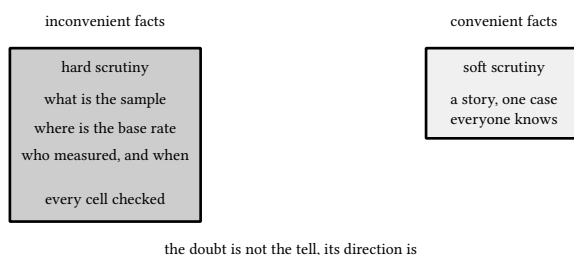


figure 64: scrutiny assigned by the conclusion, the direction is the tell

The counter flips the test. Offer your own falsifier first, name the result that would change your claim, then ask for theirs, and run the hard tests on both evidence piles with equal vigor. A speaker who can name no falsifier is carrying a conclusion, and the room hears it once the asymmetry is staged openly.

Your entire evidence base is repos you graded yourself. Our incident review, run by people with nothing at stake, says the pattern you claim does not hold. Why does your self-graded work outweigh that?

Then both piles take the same test, that is the only fair scale. Mine: the grading rule is public, clone the repo and run it, and I name the result that changes my claim, a failed suite on a fresh clone does it. Yours: who selected the incidents, what is the base rate, what would count as disconfirming. Whichever side cannot name a falsifier chose its conclusion first.

floor: volume 0, ch 1 (resume defense analysis, method), book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked)

16.6. appeal-to-x as a weapon

An appeal borrows force from somewhere other than the claim, authority, popularity, fear, pity, and as a driver it is deployed, not believed. The test is simple, did the appeal answer the question or replace it. The staff engineer would call this overengineered answers nothing about the retry budget. book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals) owns the per-appeal counters at full length, the taxonomy adds the routing rule they ride on: every appeal goes back to the question it displaced, restated verbatim, answered where it stands. The inquiry dies only if the replacement stands.

16.7. presupposition

The loaded question smuggles an unestablished fact inside a question, have you stopped padding, given your history of overpromising, as we both know. The smuggling is the payoff, the fact enters the exchange unpaid

and unexamined, and if it slides it becomes the frame every later answer inherits. The markers are the when-did-you-stop shape and the assumption offered as common knowledge. The counter runs three moves: split the presupposition out of the question, grant or reject the embedded fact explicitly in one sentence, then answer the residue on its own terms. The grant matters as much as the rejection, a smuggled fact that happens to be true still needs saying aloud.

Our principal, the strongest engineer in the building, says generalists wash out of on-call. Given that you already conceded you have no real depth, when do you drop the act and admit this interview is a courtesy?

Three layers are stacked in that question, so I take it apart instead of picking a door. The appeal: the principal's standing testifies about my pager not at all, so back to the question it displaced, can I hold on-call, name the scenario and I will walk it. The smuggled fact: I conceded no such thing, what I conceded on the record is that depth needs proving, and the repos do it.

floor: book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals), book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

16.8. each driver through the frame

book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked) hands every inference four questions, what, why, how, when, and the drivers take them too, one paragraph apiece.

Self-image maintenance guards a what, the story of the competent self. The why is admission pricing, the how is the grading rule, and it fires when a mistake surfaces, breaking when the exit is free.

Post-hoc rationalization assembles a what, the reason, after the outcome. The why is free deliberation, the how is reconstruction, and it fires when the outcome is known, breaking against the artifact that predates it.

Status negotiation prices a what, standing, the rank every claim rides. The why is the unexamined pass, the how is the bid, and it fires when a claim threatens their ground, breaking when standing is given away free.

Social-environment manipulation shapes a what, the audience, who hears the claim decides what survives it. The why is unrebuted repetition, the how is venue control, and it breaks against the open channel and the written trail.

Motivated reasoning fixes a what, the conclusion, before the evidence arrives. The why is uneven scrutiny, the how is the assignment of doubt, and it fires when the evidence turns inconvenient, breaking under the flipped test with one falsifier on the table.

Appeal-to-x as a weapon borrows a what, force from outside the claim. The why is the dead question, the how is replacement, and it fires when the point is about to be lost, breaking when the displaced question is restated verbatim and answered.

Presupposition smuggles a what, the fact, inside a question. The why is the free entry, the how is the packed shape, and it fires early, before the frame hardens, breaking at the split, the fact granted or rejected aloud, the residue answered alone.

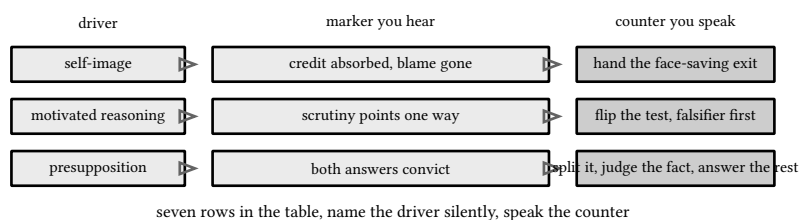


figure 65: driver to marker to counter, the routing pattern of the taxonomy

The taxonomy names the engines and their counters. book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) owns the execution layer: the pressure rules, concede scope first, evidence early, never

invent a number, are what a spoken counter rides when the room is hot. Every drill from here on assumes both layers, the driver named silently, the rule spoken plainly.

PITFALL: answering the motive instead of the move

The taxonomy invites a diagnosis, that is just self-image, you are protecting your story. No driver can be proven from the outside, the same move serves several engines, and the accusation lands as armchair psychology. Name the move the room can see, credit verbs absorbed, scrutiny pointing one way, speak its counter, and let the payoff drain without one word about the engine.

17. coming out ahead in every exchange

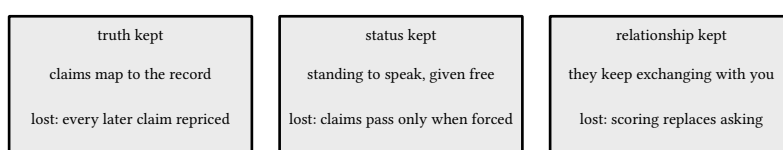
The block before this chapter named the machinery of every exchange: book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked) derived the inference under an answer, book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals) catalogued the broken shapes, book 15, ch 15 (interview defense cookbook, game theory and the evolutionary drivers) priced the payoffs, and book 15, ch 16 (interview defense cookbook, the deception taxonomy) named the drivers underneath the moves. This chapter closes the block at the layer everything else rides on, the mechanics of delivery itself. Theory picks the counter, mechanics land it, and a chosen counter delivered wrong loses the room anyway.

17.1. the exchange as a system

Every exchange settles three balances on the way out: truth kept, status kept, relationship kept. Truth is whether your claims still map to the record behind them, status is the standing to speak and be believed, relationship is the other side's willingness to keep exchanging with you. The three are priced separately and paid separately, and one answer can win the point while losing the other two.

Losing truth is the expensive one, a single bent fact discounts every straight claim after it, and the discount follows you into rooms you have not entered yet. Losing status is subtler, standing bid for and never granted reads as insecurity while standing given away for free reads as confidence. Losing relationship ends the exchange, the questions stop, the scoring starts, and the story travels without you.

Winning is leaving with all three intact, which is not having the last word. The last word spends relationship buying a point already won, the room scores the whole exchange rather than the point, and the sentence after the win has nothing left to gain.



the last word is not on the list

figure 66: three currencies leave every exchange, winning is keeping all three

PITFALL: the last word is not on the ledger

The point is scored the moment the counter lands, and every sentence after it is a new claim under new risk. Talking past the win can only spend currencies, never add one, and the room remembers the candidate who would not stop longer than the point they won. Say the counter and stop.

17.2. framing and anchoring

The first number on the table anchors the range every later number is bargained inside, and so does the first definition. book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals) prices the anchor as a bias and gives it a counter, restate from your own record before negotiating inside theirs. This chapter adds the cheaper half of the same move: frame before they anchor and there is nothing to restate against. A frame stated first reads as ground, the identical frame stated second reads as resistance.

Framing first means putting your definition and your criterion on the table before theirs arrives, what counts as production, what the number measures, what the claim excludes. book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked) owns the definitional half, fix the words before use. The anchor half is mechanical: their number heard in silence becomes the baseline the room bargains around, and moving a baseline costs more than setting one.

Every candidate we have spoken to came in under the ceiling we have in mind, and they were happy to be there. What makes you worth breaking the pattern for?

The ceiling you have in mind anchors the range, and I decline to negotiate inside a number I have not seen. My frame goes on the table first: pay against the record, the repos are public, every claim on that page resolves to something you can check, and the price of the work follows the price of the evidence. If your number and the record disagree, I want the record to win in both directions.

floor: book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals), book 16, ch 1 (the interview repertoire, the 2026 interview landscape)

17.3. question control

Answer the question asked, then stop. The instruction sounds trivial and breaks constantly under pressure, because the rehearsed answer is always richer than the asked question. The rights you actually hold when a question lands: ask for it again, split a packed question and answer the true half, name the boundary of the record, decline what an NDA covers. The right you do not hold is substitution, swapping the asked question for the rehearsed one.

Answering the question you wish you had been asked loses twice. To the room it reads as evasion, and to the record it reads as scope drift, the answer walking outside what the evidence holds. The honesty rule lives one chapter earlier, book 15, ch 1 (interview defense cookbook, how to use this book) owns it: answers stay inside what the record actually contains. Every sentence past the question is a fresh claim you now defend.

It is a yes or no question. Have you ever owned a production incident call, yes or no?

No. I have never owned the pager on a production incident call. The work that sits beside that answer, the runbooks, the failure paths I can walk at a whiteboard, stays beside it rather than in front of it, because fronting it would turn your question into a different one. That is the whole answer, and the next one is yours.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

17.4. silence, timing, and concessions

Silence after an answer is a move, not a vacancy. The questioner lets it sit and waits for the candidate to buy it back with concessions, extra scope, softer claims, a number improved in the listener's favor. Whoever fills silence pays for it, the payment is always made in a currency, and the currency is usually truth or status. Answer, stop, and let the pause belong to the person who created it.

Timing has one hard rule, and book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) owns the mechanics: correct your own numbers before they finish checking them. A self-caught error is a process working, an externally caught error is a story about you, and the difference between the two is latency, not the size of the error.

Concessions split by what they spend. Concede scope freely, it retires attacks before they arrive, and book 15, ch 15 (interview defense cookbook, game theory and the evolutionary drivers) prices it as a commitment device that buys credibility for every later move. Concede truth never, truth once conceded cannot be rebought at any price in this room or the next one.

NOTE: concede scope, never truth

The two concessions sound alike in the mouth and price oppositely at the table. "I have not run this in production" is scope, it costs one option and returns credibility across the exchange. "Perhaps my numbers

are off” is truth, it discounts every number you have not yet said. Scope is spendable again, truth is the ledger itself.

Take your time, I have all day. Surely there is more behind an answer that short.

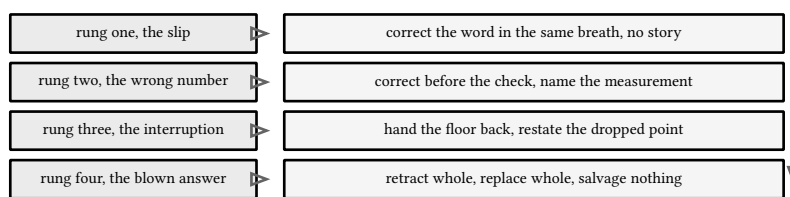
There is not more, and the length is the design. The answer you have is the whole claim, scoped to the record, and a longer version would be a softer claim wearing the same words. If something in it needs opening, point at the part and I will open that part. I am comfortable letting the quiet sit where you put it.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook)

17.5. repair

Everyone misspeaks under load, and the interview is not lost on the misspeak, it is lost on the repair that never came. The repair ladder has four rungs, one per severity, each named and rehearsed so the move is retrieval rather than improvisation. The rungs never skip, a blown answer repaired with a slip-sized correction fails twice.

Rung one is the slip, the wrong word corrected in the same breath. Rung two is the wrong number, corrected before the check arrives, the right value stated with the measurement that produced it. Rung three is the interruption, the floor handed back and the dropped point restated when it returns. Rung four is the blown answer, retracted whole and replaced whole, no partial salvage of the wreckage.



severity rises one rung at a time, repair at the rung the error lives

figure 67: the repair ladder, one rung per severity, each rung rehearsed

A moment ago you said the mesh guarantees delivery. Now the word is semantics. Which is it, and why should the next claim out of your mouth be worth anything?

Rung one, and the correction is mine: the earlier word was wrong, the mesh offers delivery semantics, at least once with idempotent reconcile, never a guarantee. No story attached, a slip is a slip. The next claim is worth exactly what the last one was, because you watched the repair fire without anyone asking for it, and that is the mechanism all my claims ride, built to be checked rather than trusted.

floor: book 15, ch 9 (interview defense cookbook, hostile cross examination playbook), book 15, ch 16 (interview defense cookbook, the deception taxonomy)

17.6. escalation and exits

Escalating precision means offering the check that settles the point, the command anyone can rerun, the repo anyone can clone, the walkthrough at whatever depth is asked. It is the costly signal from book 15, ch 15 (interview defense cookbook, game theory and the evolutionary drivers) worn as a sentence, and it is the only escalation that wins anything, louder is not more precise. Escalate when a checkable claim is priced at zero, not merely when the room is unpersuaded.

Abandoning a lost line is the opposite discipline on the same ledger. When defending a line starts costing truth or relationship, the line is already gone. Concede at the exact boundary: that one is past my floor, and here is precisely where the floor stops. The concession prices every held claim upward, a person who abandons lost ground cheaply is believed on the ground they keep.

Exit lines cost nothing when they hand the floor back without claiming a win, and they are short. The answer is the answer, and the next question is yours. That one is beyond my floor and this is where the floor stops. Run the command yourself, I would rather show you than argue. An exit claims nothing, so it risks nothing, and it closes the exchange with the relationship still on the table.

I do not believe the measurement. Anyone can type a number into a table. Convince me.

I will not convince you, I will hand you the check instead. The claim resolves to a command, clone the repo and run it, and the number comes out of the run or the claim dies in front of us both. A sentence costs the same from an honest mouth and a dishonest one, so I stop spending sentences exactly where a rerun settles the point. The offer stays open for as long as you like.

floor: book 15, ch 15 (interview defense cookbook, game theory and the evolutionary drivers)

17.7. the whole book in one loop

The block closes by running all at once. A claim lands and the loop opens. The three modes from book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) fire first: verification checks the claim is real, depth tests the floor under it, hostility implies it was inflated. Under the mode sits the inference itself, book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked) names which of the six is being made and therefore which attacks are legal. When the question arrives broken, book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals) owns the pass, the claim restated verbatim, the form restored, the question split. book 15, ch 15 (interview defense cookbook, game theory and the evolutionary drivers) prices the exchange, one-shot against repeated, and the price picks the move. book 15, ch 16 (interview defense cookbook, the deception taxonomy) reads the driver under the move and aims the counter without speaking the motive. This chapter executes the result: frame first, answer and stop, hold silence, repair at the rung, exit clean.

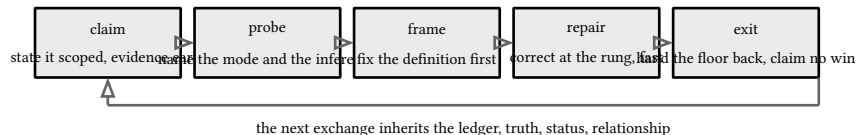


figure 68: the exchange loop, claim through exit and back to the next claim

The table prices the loop, one row per stage, the chapter that owns the stage, and what doing it wrong costs.

loop stage	owned by	what doing it wrong costs
claim	book 15, ch 9 (interview defense cookbook, hostile cross examination playbook), scoped, evidence early	an inflated ceiling the first check cuts down in public
probe	book 15, ch 9 (interview defense cookbook, hostile cross examination playbook) with book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked), mode times inference	a right answer in the wrong mode reads as dodging
frame	book 15, ch 12 (interview defense cookbook, the types of logic, derived and attacked) with book 15, ch 13 (interview defense cookbook, fallacies, biases, and appeals), definition fixed, shape checked	negotiating inside their anchor on their words
repair	this chapter with book 15, ch 16 (interview defense cookbook, the deception taxonomy), the rung, truth cheap	a small error defended becomes a character question
exit	this chapter with book 15, ch 15 (interview defense cookbook, game theory and the evolutionary drivers), the repeated game prices the close	the last word, bought with the relationship

The loop runs in every room book 16, ch 1 (the interview repertoire, the 2026 interview landscape) maps, and it never runs alone: the panel forwards the resume, the reference call follows the answer, and each exit becomes the frame of the next claim. Coming out ahead is not a moment, it is the compounding habit of leaving every exchange with the truth intact, the standing given rather than taken, and the other side still willing to ask.

18. dotfiles deep dive

The first contribution line and the one an interviewer can check fastest, because everything it counts is a plain file in a public repo. The play from book 15, ch 4 (interview defense cookbook, contributions defense) still holds: never defend 35 stars as impressive, defend the provisioning as real and countable. This chapter is the counted version, register row repo-dotfiles-provisioned.

18.1. verified facts

fact	measured value
stars	35, measured 2026-10-05, snapshot coverage/gh/dotfiles.json
forks	3, same snapshot
created	2023-06-02, same snapshot
last push	2026-10-05, head docs on the opening book passage, release v6.2 same day
primary languages	Typst 10,304,484 bytes, Shell 312,036, PowerShell 236,688, Lua 8,453
license	MIT, archived no
lsp servers	20 in init.lua, plus conditional jdtls off Windows
tree-sitter parsers	30 in init.lua, the nvim-treesitter v2 ts_parsers table
installables	46 in packages.yaml, cli_tools alone 18

The three counts are the whole 40+ tools story from chapter 4 in measured form. The languages split flipped at the 2026-10-05 v6.2 release: the repo now publishes the 15 volume typst corpus inside itself, so Typst owns the byte count, while the working config underneath is still shell for Linux and macOS, PowerShell for Windows, Lua for the editor, and the release moved the license Apache-2.0 to MIT.

The card the night-before pass reads, every number next to the place it is checked:

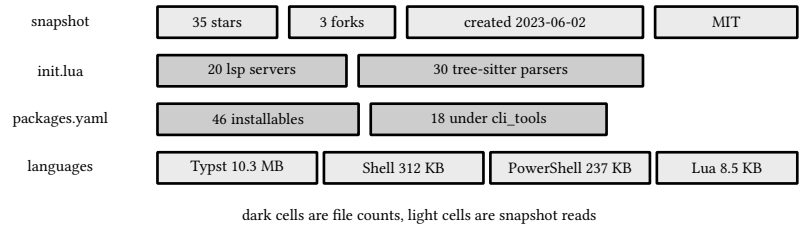


figure 69: the dotfiles fact card, every number grouped by where it is checked

18.2. the shape in thirty seconds

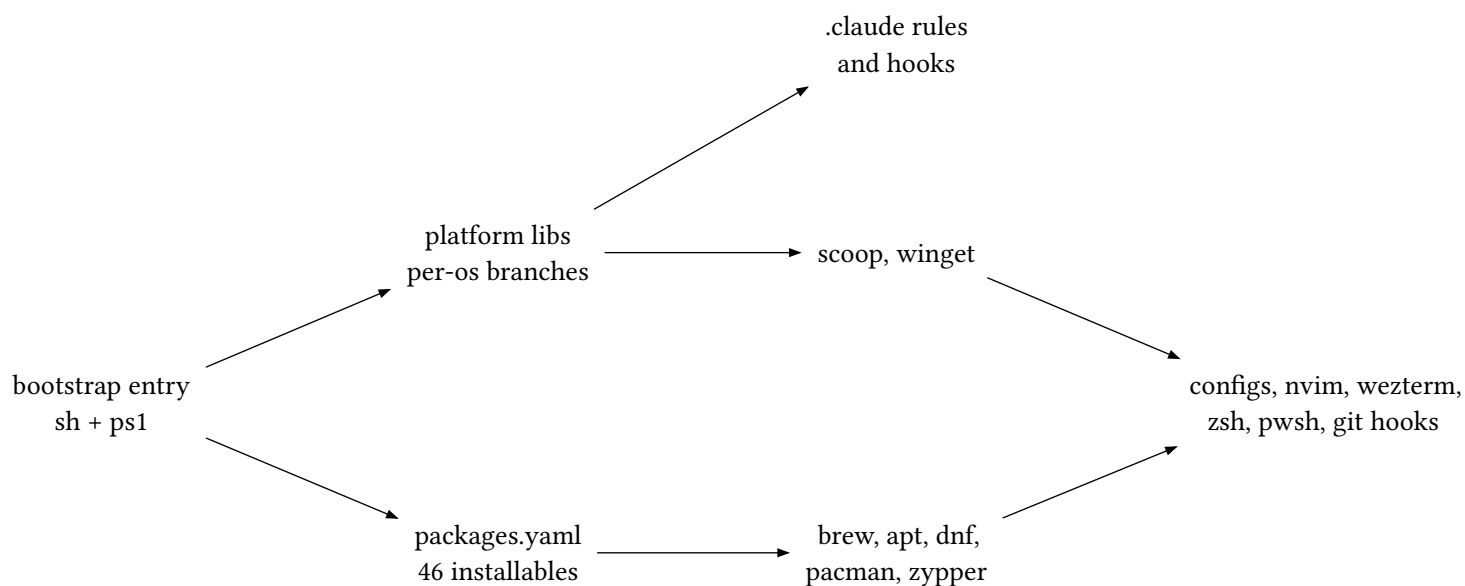


figure 70: dotfiles bootstrap, one manifest converging two platforms, drawn from the repo tree and the snapshot

Idempotence lives in the manifest design: every entry declares a `check_cmd` and a `min_version` before any install command, so the second run detects the installed state and converges instead of duplicating.

18.3. open this file first

`\.config/nvim/init.lua` in the repo, vendored here. The two lists the resume counts from:

```

-- Build LSP server list, excluding jdtls on Windows (it doesn't work)
local lsp_servers = {
  "lua_ls",
  "clangd",
  "gopls",
  "rust_analyzer",
  "pyright",
  "ts_ls",
  "html",
  "cssls",
  "svelte",
  "bashls",
  "metals",
  "csharp_ls",
  "dartls",
  "tinymist",
  "docker_language_server",
  "helm_ls",
  "yamlls",
  "tombi",
  "intelephense",
  "codebook",
}
-- jdtls is not supported on Windows
if vim.fn.has("win32") == 0 then
  table.insert(lsp_servers, "jdtls")

```

listing 1: the 20 language servers, with jdtls conditional on non-Windows (source: defense-cookbook/vendored/dotfiles/.config/nvim/init.lua, lines 59-84)

```
local ts_parsers = {
  -- Core & config
  "lua",
  "vim",
  "vimdoc",
  "query",
  -- System languages
  "c",
  "cpp",
  "rust",
  "go",
  "python",
  "java",
  "c_sharp",
  "php",
  "scala",
  -- Web & frontend
  "javascript",
  "typescript",
  "tsx",
  "jsx",
  "html",
  "css",
  "scss",
  "svelte",
  -- Data & config formats
  "yaml",
  "json",
  "toml",
  -- Documentation
  "markdown",
  "markdown_inline",
  -- DevOps & infrastructure
  "bash",
  "powershell",
  "dockerfile",
  -- Typesetting
  "typst",
}
```

listing 2: the 30 tree-sitter parsers, the nvim-treesitter v2 `ts_parsers` table, installed on startup only when missing (source: defense-cookbook/vendored/dotfiles/.config/nvim/init.lua, lines 196-234)

Count them live, both lists are flat tables. Then bootstrap/config/packages.yaml, vendored, for the 40+ tools reading:

```
cli_tools:
- name: fzf
  display_name: Fuzzy Finder
  platforms: [all]
  check_cmd: fzf
  version_flag: --version
  min_version: "0.40.0"
  install:
    macos: brew install fzf
    linux_debian: sudo apt install -y fzf
    linux_fedora: sudo dnf install -y fzf
    linux_arch: sudo pacman -S --noconfirm fzf
    linux_opensuse: sudo zypper install -y fzf
    windows: scoop install fzf

- name: zoxide
  display_name: Zoxide
  platforms: [all]
  check_cmd: zoxide
  version_flag: --version
  min_version: "0.9.0"
  install:
    macos: brew install zoxide
    linux_debian: sudo apt install -y zoxide
    linux_fedora: sudo dnf install -y zoxide
    linux_arch: sudo pacman -S --noconfirm zoxide
    linux_opensuse: sudo zypper install -y zoxide
    windows: scoop install zoxide
```

listing 3: two of the 18 cli_tools entries, the check and install pattern all 46 share (source: defense-cookbook/vendored/dotfiles/bootstrap/config/packages.yaml, lines 322-349)

Every countable number is a line range in a vendored file, the route in three slices:

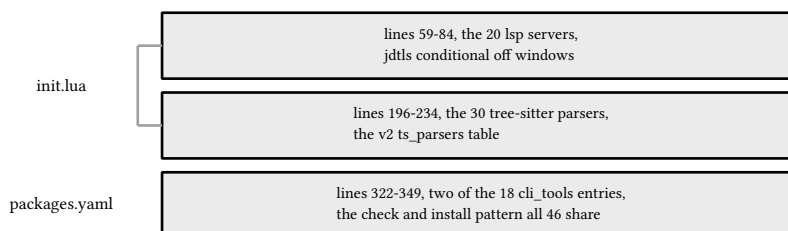


figure 71: the counting route, three line ranges carrying the 20, the 30, the 46, and the 18

18.4. drills

Walk me through what your bootstrap actually does on a fresh machine.

Two entry points, bootstrap.sh and bootstrap.ps1, one shared manifest packages.yaml with 46 installables across system package managers, sdks, language servers, linters, cli tools. Every entry declares how to check itself and a minimum version before any install command, so the script converges: package managers are probed first, existing tools are detected, configs are full file writes. Linux and macOS install through brew or the distro manager, Windows through scoop and winget. The editor config is a single 280 line init.lua, no plugin manager ceremony, vim.pack.add plus the two countable lists.

floor: book 15, ch 11 (interview defense cookbook, the repo evidence method)

You claim 40+ CLI tools. I see 18 in the yaml.

You see 18 under `cli_tools`, that is exact. The 40+ reading counts all 46 installables in the manifest: 18 cli tools, 10 language servers, 8 linters and formatters, 7 sdks, 2 package managers, 1 gui app. I will grant the narrower count before you finish counting, the manifest is one file and both numbers come off it in ten seconds.

Why does a dotfiles repo need MIT and a changelog?

Because 35 people starred a bootstrap they might actually run, and anything people run deserves a license and release notes. The repo versions itself, v6.2 at the snapshot, same discipline as any tool I ship for other people.

Each drill routes the same way, concede or assert, then land on the pin:

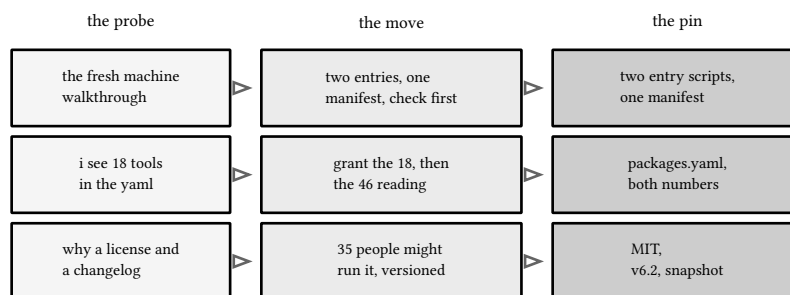


figure 72: the three concede and locate moves, each drill landing on its pin

VERIFY: thirty second locate drill

Repo page for the star count, `init.lua` for the 20 and the 30, `packages.yaml` for the 46 and the 18. Three locations, every number in the table above, rehearsed until each one lands inside thirty seconds.

19. modern-swe-library deep dive

The one repo in the set whose resume line is historical, which makes it the most rehearsed history-plus-measurement in the book. Chapter 4 carries the framing, register rows contrib-swe and repo-swe-archive, and this chapter carries what the repo actually is: a single curated document, archived on arrival, with a study path whose length is an estimate budget rather than a log.

19.1. verified facts

fact	measured value
stars	repo 9, gist 0, measured 2026-10-05, snapshot coverage/gh/modern-swe-library.json
forks	repo 4, gist 2, coverage/gh/gist-stars.md
created	2025-01-01, same snapshot
last push	2025-01-01, created and pushed the same day
primary languages	none, the languages endpoint returns an empty object
contents	one file, readme.md, 1,296 lines vendored
archived	yes, since 1/2025
old version gist	still-lake, 0 stars, 2 forks, created 2024-08-29

One card for the night before, both homes and their numbers side by side:

repo	9 stars	4 forks	created 2025-01-01	archived
the gist	gist 0 stars 2 forks 32 gists, no other copy			
contents	one file, readme.md	1,296 lines	languages: empty	

languages empty: one markdown file, no bytes to count

figure 73: the modern-swe-library fact card, two homes, one document, empty languages

19.2. the shape in thirty seconds

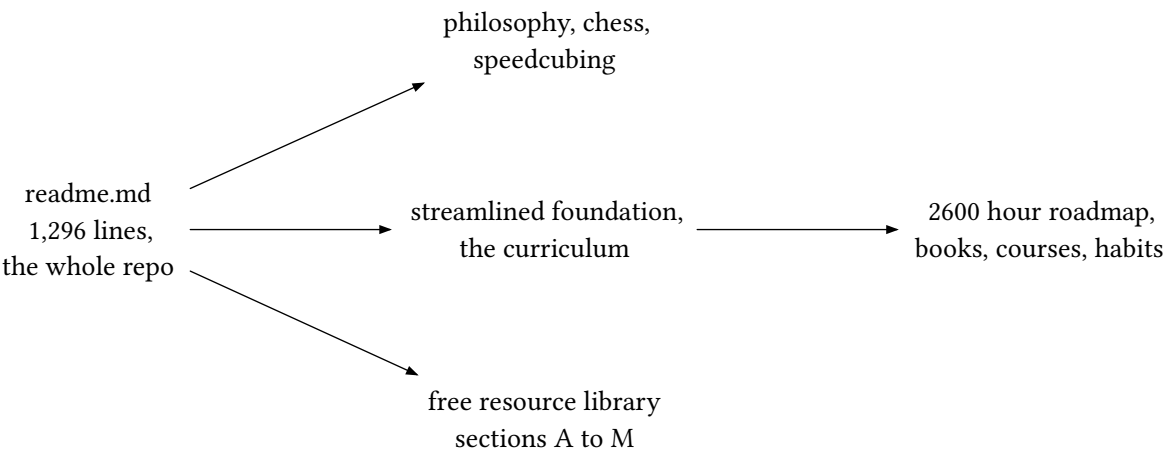


figure 74: modern-swe-library, one markdown document, its own section spine, read from the vendored readme

There is no build, no code, no service. The architecture is the table of contents, and the honest thirty second description is exactly that: a curated document with navigation, the current home being the repo itself, with the gist as the older snapshot it grew out of.

19.3. open this file first

readme.md, the only file, vendored whole. Line 9 is the load bearing line for the stars story, the pointer to the old gist:

```
# Modern Software Engineering

## Free Resource Library for NEETs

**Information, Knowledge, Education, and Instruction must always be open & free.**



Old version: <https://gist.github.com/still-lake/0aa599daf084ad2c095fd6d21d857002>

## The Foundation ([jump](#n-streamlined-swe-foundation))
```

listing 4: title, tagline, and the line 9 gist pointer the star verification started from (source: defense-cookbook/vendored/modern-swe-library/readme.md, lines 1-11)

The 2,600 hour claim lives in the foundation section:

```
1. **Lua, Python, ComSci, Computer Engineering, Operation System, Linux, Neovim, Fundamentals, Portfolio**
1. **Ruby, Rails, C#, .NET MAUI, Java, OOP, FP, Database, SOLID, Design Patterns, Maths, DSA, AI, 3rd Party**
1. **Web, OSI, Encryption, Security, OPSEC, OSINT, HTTP/1.1/2, WebSocket, SQLite, Dart, Flutter, Spring Boot**
1. **JavaScript, TS, Vue, Node, React, Native, MongoDB, MERN, Nest, Next, GraphQL, NATS, Tailwind, Shadcn**
1. **Go, HTMX, Rest APIs, GRPC, PostgreSQL, CI/CD, Pprof, OTel, Pyroscope, Grafana, Nginx, AWS, Microservices**
1. **System Design, Distributed Systems, Redis, RabbitMQ, Kafka, Docker, K8s, Helm, Argo, Flink, Event-Driven**
1. **Rust, C++, OpenGL, Jenkins, Ansible, Terraform, CNCF, Blockchain, ML, Stream Processing, Bots, Chaos**

- Finishing this Curriculum will put you at least at **SWE L5 level in the industry** and an equivalent of a **Master's degree in software engineering** (2600h), 97% of the web dev industry are just glorified CRUD apps with bloated UI and shitty UX.
```

listing 5: the seven stage curriculum and the line that states the 2600 hour estimate (source: defense-cookbook/vendored/modern-swe-library/readme.md, lines 338-346)

Both slices of the one file, the load-bearing lines in one view:

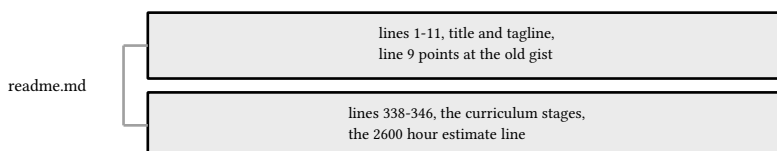


figure 75: the two load-bearing slices, line 9 for the star story, 338-346 for the hours

19.4. drills

100+ stars where, exactly?

On the gist the library lived on, which I deleted in January 2025 when the project was archived to GitHub, so it no longer exists to check. Today, measured on October 5: the repo has 9 stars and 4 forks, the old version gist it links on line 9 has 0 stars and 2 forks, and I enumerated every gist under both accounts to rule out another copy. What is real: one 1,296 line curated document, public, with a 2,600 hour study path inside it.

floor: book 15, ch 11 (interview defense cookbook, the repo evidence method)

2,600 hours is a strange number. Did you time it?

No, and the document says so if you read the section. It is the sum of per item estimates ordered so dependencies come first, 13 months at roughly 200 focused hours a month, the document even shows that arithmetic. An estimate budget, not a stopwatch log, and I say exactly that before anyone asks what I did with hour 2,317.

You archived it the day you created it. Is that an abandoned project?

It is a finished one, and the timestamps are right in the snapshot, created and pushed 2025-01-01, archived since. The document was complete, the gist before it ran from 2024-08 to 2024-12, and freezing a done thing beats pretending it needs maintenance. I still hand people sections of it, the foundation path is the one I actually followed toward the handbooks.

The routing under all three probes, history stated first, today’s numbers second:

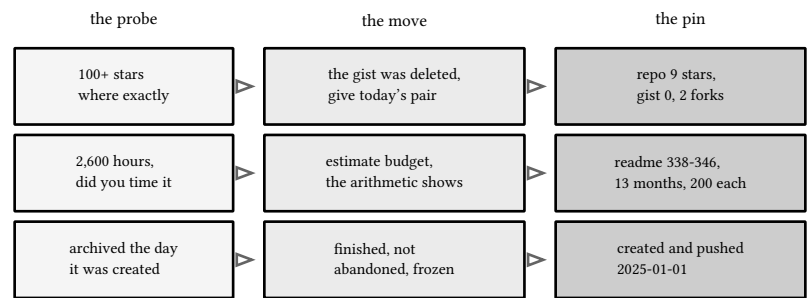


figure 76: the most rehearsed concede in the book, both homes covered before asked

PITFALL: the “and the gist?” follow-up

The concession must cover both homes before the interviewer finds the second one. Repo 9 stars, gist 0 stars, one fork, no other copy under either account, measured 2026-09-10 in coverage/gh/gist-stars.md. Volunteer all of it, because the worst version of this exchange is conceding the repo number and then being handed the gist.

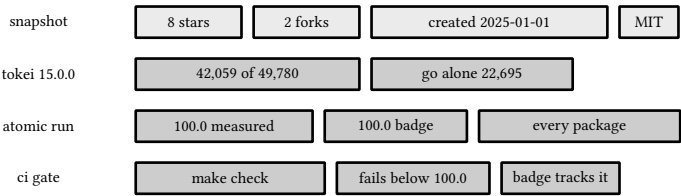
20. llm-tournament deep dive

The discipline project and the repo that proves the TDD tag is a fact rather than an adjective. It is also the repo whose size headline is historical, so this chapter opens with the audit the way the drill does: 42,059 code lines of 49,780 total, measured with tokei 15.0.0 over main at f046fb2, not the resume’s 100k. 100.0 percent total Go statement coverage from an independent atomic-mode run on go1.27.1, every package with statements at exactly 100.0, with the repo badge and a CI gate holding the same number. The 100k names the pre-python_service tree the line was written against, and the coverage figure, off at the 2026-09-10 pass, reads true again since the 2026-09-30 head removed python_service. Register rows repo-llm-sloc and repo-llm-coverage carry these measurements, contrib-llm carries the historical framing.

20.1. verified facts

fact	measured value
stars	8, measured 2026-10-05, snapshot coverage/gh/llm-tournament.json
forks	2, same snapshot
created	2025-01-01, same snapshot
last push	2026-09-30, head docs: fix stale code anchors and enforcement claims
primary languages	Go 819,771 bytes, CSS 184,793, HTML 146,765, Python 11,198
license	MIT, archived no
size, code lines	42,059 of 49,780 total, tokei 15.0.0, main @ f046fb2
size, go alone	22,695 code lines of 28,609 with comments and blanks
coverage, measured	100.0% total statements, atomic mode, go1.27.1, every package with statements at exactly 100.0
coverage, repo badge	100.0%, readme table all 100.0, make check ends with coverage-enforce
untested	none in Go; 5 Python files, 288 code lines, playground and tools glue, no tests

The audit compressed to one card, measured beside badged beside gate:



measured 100.0, badged 100.0, gate-enforced, all stated on request

figure 77: the llm-tournament fact card, measured, badged, and gate-enforced

20.2. the shape in thirty seconds

manual scoring,
no api keys, fully offline

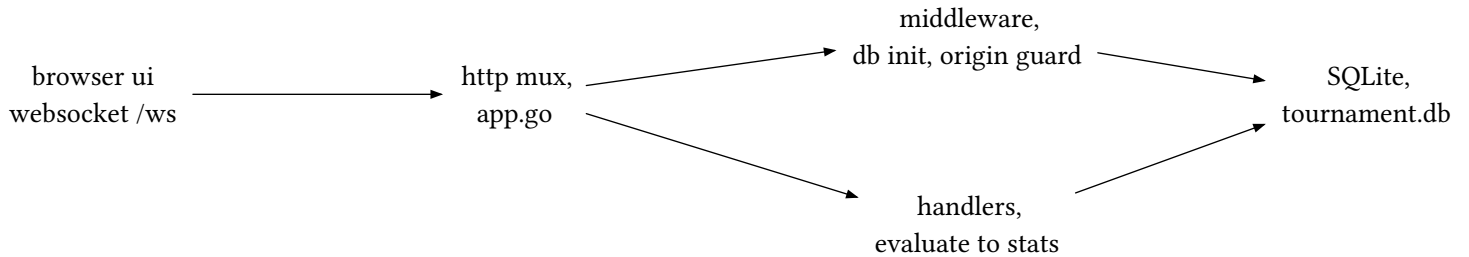


figure 78: llm-tournament request path, drawn from app.go, middleware, go.mod, the repo tree, and the coverage run

Five dependencies in go.mod, websocket, sqlite driver, an html sanitizer, a markdown renderer, text handling, and the whole tournament state lives in one SQLite file, every read scoped to the current suite through middleware/state.go. The 2026-09-30 head removed the automated evaluator and its job queue along with python_service, so the state layer is the durability story now: nothing in memory is authoritative, a restart is process death plus a page reload.

20.3. open this file first

app.go at the repo root, vendored, the wiring the whole service hangs off:

```
// SetupRoutes registers all routes on the given ServeMux
func SetupRoutes(mux *http.ServeMux) {
    for path, handler := range routes {
        mux.HandleFunc(path, handler)
    }
    mux.HandleFunc("/", router)
    mux.HandleFunc("/ws", middleware.HandleWebSocket)
    mux.Handle("/templates/", http.StripPrefix("/templates/", middleware.StaticFiles(http.Dir("templates"))))
    mux.Handle("/assets/", http.StripPrefix("/assets/", middleware.StaticFiles(http.Dir("assets"))))
}

// ServerHandler returns the fully wired HTTP handler: all routes behind the
// cross-origin guard that blocks browser-driven cross-site posts against the
// local server.
func ServerHandler() http.Handler {
    return middleware.CheckOrigin(NewServeMux())
}
```

listing 6: route registration, websocket endpoint, asset servers, the cross-origin guard (source: defense-cookbook/vendored/llm-tournament/app.go, lines 69-84)

Then the state layer, middleware/state.go, vendored:

```

func ReadResults() map[string]Result {
    suiteName := GetCurrentSuiteName()
    var suiteID int
    err := db.QueryRow("SELECT id FROM suites WHERE name = ?", suiteName).Scan(&suiteID)
    if err != nil {
        log.Printf("Error getting suite ID: %v", err)
        return make(map[string]Result)
    }

    // Get all prompts for this suite
    promptCountQuery := "SELECT COUNT(*) FROM prompts WHERE suite_id = ?"
    var promptCount int
    err = db.QueryRow(promptCountQuery, suiteID).Scan(&promptCount)
    if err != nil {
        log.Printf("Error counting prompts: %v", err)
        return make(map[string]Result)
    }

    // Get all models for this suite
    modelQuery := "SELECT id, name FROM models WHERE suite_id = ?"
    modelRows, err := db.Query(modelQuery, suiteID)
    if err != nil {
        log.Printf("Error querying models: %v", err)
        return make(map[string]Result)
    }
    defer func() { _ = modelRows.Close() }()

    results := make(map[string]Result)
    for modelRows.Next() {
        var modelID int
        var modelName string
        if err := modelRows.Scan(&modelID, &modelName); err != nil {
            log.Printf("Error scanning model: %v", err)
            continue
        }

        // Initialize scores array
        scores := make([]int, promptCount)

        // Get scores for this model

```

listing 7: ReadResults, the suite-scoped results grid read, every query parameterized (source: defense-cookbook/vendored/llm-tournament/middleware/state.go, lines 166-205)

And the manifest, go.mod, vendored, for the dependency count and the module shape:

```

module llm-tournament

go 1.27.1

require (
    github.com/gorilla/websocket v1.5.3
    github.com/mattn/go-sqlite3 v1.14.52
    github.com/microcosm-cc/bluemonday v1.0.27
    github.com/russross/blackfriday/v2 v2.1.0
    golang.org/x/text v0.42.0
)

require (
    github.com/aymerick/douceur v0.2.0 // indirect
    github.com/gorilla/css v1.0.1 // indirect
    golang.org/x/net v0.59.0 // indirect
)

```

listing 8: five direct dependencies, one module at the repo root, go 1.27.1 (source: defense-cookbook/vendored/llm-tournament/go.mod, lines 1-17)

Three files carry the whole walkthrough, wiring to state to manifest:

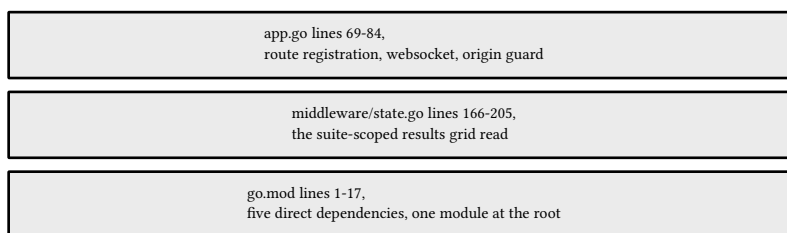


figure 79: three vendored slices, the wiring, the suite-scoped state, the five dependencies

20.4. drills

100% coverage on 100k lines, how is that measured and who audits it?

The size headline fails the audit and the coverage headline now passes it, so here is both. Measured with tokei 15.0.0 over main at f046fb2: 42,059 code lines of 49,780 total, Go alone 22,695, so 100k names the earlier tree the line was written against, the January head measured 46,545 of 62,058. Coverage, the repo's own command under atomic covermode on go1.27.1, the mode whose merged counts stay correct when tests run in parallel: 100.0 percent total statements, every package with statements at exactly 100.0, the repo's own badge at 100.0, and make check ends with coverage-enforce, so CI fails below 100. The python_service that landed after the line was written is what moved the number off 100 at the 2026-09-10 pass, and the 2026-09-30 head removed it. The history plus today's measurement, and every command here runs live during the call.

floor: book 3, ch 14 (go 1.27, testing), book 15, ch 11 (interview defense cookbook, the repo evidence method)

You keep everything in one SQLite file. What happens on restart?

Nothing in memory is authoritative. Suites, prompts, models, profiles, and scores are rows keyed by suite id, middleware holds the current suite name, and every handler read in state.go scopes its queries through it, so a restart is process death plus a page reload: the next request re-reads the same rows and renders the same grid. The same rows back the export and import buttons, which is the backup story, and the evaluator that used to own a job queue was removed at the 2026-09-30 head, the state layer is the whole durability story now.

Eight stars. Was it worth it?

Worth 42,059 code lines, the measured count, and yes. It was never promoted, its audience is me plus any interviewer, and what it holds is the discipline: 100.0 percent coverage that survives an independent atomic-mode run and a CI gate, suite-scoped state in one SQLite file, a fully offline arena with no api keys. Popularity was not the goal, the number that matters is held by the repo.

Each probe lands on a rerunnable command or a vendored slice:

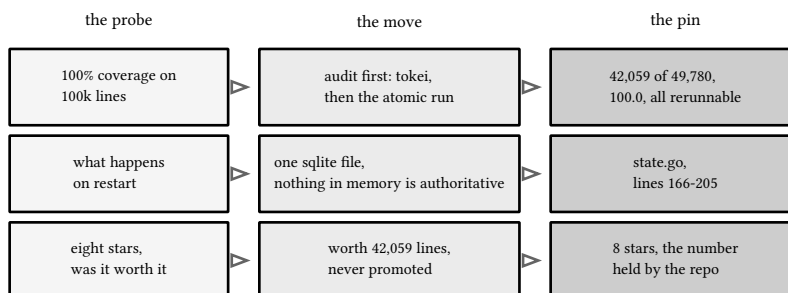


figure 80: the drills as routing, audit first, crash recovery, the worth concede

VERIFY: the numbers to be able to write on a whiteboard

42,059 of 49,780. Go 22,695. 100.0 measured, 100.0 badged, every package with statements at 100.0, CI gate enforcing the floor, 8 stars. All from coverage/gh/llm-tournament-measurements.md, all rerunnable in one command each.

21. go-edge-mesh deep dive

The public shadow of the Dikshatek edge mesh work, register row repo-mesh-stack, and the evidence that the southbound edge mesh bullet is a real architecture rather than a buzzword. It is a demo by design: fake sensors, a pseudo third party, one Go bootstrapper, and a full EdgeX compose stack underneath. What it proves in an interview is that I can walk every hop from a generated temperature reading to a Grafana panel.

21.1. verified facts

fact	measured value
stars	5, measured 2026-09-10, snapshot coverage/gh/go-edge-mesh.json
forks	3, same snapshot
created	2023-05-12, same snapshot
last push	2026-05-13, same snapshot
primary languages	Makefile 29,826 bytes, Go 22,712, Shell 11,126, Go Template 5,619
license	none, archived no
stack, from the description	EdgeX compose, Mosquitto MQTT, Telegraf, InfluxDB, Grafana
own code	bootstrapper_go, admin, sensor, web packages

The language mix is itself the honest shape: the orchestration outweighs the Go, because the value of the demo is the topology, not the code volume.

The card, with the byte split that gives the chapter its shape:

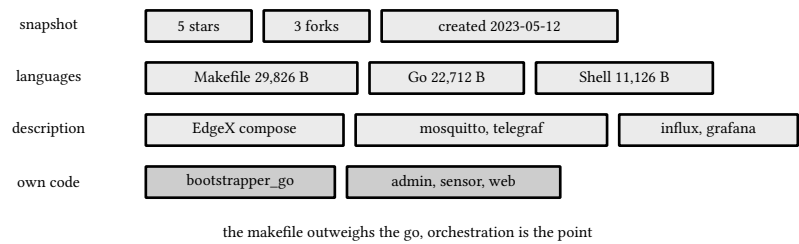


figure 81: the go-edge-mesh fact card, orchestration outweighing the go

21.2. the shape in thirty seconds

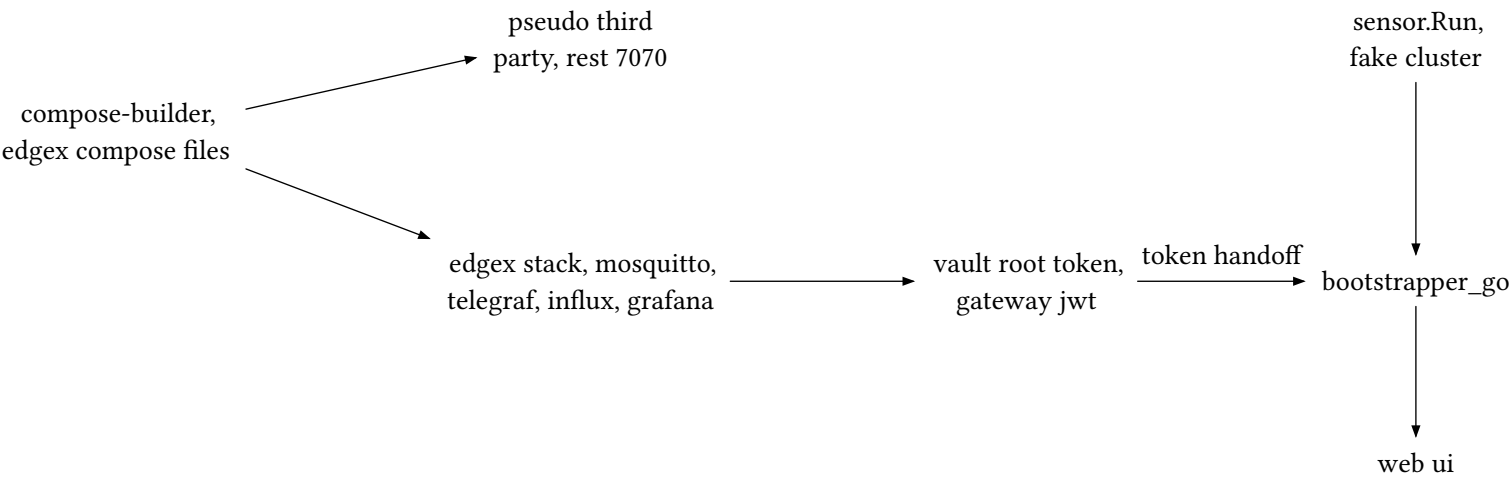


figure 82: go-edge-mesh run target, read off the vendored Makefile lines 69 to 81 and bootstrapper_go

The interesting hop is the security handoff: the run target pulls the root token out of the EdgeX vault volume and mints a gateway JWT, and the Go bootstrapper presents that bearer token when it registers the fake sensor cluster and posts events. Nothing in the demo bypasses the gateway.

21.3. open this file first

The root Makefile, vendored, because it is the executable documentation of the topology:

```
run:
# boot up all of the EdgeX's microservice containers
  docker-compose -p edgex -f docker-compose${NO_SECURITY}${UI}${ARM64}.yaml up -d ${SERVICES}
# start a fake third party REST API at localhost:7070
  -make start-thirdparty
# wait for them properly running
  sleep 15
# get the root token and transfer it to the bootstrapper
  docker run --rm -ti -v edgex_vault-config:/vault/config:ro alpine:latest cat /vault/config/assets/resp-
init.json | tee ./bootstrapper_go/common/root_token.json
# get the proper JWT token and transfer it to the bootstrapper
  make get-token | sed '1d;$d' | tee ./bootstrapper_go/common/gateway_jwt_token
# start the bootstrapper custom service
  -make start-bootstrapper
```

listing 9: the run target, edgex up, third party, vault token, jwt, bootstrapper, in order (source: defense-cookbook/vendored/go-edge-mesh/Makefile, lines 69-81)

Then the entire Go entry point, 19 lines, vendored:

```
package main

import (
    "bootstrapper_go/admin"
    "bootstrapper_go/sensor"
    "bootstrapper_go/web"
    "context"
    "log"
    "time"
)

func main() {
    log.SetFlags(log.Ldate | log.Ltime | log.Lshortfile)
    time.Sleep(time.Second * 1)
    ctx := context.Background()
    admin.Bootstrap(ctx)
    go sensor.Run(ctx)
    web.Start(ctx)
}
```

listing 10: bootstrap the cluster, run the sensor goroutine, serve the web ui (source: defense-cookbook/vendored/go-edge-mesh/bootstrapper_go/main.go, lines 1-19)

And how a reading becomes an event, vendored:

```

func Run(ctx context.Context) {
    for {
        temperature, humidity = generateSensorData(temperature, humidity)
        sendData(ctx, "combine", temperature, humidity)
        log.Printf("sent: temperature %dC, humidity %d%%", temperature, humidity)
        time.Sleep(common.Interval)
    }
}

func sendData(ctx context.Context, resource string, data1 int64, data2 int64) {
    client := oauth2.NewClient(ctx, oauth2.StaticTokenSource(&oauth2.Token{
        AccessToken: common.PreProcessToken(common.GatewayJwtTokenWithCRLF),
        TokenType:   "Bearer",
    }))
    eventUuid := uuid.NewString()
    readingUuid1 := uuid.NewString()
    readingUuid2 := uuid.NewString()
    timestamp := time.Now().UnixNano()
    payload := fmt.Sprintf(common.ResourceSchema, resource, eventUuid, timestamp, readingUuid1, timestamp,
        data1, readingUuid2, timestamp, data2)
    body := bytes.NewBufferString(payload)
    resp, err := client.Post(resourceUrl+"/"+resource, payloadType, body)
    common.CheckFatal(err)
    defer resp.Body.Close()
    log.Println("resource payload:\n\t", payload)
}

func generateSensorData(inputTemperature, inputHumidity int64) (int64, int64) {
    temperatureMin := inputTemperature - 2
    if temperatureMin < common.TemperatureLowerBound {
        temperatureMin = common.TemperatureLowerBound
    }
    temperatureMax := inputTemperature + 2
    if temperatureMax > common.TemperatureUpperBound {
        temperatureMax = common.TemperatureUpperBound
    }
    humidityMin := inputHumidity - 4
    if humidityMin < common.HumidityLowerBound {
        humidityMin = common.HumidityLowerBound
    }
    humidityMax := inputHumidity + 4
    if humidityMax > common.HumidityUpperBound {
        humidityMax = common.HumidityUpperBound
    }
    temperature := rd.Int63n(temperatureMax-temperatureMin+1) + temperatureMin
    humidity := rd.Int63n(humidityMax-humidityMin+1) + humidityMin
    return temperature, humidity
}

```

listing 11: bounded random walk per reading, oauth2 bearer, post to the gateway event route (source: defense-cookbook/vendored/go-edge-mesh/bootstrapper_go/sensor/fake_data.go, lines 26-72)

The documentation is executable, the route in three files:

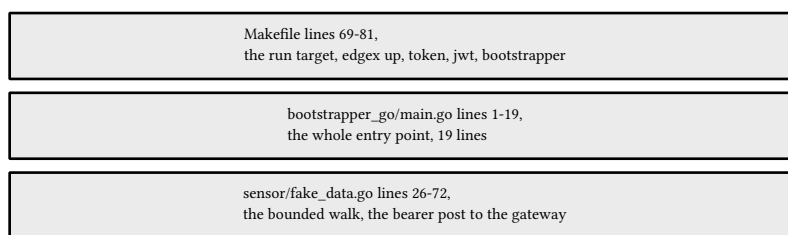


figure 83: the executable documentation route, run target, entry point, posting loop

21.4. drills

This is a demo with fake sensors. What did the real system look like?

Same shape, different edges. The demo runs a fake sensor cluster through the real EdgeX stack, mosquitto, telegraf, influxdb, grafana, with the same gateway auth the production mesh used, vault root token to gateway JWT to bearer calls. Production swapped the fake data source for real site hardware and the pseudo third party for an actual backend, and added the parts a demo cannot fake, partition tolerance on flaky site links and reconcile after reconnect. I scope it exactly that way: the public repo proves the topology and the auth path, the production claims rest on client-confidential work.

floor: book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals)

Why is the biggest language in the repo a Makefile?

Because the demo's job is to boot a multi-service edge topology reproducibly, and that logic is orchestration: compose files per service, an arm64 variant, a no-security variant, token extraction from the vault volume. The Go is deliberately thin, register a device profile, post events, serve a status page. If the Go were thick the demo would be about my code instead of about the edge mesh, and it exists to be about the mesh.

Walk me through one temperature reading end to end.

The sensor goroutine does a bounded random walk around the last value, plus or minus two degrees clamped to configured bounds, builds the EdgeX event payload with fresh uuids and a nanosecond timestamp, and posts it to the gateway event route with the JWT as a bearer token through an oauth2 client. From there it is EdgeX plumbing: the event rides the message bus, telegraf picks it up, influxdb stores it, grafana panels it. The whole path is in three vendored files you can read in under a minute.

Each drill concedes the demo, then lands on what it proves:

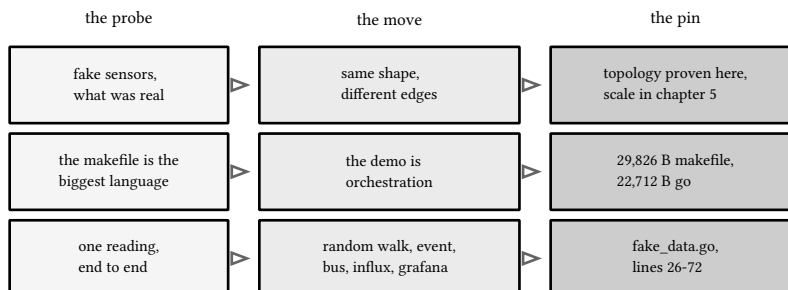


figure 84: demo scoping, orchestration over code, one reading end to end

VERIFY: the floor under this chapter

The demo repo answers topology questions, book 11, ch 10 (design patterns, concurrency, and distributed systems, distributed fundamentals) answers the systems questions that follow, CAP tradeoffs, delivery semantics, why this mesh did not need Raft. Rehearse the handoff: evidence [here](#), whiteboard [there](#).

22. order-demo deep dive

The NTQ orders story in miniature, register row repo-order-stack. The production work was gRPC and Protobuf over PostgreSQL with MongoDB pipelines, client confidential, and this repo is the public, runnable version of the same discipline: a small order service where the sqlc queries, the authorization rules, and the token layer are all readable in an afternoon. It is linked from the resume as the demo for that stint, and the honest framing is that it demonstrates the stack, not the scale.

22.1. verified facts

fact	measured value
stars	6, measured 2026-09-10, snapshot coverage/gh/order-demo.json
forks	2, same snapshot
created	2021-12-20, same snapshot
last push	2026-08-31, same snapshot
primary languages	Go 70,162 bytes, Shell 4,449, Makefile 1,488, Dockerfile 807
license	none, archived no
shape	gin handlers over sqlc generated stores, PASETO auth
repo tree	api, db/migration, db/query, db/sqlc, token, util packages

The card, the whole stack pattern at a glance:

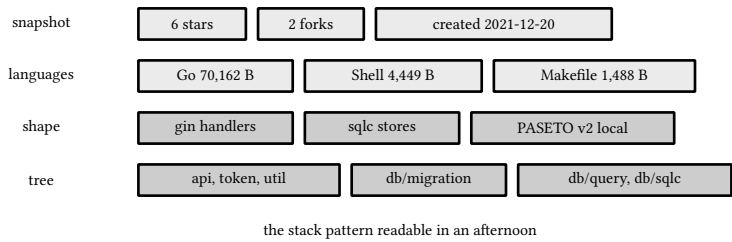


figure 85: the order-demo fact card, the stack pattern in one strip

22.2. the shape in thirty seconds

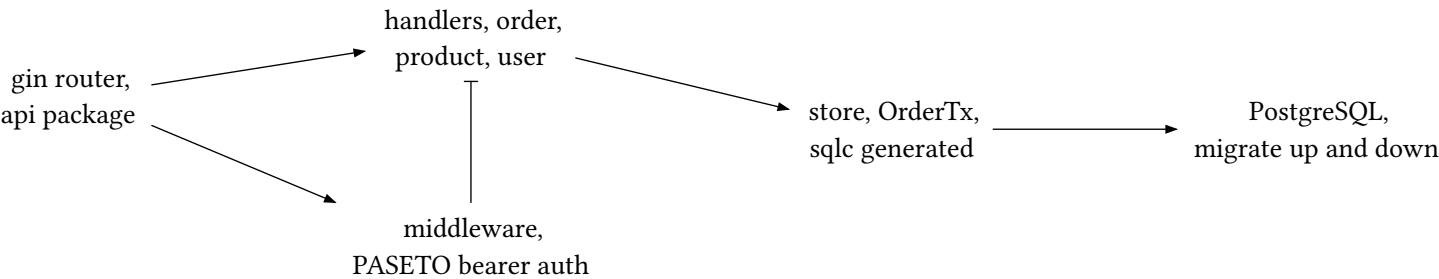


figure 86: order-demo wiring, read from the vendored main.go, api/order.go, and the repo tree

Two choices carry the design. Queries are written in sql and compiled to typed Go, so the database contract is checkable in review instead of stringly at runtime. Tokens are PASETO v2 local, a symmetric authenticated cipher, rather than a hand-rolled JWT handler, and the repo keeps a jwt_maker beside it behind the same Maker interface, so the choice is a config line, not a rewrite.

22.3. open this file first

main.go, vendored, the whole dependency chain in 32 lines:

```
package main

import (
    "database/sql"
    "log"
    "order-demo/api"
    db "order-demo/db/sqlc"
    "order-demo/util"

    _ "github.com/lib/pq"
)

func main() {
    log.SetFlags(log.LstdFlags | log.Lshortfile)
    config, err := util.LoadConfig(".")
    if err != nil {
        log.Fatal("cannot load config", err)
    }
    conn, err := sql.Open(config.DBDriver, config.DBSource)
    if err != nil {
        log.Fatal("cannot connect to db", err)
    }
    store := db.NewStore(conn)
    server, err := api.NewServer(config, store)
    if err != nil {
        log.Fatal("cannot create server", err)
    }
    err = server.Start(config.ServerAddress)
    if err != nil {
        log.Fatal("cannot start server", err)
    }
}
```

listing 12: config to connection to store to server, the entire composition root (source: defense-cookbook/vendored/order-demo/main.go, lines 1-32)

The authorization logic the stint is known for, vendored:

```
func (server *Server) listOrders(ctx *gin.Context) {
    var req listOrdersRequest
    if err := ctx.ShouldBindQuery(&req); err != nil {
        ctx.JSON(http.StatusBadRequest, ErrorResponse(err))
        return
    }
    // Enforce authorization Rule0 and Rule6
    authPayload := ctx.MustGet(authorizationPayloadKey).(*token.Payload)
    var orders []db.Order
    var err error
    if authPayload.Username == "admin" {
        arg := db.ListOrdersParams{
            Limit:  req.PageSize,
            Offset: (req.PageID - 1) * req.PageSize,
        }
        orders, err = server.store.ListOrders(ctx, arg)
    } else {
        arg := db.ListOrdersByOwnerParams{
            Limit:  req.PageSize,
            Offset: (req.PageID - 1) * req.PageSize,
            Owner:  authPayload.Username,
        }
        orders, err = server.store.ListOrdersByOwner(ctx, arg)
    }
    if err != nil {
        ctx.JSON(http.StatusInternalServerError, ErrorResponse(err))
        return
    }
    ctx.JSON(http.StatusOK, orders)
}
```

listing 13: admin sees all orders, everyone else is scoped to their own by the token payload (source: defense-cookbook/vendored/order-demo/api/order.go, lines 17-46)

And the token layer, vendored:

```

func NewPasetoMaker(symmetricKey string) (Maker, error) {
    if len(symmetricKey) != chacha20poly1305.KeySize {
        return nil, fmt.Errorf("invalid key size: must be exactly %d characters", chacha20poly1305.KeySize)
    }
    maker := &PasetoMaker{
        paseto:      paseto.NewV2(),
        symmetricKey: []byte(symmetricKey),
    }
    return maker, nil
}

func (maker *PasetoMaker) CreateToken(username string, duration time.Duration) (string, error) {
    payload, err := NewPayload(username, duration)
    if err != nil {
        return "", err
    }
    return maker.paseto.Encrypt(maker.symmetricKey, payload, nil)
}

func (maker *PasetoMaker) VerifyToken(token string) (*Payload, error) {
    payload := &Payload{}
    err := maker.paseto.Decrypt(token, maker.symmetricKey, payload, nil)
    if err != nil {
        return nil, ErrInvalidToken
    }
    err = payload.Valid()
    if err != nil {
        return nil, err
    }
    return payload, nil
}

```

listing 14: chacha20poly1305 key size enforced, encrypt and decrypt behind the Maker interface (source: defense-cookbook/vendored/order-demo/token/paseto_maker.go, lines 16-46)

The walkthrough is three files, the wiring, the rule, the token:

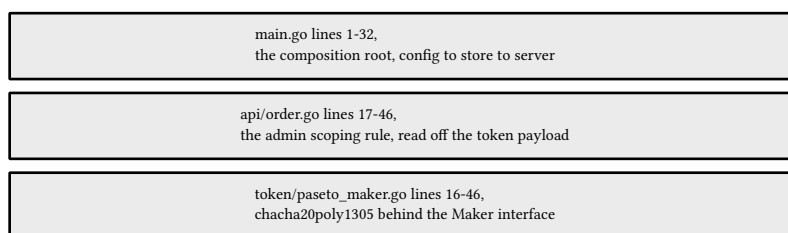


figure 87: the composition root, the scoping rule, the maker over chacha20poly1305

22.4. drills

Your resume says gRPC and Protobuf. This repo is gin and JSON.

Correct, and I keep them separate. The production orders service at NTQ was gRPC with Protobuf contracts and MongoDB aggregation behind it, and none of that is publishable. The demo repo is the same discipline in a public shape: typed data access through sqlc, authorization rules enforced in the handler from the token payload, transactional order creation in OrderTx, PASETO instead of raw JWT. It demonstrates the stack patterns, it does not demonstrate the scale, and the migration work I claim is a separate bullet with its own evidence.

floor: book 3, ch 13 (go 1.27, stdlib tour 2: json/v2, uuid, net/http, crypto, simd, os, io, testing)

Why PASETO over JWT?

JWT's spec lets implementers pick weak algorithms, and the classic failure is trusting an alg header an attacker controls. PASETO v2 local gives one authenticated cipher, chacha20poly1305 here with the key size checked at construction, and the payload is encrypted, not just signed, so nothing leaks in the token. The repo also has a `jwt_maker` behind the same `Maker` interface, which is the honest way to compare them: the interface makes the swap a config choice instead of a rewrite.

Where does an order actually become a database row?

`OrderTx` in the store. The handler binds and validates the request, loads the user from the token payload and the product by id, then hands both to `OrderTx`, which runs the order and its items inside one database transaction, so a short stock or a foreign key miss rolls the whole thing back. The pq error code switch in the handler maps `foreign_key_violation` and `unique_violation` to a 403 rather than a 500, which is the kind of detail review actually catches.

Each drill keeps the demo and the production claim apart, then pins the location:

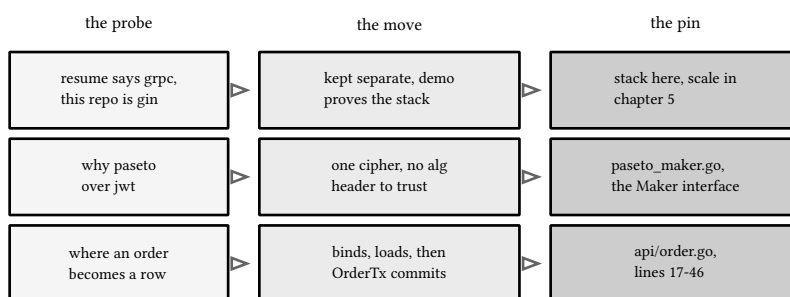


figure 88: stack not scale, paseto not jwt, the row inside `ordertx`

VERIFY: the thirty second locate drill

Repo tree for the package layout, `main.go` for the composition root, `api/order.go` for the authorization rules, `token/paseto_maker.go` for the auth layer. The production-scale claims stay in chapter 5, this chapter only has to prove the stack is real.

23. FlowerShop deep dive

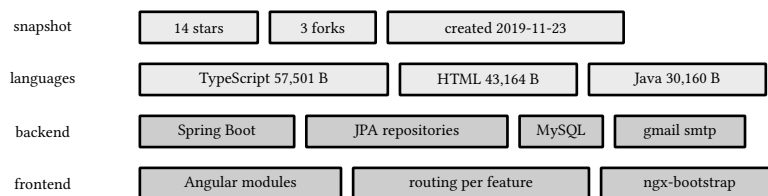
The oldest repo in the set and the PALTech evidence, register row repo-flower-stack. Created 2019, when the whitelabel exchange work was the day job, and kept public since. It is a full-features storefront, Spring Boot behind an Angular frontend, and its value in a defense is twofold: it shows I was building complete vertical slices from the first job on, and its age makes the chronology claims of chapter 6 checkable against commit history.

23.1. verified facts

fact	measured value
stars	14, measured 2026-09-10, snapshot coverage/gh/FlowerShop.json
forks	3, same snapshot
created	2019-11-23, same snapshot
last push	2026-05-13, same snapshot
primary languages	TypeScript 57,501 bytes, HTML 43,164, Java 30,160, SCSS 3,113, JavaScript 1,327
license	none, archived no
backend	Spring Boot, JPA repositories, MySQL, gmail smtp
frontend	Angular modules, routing per feature, ngx-bootstrap, translate

The byte split tells the truth about where the work went: the frontend outweighs the backend, because a storefront is mostly frontend, and the Java that exists is a clean repeated vertical pattern per entity.

The card, the byte split front and center:



the bytes say the work went to the frontend

figure 89: the flowershop fact card, the byte split saying where the work went

23.2. the shape in thirty seconds

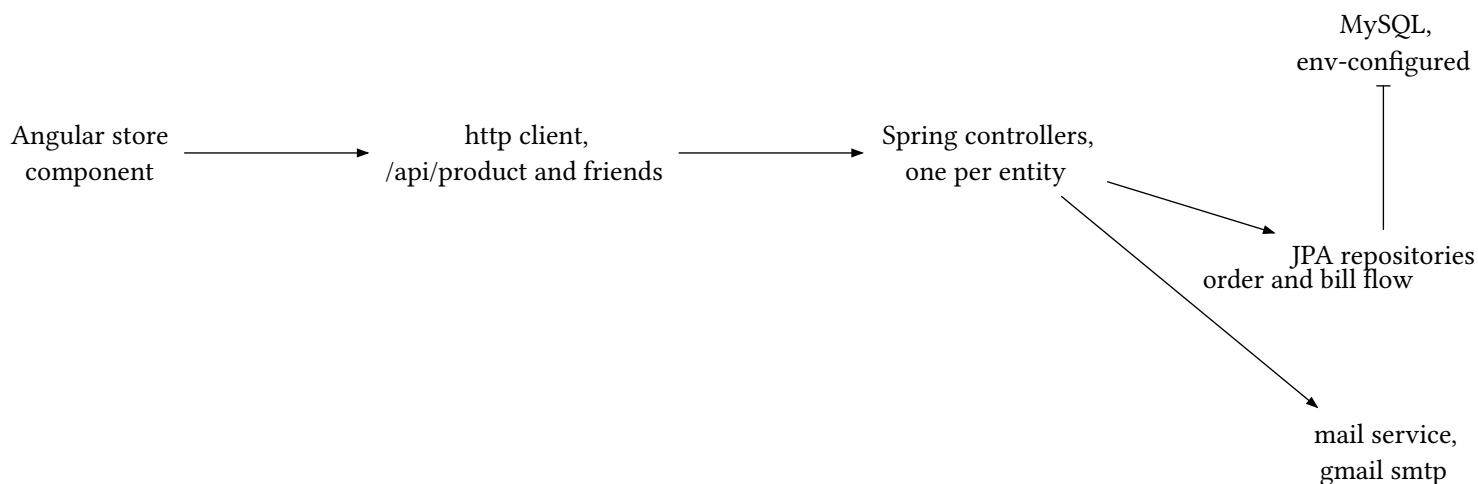


figure 90: FlowerShop request path, read from the vendored controller, component, and application.yml

Each domain entity, product, category, type, bill, user, gets the same triple: an entity class, a repository interface, a controller. The frontend mirrors it with a module and a routed component per feature. That repetition is the architecture, unglamorous and exactly what a whitelabel shop ships.

23.3. open this file first

The backend pattern in one file, vendored:

```
@RestController
@RequestMapping("/api/product")
public class ProductController {
    private ProductRepository productRepository;

    public ProductController(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        return ResponseEntity.ok(productRepository.findAll());
    }

    @PostMapping
    public ResponseEntity<List<Product>> createMany(@RequestBody List<Product> categories) {
        return ResponseEntity.ok(productRepository.saveAll(categories));
    }

    @DeleteMapping
    public ResponseEntity<?> deleteMany(@RequestBody(required = false) List<Long> ids) {
        if (ids == null) {
            productRepository.deleteAll();
            return ResponseEntity.ok().build();
        }
        productRepository.deleteAll(productRepository.findAllById(ids));
        return ResponseEntity.ok().build();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        Optional<Product> product = productRepository.findById(id);
        if (product.isEmpty()) {
            return ResponseEntity.badRequest().build();
        }
        return ResponseEntity.ok(product.get());
    }

    @PostMapping("/create")
    public ResponseEntity<Product> create(@RequestBody Product product) {
        return ResponseEntity.ok(product);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id, @RequestBody Product product) {
        if (productRepository.findById(id).isEmpty()) {
            return ResponseEntity.badRequest().build();
        }
    }
```

```

        return ResponseEntity.ok(productRepository.save(product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> delete(@PathVariable Long id) {
        if (productRepository.findById(id).isEmpty()) {
            return ResponseEntity.badRequest().build();
        }
        productRepository.deleteById(id);
        return ResponseEntity.ok().build();
    }
}

```

listing 15: constructor injection over the repository, the full crud surface every entity shares (source: defense-cookbook/vendored/FlowerShop/src/main/java/com/lavantien/flowershop/api/product/ProductController.java, lines 9-68)

The configuration, vendored, one screen:

```

spring:
  datasource:
    url: jdbc:mysql://${LOCAL_MYSQL_DB_HOST}:${LOCAL_MYSQL_DB_PORT}/flowershop
    username: ${LOCAL_MYSQL_DB_USERNAME}
    password: ${LOCAL_MYSQL_DB_PASSWORD}
  jpa:
    properties:
      hibernate:
        dialect: org.hibernate.dialect.${LOCAL_MYSQL_DB_DIALECT}
    hibernate:
      ddl-auto: update
  mail:
    host: smtp.gmail.com
    username: ${MAIL_SENDER_EMAIL}
    password: ${MAIL_SENDER_PASSWORD}
    port: 587
    properties:
      mail:
        smtp:
          auth: true
          starttls:

```

listing 16: MySQL through environment variables, hibernate ddl-auto, gmail smtp with starttls (source: defense-cookbook/vendored/FlowerShop/src/main/resources/application.yml, lines 1-21)

And the storefront component that consumes it, vendored:

```

getProducts() {
  this.http.get<Product[]>('/api/product').subscribe(data => {
    if (!!data) {
      this.products = data;
      this.products.forEach(product => {
        product.imageUrl = product.imageUrl ? atob(product.imageUrl) : '';
        product.price = this.dataTranslateService.getPrice(product.price, 'vi');
        // TODO: Translate product name
        // TODO: Translate product production
      });
      this.searchResults = data;
      this.paging(this.searchResults);
    }
  }, error => {
    console.log(`Error: ${error}`);
    this.products = [];
  }, () => {
  });
}

```

listing 17: fetch products, decode base64 image urls, seed the search and paging state (source: defense-cookbook/vendored/FlowerShop/frontend/src/app/store/store.component.ts, lines 84-102)

One file per lesson, the pattern, the config, the consumer:

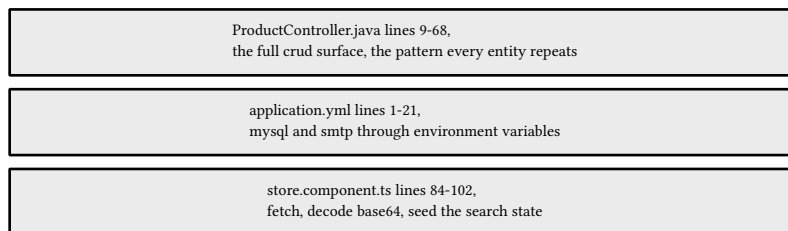


figure 91: the repeated triple in one controller, env-var config, the consumer

23.4. drills

This is a 2019 student-looking app. Why is it on your resume?

Because the PALTech bullet it evidences is from that era, whitelabel platforms with Angular frontends, and the repo's own timestamps back the chronology, created 2019-11-23, pushes since. It is not the strongest thing on the page and I do not present it as the strongest. It is the honest artifact of the first job, complete vertical slices, and it has stayed public for seven years, which is its own kind of answer.

Where is the exchange work? You said whitelabel trading platforms.

Client confidential, and FlowerShop is the public neighbor, not the exchange itself. What I can show from this repo is the same shape at shop scale: Angular modules per feature, Spring controllers over JPA repositories, MySQL, mail integration, decimal prices rendered through a translate service. The trading platform adds order books and compliance, none of which I can publish, so the exchange claims rest on the interview conversation and references, and I say so plainly.

floor: book 11, ch 15 (design patterns, concurrency, and distributed systems, resilience)

What would you change if you rebuilt it today?

Plenty, and the code shows its age honestly. Tokens in localStorage read through atob in the component is the one I would fix first, that is base64, not security, and session state belongs in an httpOnly cookie behind a real auth endpoint. Client-side paging over the full product list would move to server paging. The JPA repositories with no DTO layer would get projections. The architecture is fine, the 2019 browser habits are what date it.

Each drill answers with the timestamps or the code, not the claim:

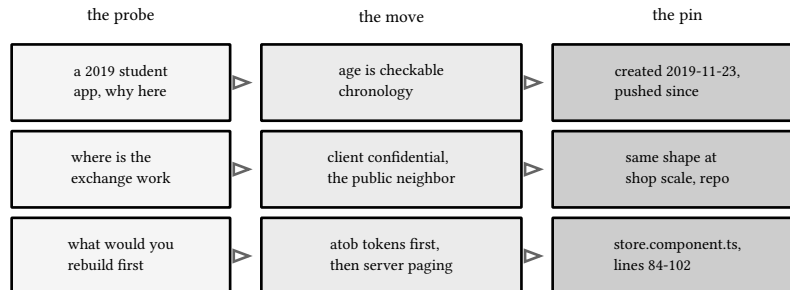


figure 92: age as chronology, the confidential neighbor, the rebuild order

NOTE: the age is the asset

The hostile version of the age question is “old code, embarrassing?”. The rehearsed turn: old public code is checkable chronology, and a seven year old repo with no secrets in it, env-var configured from the start, is what disciplined looks like in a first job.

24. SumoBot deep dive

The participations claim, register row repo-sumo-firmware, and the oldest code I own: embedded C for an 8051-family AT89S52, written for the UIT Sumo Robot 2017 competition the repo description says it won at first position. One source file, 329 lines, every pin and timer visible. The deep dive is short because the artifact is short, and the honest drill is to concede exactly what the firmware does and does not contain.

24.1. verified facts

fact	measured value
stars	6, measured 2026-09-10, snapshot coverage/gh/sumobot.json
forks	2, same snapshot
created	2019-10-02, same snapshot
last push	2026-08-31, same snapshot
primary languages	C, 6,074 bytes, the whole codebase
target	AT89S52, 8051 family, regx52.h, Keil-style C
firmware size	one file, sumobot.c, 329 lines vendored
repo extras	Proteus project, hex build, build artifacts, 2017 photos

The card, the whole artifact in six kilobytes:

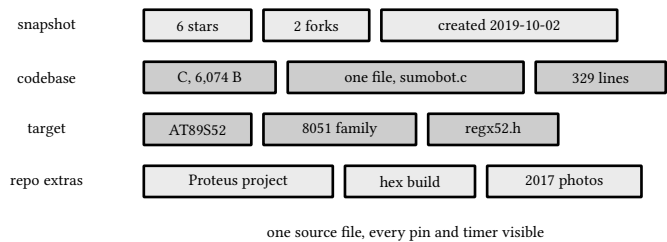


figure 93: the sumobot fact card, one file, six kilobytes, every pin and timer visible

24.2. the shape in thirty seconds

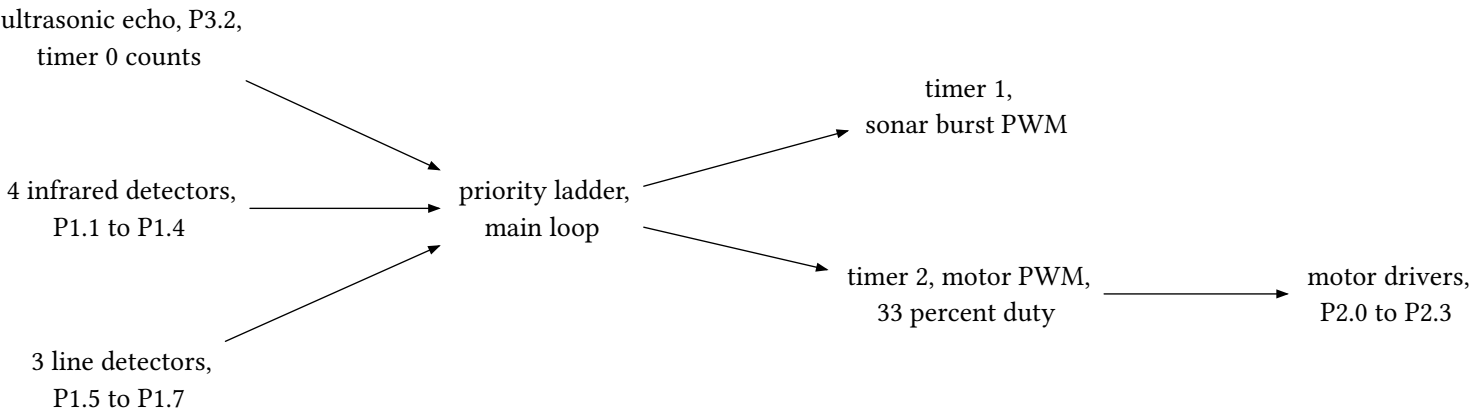


figure 94: sumobot.c control loop, read from the vendored firmware, sensors on the left, actuators on the right

The strategy is a fixed-priority reflex ladder polled forever: the white line outranks everything, enemy contact outranks search, and the sonar only fires when nothing else has. Timers do the only real math on the chip: timer 2 reload values set the motor duty cycle, timer 1 drives the sonar burst, timer 0 measures the echo width into DPTR, and a single threshold, DPTR below 3000, decides charge.

24.3. open this file first

sumobot.c, vendored whole, the only code in the repo. The pin map is the bit masking claim made concrete:

```
sbit PWM_MOTOR = P2 ^ 7; //timer2, PWM for motors
sbit PWM_SONIC = P2 ^ 6; //timer1, PWM for sonic
sbit ECHO = P3 ^ 2; //ultrasonic sensor echo pin

sbit B0 = P2 ^ 0; //motor B
sbit B1 = P2 ^ 1;
sbit A0 = P2 ^ 2; //motor A
sbit A1 = P2 ^ 3;

sbit LFL = P1 ^ 5; //line detector at front, left
sbit LFR = P1 ^ 6; //line detector at front, right
sbit LB = P1 ^ 7; //line detector at back, left

sbit IF = P1 ^ 1; //infrared enemy detector at front
sbit IL = P1 ^ 2; //infrared enemy detector at left
sbit IR = P1 ^ 3; //infrared enemy detector at right
sbit IB = P1 ^ 4; //infrared enemy detector at back
```

listing 18: sbit-declared pins, single bits of ports 1, 2, and 3, the 8051 way of bit masking (source: defense-cookbook/vendored/sumobot/sumobot.c, lines 4-20)

The brain, the ladder itself:

```

for (;;) {
    //line detected
    if (!LFL) {
        run_back();
        PWM2_max_speed();
        delayms(750);
        PWM2_less_speed();
        turn_r();
    } else if (!LFR) {
        run_back();
        PWM2_max_speed();
        delayms(750);
        PWM2_less_speed();
        turn_l();
    } else if (!LB || !IF) { //line at back or enemy in front
        run_forward();
        PWM2_max_speed();
    } else if (!IL) {
        leftIRres();
        PWM2_max_speed();
    } else if (!IR) {
        rightIRres();
        PWM2_max_speed();
    } else if (!IB) { //enemy kiss my ass
        spin_r();
        PWM2_max_speed();
    } else if (ECHO && !sent) { //sonic send pulses
        TH0 = TL0 = 0x00;
        TR0 = 1;
        sent = 1;
    } else if (!ECHO && sent) { //return pulses recived
        TR0 = 0;
        TF0 = 0;
        sent = 0;
        DPH = TH0;
        DPL = TL0;
        if (DPTR < 3000) {
            run_forward();
            PWM2_max_speed();
        }
    }
}
}

```

listing 19: the forever loop, line first, then contact, then sonar charge (source: defense-cookbook/vendored/sumobot/sumobot.c, lines 71-112)

And the only actuator trick, software PWM by timer reload:

```

void PWM2_start(unsigned ck) {
    T2MOD = 0x00;
    T2CON = 0x00;
    t2 = ck;
    T2on = (unsigned long)t2 * PERCENT_SPEED / 100;
    T2off = t2 - T2on;
    T2on_high_reload = (65536 - T2on) >> 8;
    T2on_low_reload = (65536 - T2on) & 0x00FF;
    T2off_high_reload = (65536 - T2off) >> 8;
    T2off_low_reload = (65536 - T2off) & 0x00FF;
    if (!EA) {
        EA = 1;
    }
    PWM2_less_speed();
    RCAP2H = T2on_high_reload;
    RCAP2L = T2on_low_reload;
    TR2 = 1;
}

```

listing 20: duty cycle as on and off reload values, the 33 percent constant from line 24 (source: defense-cookbook/vendored/sumobot/sumobot.c, lines 116-133)

The three slices, pins, ladder, duty cycle, all one file:

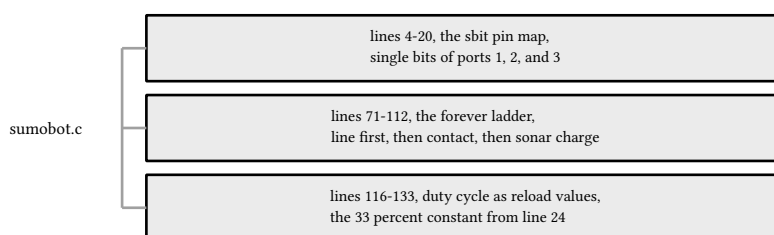


figure 95: sumobot.c 329 lines, the pin map, the ladder, the duty cycle reloads

24.4. drills

Your resume says fuzzy logic and PID. I see if-else chains.

You see the shipped firmware, and you are right about what is in it: a fixed-priority ladder, timer reload PWM, and one sonar distance threshold. The fuzzy logic and PID language describes the tuning work around it, how the 33 percent cruise duty, the 750 ms retreat, and the DPTR threshold of 3000 were chosen against real opponents, membership in my head rather than on the chip. The honest scope is that the committed file implements a tuned reflex controller, and I should say tuned, not fuzzy, when the file is on screen.

floor: book 14, ch 5 (game systems and architecture, finite state machines)

DPTR below 3000, what does that number mean?

It is the charge distance. Timer 0 counts echo time into TH0 and TL0, the loop copies them into DPH and DPL, and the sfr16 alias at 0x82 reads the pair as one 16-bit DPTR value, where 3000 counts is roughly where an opponent is close enough that ramming beats searching. It was tuned by testing against the practice ring until it stopped charging late. Everything in this firmware is one calibrated constant away from the hardware, that is what 8051 work is.

Why does a 2017 robot repo still get pushes in 2026?

Housekeeping, not firmware changes. The snapshot's recent commits are chore messages, sync system instructions and a clean, the code and the photos are from the 2017 build. The competition claim itself is pinned by the repo description, first position at UIT Sumo Robot 2017, and the photos in the repo, not by the push history.

The tuning lived off the chip, the file runs the ladder:

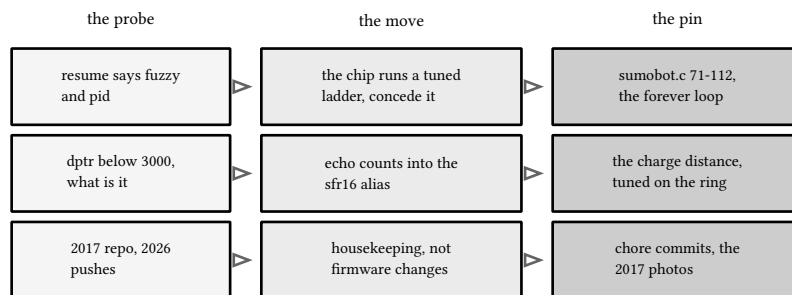


figure 96: on the chip a tuned ladder, off the chip the tuning, then housekeeping

PITFALL: the oldest code trap

The hostile move on a 2017 repo is grading it against 2026 practice. Concede the vintage, then re-anchor: the concepts in this file, fixed-priority arbitration, timer-driven actuation, sensor fusion on eight input bits, are the same concepts book 14, ch 5 (game systems and architecture, finite state machines) teaches with entity state machines. The chip got smaller, the ladder stayed.

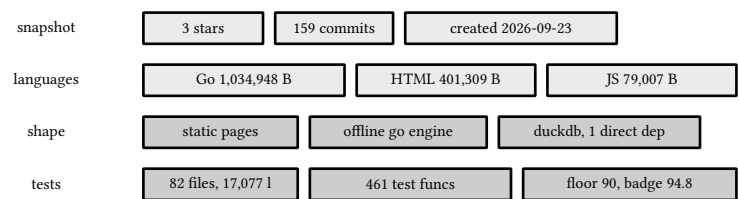
25. dota-helper deep dive

The hobbies line converted into repo evidence, register row repo-dota-helper. The resume’s hobbies line reads Dota 2 as draft and macro strategy, and this repo is that hobby executed as an engineer: an offline pick simulator and hero-pool guide for patch 7.41f, static pages with zero runtime deps over an offline Go engine that mines divine-immortal STRATZ drafts into DuckDB and emits generated JS the pages import. The honest framing is the conversion itself: a hobby row becomes defensible evidence exactly when it ships a public artifact with real statistics inside it, and the scale stays hobby scale. The repo is 12 days old, one user, patch-bounded, and it proves method, not production. The hobby line itself is defended in volume 0, ch 9 (resume defense analysis, hobbies line), this chapter defends the repo.

25.1. verified facts

fact	measured value
stars	3, forks 0, measured 2026-10-05, snapshot coverage/gh/dota-helper.json
created	2026-09-23, last push 2026-10-05, same snapshot
commits	159 over 9 active days at HEAD f30dc68 on tag v0.6, counted 2026-10-05 with git rev-list -count HEAD
primary languages	Go 1,034,948 bytes, HTML 401,309, JavaScript 79,007, PowerShell 63,331, CSS 19,521, Python 18,164, Makefile 11,629, same snapshot
license	MIT, archived no
shape	static pages, offline go engine over duckdb, generated js the pages import
repo tree	engine (ingest, mine, analytics, emit, eval, order, stats, builds, config, gates, store), picker, guide, scripts, ref, config.json + content.json hub
engine module	poolguide, go 1.27.0, 1 direct require (duckdb-go v2.10505.0), 22 indirect, read 2026-10-05 from engine/go.mod
source vs tests	67 source files 14,773 lines, 82 test files 17,077 lines, 461 test functions, counted 2026-10-05 with find and grep
coverage	floor 90.0 at scripts/coverage-badge.py line 8, badge 94.8% from the ci-committed coverage.json, cited 2026-10-05, not re-run here, the dock-erized suite runs 30-60m
concurrency	0 goroutine statements and 0 sync imports in engine non-test source, the 2 sync imports sit in test servers, grep 2026-10-05
fuzz targets	0, grep for func Fuzz 2026-10-05
config hub	config.json, 25 top-level keys, pool 53 entries, 5 roles, patch 7.41f at line 2, read 2026-10-05
the twin	picker-score.js 232 lines with 20 node tests, emitted picker-data-generated.js 252 lines, roster 127 heroes, measured 2026-10-05

The card, the whole architecture at a glance:



a 12-day-old hobby repo with a real statistics engine

figure 97: the dota-helper fact card, a mined pipeline in one strip

25.2. the shape in thirty seconds

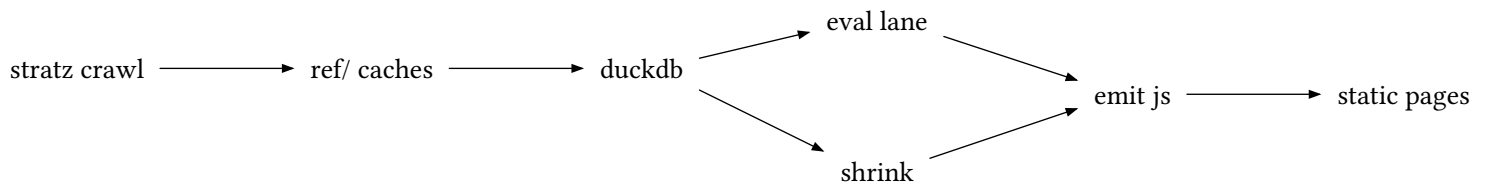


figure 98: dota-helper wiring, read from the readme data flow, the repo tree, and the vendored files

Three choices carry the design. The engine is offline and serial: every build and test runs through make targets wrapping the golang:1.27 docker image because the duckdb cgo driver has no supported windows toolchain, and the engine source contains no goroutine statement and no sync import, so the whole refresh pipeline is single-owner by construction. The hub is authored once: config.json holds pool, roles, scope, and every scoring constant, pool counts and hero lists are always derived from it, and a single writer re-sorts it under guards, no hand edit touches it. Two authored keys shape the picker's seating: sharedRolePools, in the hub since v0.3, seats the candidate fields, pos 1 to 3 in core and pos 4 to 5 in support with full interchange inside a field, and orderPinnedRoles, new in v0.6, pins pos 5's display order as authored, display-only, the scoring never reads the pin. And the statistics are guarded: shrinkage pulls thin cells toward a neutral, completion refills uncrawled cells by masked-cell cross-validation, weights hill-climb a train split and must clear holdout gains with bootstrap intervals before config.json changes. The engine is mined into the corpus in book 3, ch 31 (go 1.27, numerical statistics in go), book 3, ch 32 (go 1.27, fitting, backtesting, and guarded promotion), and book 3, ch 33 (go 1.27, the offline engine and its js twin), this chapter defends it as resume evidence.

25.3. open these five files first

The multiple-testing layer, vendored, Holm's step-down in 19 lines:

```

// Holm runs the step-down multiple-test procedure of Holm (1979) at
// family-wise level alpha: sort the p-values ascending, compare p(k)
// against alpha/(m-k+1) from k=1 up, reject while p(k) ≤ threshold, and
// every hypothesis from the first miss on is not rejected. The returned
// slice preserves the input order. Panics if any p or the level falls
// outside [0,1].
func Holm(ps []float64, alpha float64) []bool {
    validateFamily(ps, alpha)
    m := len(ps)
    order := ascending(ps)
    reject := make([]bool, m)
    for i, k := 0, 1; i < m; i, k = i+1, k+1 {
        if ps[order[i]] > alpha/float64(m-k+1) {
            break
        }
        reject[order[i]] = true
    }
    return reject
}

```

listing 21: holm: sort ascending, reject while $p(k) \leq \alpha/(m-k+1)$, first miss stops the family (source: defense-cookbook/vendored/dota-helper/engine/internal/stats/multiple.go, lines 9-27)

BH at lines 29-49 is the Benjamini-Hochberg step-up twin, and validateFamily at lines 63-74 panics on any p or level outside [0,1], NaN included, because a p-value is a probability and the caller has no error channel.

The deterministic bootstrap, vendored:

```
// PercentileCI bootstraps a percentile confidence interval for metric. It
// draws resamples replicates of n row indices with replacement (the ch17
// bootstrap, each replicate leaving roughly 1/e of the rows out of bag) from
// a PCG stream seeded (seed, seed), so identical calls are byte-identical,
// applies metric to each replicate, and returns the loPct and hiPct
// percentiles of the collected values. Percentiles are nearest-rank: the
// ceil(p/100 * m)-th smallest, clamped so p=0 is the minimum and p=100 the
// maximum. The idx slice handed to metric is reused between replicates and
// must not be retained. n < 1 or resamples < 1 leaves nothing to rank and
// returns NaN without calling metric.
func PercentileCI(metric func(idx []int) float64, n, resamples int, seed uint64, loPct, hiPct float64) (lo,
hi float64) {
    if n < 1 || resamples < 1 {
        return math.NaN(), math.NaN()
    }
    rng := rand.New(rand.NewPCG(seed, seed))
    idx := make([]int, n)
    vals := make([]float64, resamples)
    for r := range vals {
        for j := range idx {
            idx[j] = rng.IntN(n)
        }
        vals[r] = metric(idx)
    }
    sort.Float64s(vals)
    return nearestRank(vals, loPct), nearestRank(vals, hiPct)
}
```

listing 22: a percentile CI from a pcg stream seeded (seed, seed), identical calls byte-identical, nearest-rank percentiles (source: defense-cookbook/vendored/dota-helper/engine/internal/stats/bootstrap.go, lines 9-34)

nearestRank at lines 39-48 takes the ceil of pct times m before dividing by 100, so a dyadic percentile with an exact integer rank lands on it instead of one ulp above, a subtlety the repertoire wave's attack round measured once with a scan of dyadic percentiles against 20000 resamples and recorded in its audit ledger.

The golden walker, vendored:

```
// updateGoldens rewrites the testdata golden files instead of comparing, so
// pool or hub changes regenerate pins with `go test ./internal/emit -update`.
var updateGoldens = flag.Bool("update", false, "rewrite golden files under testdata")

func checkGolden(t *testing.T, name string, got []byte) {
    t.Helper()
    path := filepath.Join("testdata", name)
    if *updateGoldens {
        if err := os.MkdirAll("testdata", 0o755); err != nil {
            t.Fatal(err)
        }
        if err := os.WriteFile(path, got, 0o644); err != nil {
            t.Fatal(err)
        }
        return
    }
    want, err := os.ReadFile(path)
    if err != nil {
        t.Fatalf("read golden: %v (run go test ./internal/emit -update to generate)", err)
    }
    if string(want) != string(got) {
        t.Fatalf("%s drifted from testdata/%s:\n%s", name, name, firstDivergence(string(want), string(got)))
    }
}
```

listing 23: -update regenerates the testdata goldens, the plain run compares bytes and reports the first divergence (source: defense-cookbook/vendored/dota-helper/engine/internal/emit/golden_test.go, lines 10-33)

Two goldens pin the emitted artifacts, guide.golden and picker.golden, so any prose or data drift in the emitted JS fails a test with the differing window instead of shipping silently.

The serial rate limiter, vendored:

```
// rateLimiter combines a min spacing between requests with a token bucket:
// stratz enforces both, and the bucket takes over once the per-second burst
// allowance is spent.
type rateLimiter struct {
    c          clock
    interval   time.Duration
    rate       float64 // tokens per second, also the bucket capacity
    tokens     float64
    lastReq    time.Time
    lastRefill time.Time
}

func newRateLimiter(intervalMs, burstPerSec int, c clock) *rateLimiter {
    now := c.Now()
    return &rateLimiter{
        c:          c,
        interval:   time.Duration(intervalMs) * time.Millisecond,
        rate:       float64(burstPerSec),
        tokens:     float64(burstPerSec),
        lastRefill: now,
    }
}
```

listing 24: min spacing plus a token bucket, the clock injected so limiter tests run without real waiting (source: defense-cookbook/vendored/dota-helper/engine/internal/ingest/ratelimit.go, lines 16-37)

Wait at lines 40-61 blocks until the next request is allowed, deriving its wait from spacing first and the bucket second, and the stratz wiring reads both knobs from the hub: interval 420 ms and burst 7 per second in the config.json stratz block. The injected clock is what makes the limiter testable without sleeping, the same seam the corpus teaches.

The one writer over the hub, vendored, the guard sequence:

```
orderedFlat := make([]config.PoolEntry, 0, len(deps.Cfg.Pool))
for _, r := range deps.Cfg.Roles {
    if len(ordered[r.ID]) == 0 {
        return fmt.Errorf("order: role %s has no pool entries", r.ID)
    }
    orderedFlat = append(orderedFlat, ordered[r.ID]...)
}
cfgOut, err := renderConfigPool(cfgOrig, orderedFlat)
if err != nil {
    return err
}
gatesOut, err := renderGatesFallback(gatesOrig, deps.Cfg.Roles, ordered)
if err != nil {
    return err
}
if err := assertConfigPreserved(cfgOrig, cfgOut, orderedFlat); err != nil {
    return err
}
if err := assertGatesPreserved(gatesOrig, gatesOut, deps.Cfg.Roles, ordered); err != nil {
    return err
}
if err := validateRenderedConfig(cfgOut, orderedFlat); err != nil {
    return err
}
if err := validateRenderedGates(gatesOut, deps.Cfg.Roles, ordered); err != nil {
    return err
}
```

listing 25: the re-sort computes, splices, then must survive preserved-value and rendered-validation asserts before any write (source: defense-cookbook/vendored/dota-helper/engine/internal/order/order.go, lines 155-181)

And the splice itself:

```
// renderConfigPool splices the pool array body in place; every byte outside
// the array rides through verbatim.
func renderConfigPool(src []byte, ordered []config.PoolEntry) ([]byte, error) {
    valAt, err := findTopLevelValue(src, "pool")
    if err != nil {
        return nil, err
    }
    closeAt, err := matchBracket(src, valAt)
    if err != nil {
        return nil, err
    }
    parts := make([]string, 0, len(ordered))
    for _, e := range ordered {
        parts = append(parts, renderPoolEntry(e))
    }
    out := make([]byte, 0, len(src)+256)
    out = append(out, src[valAt+1]...)
    out = append(out, []byte(strings.Join(parts, ",")+"\n "))...
    out = append(out, src[closeAt:]...)
    return out, nil
}
```

listing 26: the pool array body replaced in place, every byte outside the array rides through verbatim (source: defense-cookbook/vendored/dota-helper/engine/internal/order/order.go, lines 320-340)

The package doc at lines 1-7 owns exactly ordering in two files, config.json and gates.json, membership is never touched, and since v0.6 it names the display pin: roles in the hub's orderPinnedRoles keep their authored declaration order, pos 5 today, display-only, the scoring never reads it. validateRenderedConfig at lines 486-512 reloads the rendered bytes through the full hub loader before anything reaches disk. That is the single-authored-hub rule enforced by code instead of discipline.

The walkthrough is five files, the family, the interval, the walker, the limiter, the writer:

stats/multiple.go lines 9-27, holm step-down, bh twin, panics outside [0,1]
stats/bootstrap.go lines 9-34, the pcg-seeded percentile CI, nearest-rank exactness
emit/golden_test.go lines 10-33, -update regenerates, the plain run reports first divergence
ingest/ratelimit.go 16-37 and order/order.go 155-181, injected clock and guarded hub re-sort

figure 99: holm, the pcg interval, the golden gate, the spaced limiter, the guarded splice

25.4. drills

Your engine has zero goroutines. In 2026. Is that junior fear or a decision?

A decision with a stated boundary. The workload is one crawl paced at a request per 420 ms under a 7 per second burst cap, one engine process, one refresh command the Makefile documents as serial only, never -j. Serial removes whole bug classes, races, lock order, lost wakeups, that would otherwise need the race detector and review attention the project would rather spend on the statistics. The duckdb handle and the raw caches have a single owner by construction, and the measured fact backs it: 0 goroutine statements and 0 sync imports in non-test engine source, the 2 sync imports sit in test servers. It would be wrong the day the workload stops being one stream: a crawl fanned out over heroes with independent rate buckets, or an ingest and emit split into concurrent stages, would need the patterns book's toolkit and

the race detector, and at that point serial is leaving real throughput on the table. The engine docker discipline is taught in the twin chapter.

floor: book 3, ch 33 (go 1.27, the offline engine and its js twin)

Why DuckDB of all things? SQLite would do, or Postgres.

The workload is analytical SQL over a local single-user dataset: pooling weekly rows into (wins, n) cells per directed pair, then scanning whole matrices for shrinkage and completion. That is a columnar scan shape, which is what DuckDB is, embedded, one file under var/, no server to run or operate, and the whole engine stays offline and reproducible. SQLite is row-oriented and would carry the same queries slower, Postgres is a server this repo has no operations story for. The honest cost is the cgo seam: the duckdb driver has no supported native windows toolchain, so every build and test wraps the golang:1.27 docker image through make, which is also why the suite runs 30 to 60 minutes. One direct dependency carries that trade, and it is written down in engine/go.mod.

How do you know the scorer is not overfit to your own matches?

Four separate guards, each pinned by tests. The fit splits time-ordered league drafts 70/30, SplitMatches in metrics.go takes the first 70 percent as train and the last 30 as holdout, and weight candidates hill-climb train mean pick percentile only, gains are measured on the holdout. Promotion refuses a proposal unless the holdout gain clears the configured minimum with a bootstrap interval whose low end is above zero, the alphas guard requires a bootstrap interval excluding the current value, and the completion guard requires a difference interval above zero from masked-cell cross-validation, rank and lambda picked by held-out Spearman. The backtest replays drafts pick by pick using only previously visible information, so no lookahead leaks into the metrics. And the guard re-checks against live config before any rewrite: a failing guard leaves every file untouched. None of that proves a draft model generalizes to a new patch, which is why the repo is patch-bounded and says so.

floor: book 3, ch 32 (go 1.27, fitting, backtesting, and guarded promotion)

Why an exact binomial instead of a normal approximation or a Wilson interval?

The cells are small. A weekly pair sample runs thin, and the normal approximation to the binomial is bad exactly there, in the tails and near p 0 or 1 where these counts live, while Wilson is an interval estimator answering a different question. What the audit needs is a point probability: how surprising is this observed win count under the hero baseline. BinomialTwoSided sums the pmf over every outcome at most as likely as the observed one, evaluated in log space through lgamma so large n never multiplies tiny factorials, with a relative tolerance so floating-point ties still count, and point masses at p 0 and 1 answered outright. The exactness costs a loop over n cells, which is nothing at this scale, and it composes with the family corrections, Holm and Benjamini-Hochberg, which need honest p-values as inputs.

floor: book 3, ch 31 (go 1.27, numerical statistics in go)

Why emit a JS artifact instead of serving JSON from an API?

Because the product is static pages with zero runtime dependencies, and a served API would trade that for a process to run, monitor, and pay for. The engine emits picker-data-generated.js, the pages import it and run the whole draft score in the browser, a linear score plus authored gate rules over precomputed cells, so the only moving part in production is a static file host. The artifact is not hand-maintained: it is engine output, pinned by goldens, regenerated by make refresh, and carrying its own date and patch fields, 2026-10-05 and 7.41f in the emitted header. The honest cost is freshness: the data is as current as the last commit, which is the right trade for a patch-bounded tool. And the score living in picker-score.js, 232 lines with 20 node tests, is what makes the browser twin auditable rather than a black box.

floor: book 3, ch 33 (go 1.27, the offline engine and its js twin)

You have apriori, naive bayes, knn, and ensembles in there. Why does none of it feed the live score?

Because they are comparators, not components. The live score is a 7-term linear scorer with authored gates, and its claim to be good enough is an empirical one, so the eval lane benchmarks it against baselines on the same drafts: levelwise apriori over side-composition triples, a bernoulli naive bayes with Laplace smoothing, cosine knn over one-hot drafts, and a bagged AUC-fitted blend. They run in eval, their output lands in the report, and nothing in the emit pipeline reads it, the emitted JS is built from the mined cells and the hub weights only. The moment a comparator fed the live score, the tool would stop being a defensible small model and become an ensemble nobody can audit in an interview, and the report-only boundary is what keeps the linear scorer falsifiable against real alternatives instead of just asserted.

floor: book 3, ch 32 (go 1.27, fitting, backtesting, and guarded promotion)

What do the -count=1 flags and the golden gates actually buy?

Execution on demand and a hard pin on the artifact. Every engine test and cover target in the Makefile passes -count=1, so a run about to trust a golden never answers from a warm cache, it executes the suite, and the discipline needs no argument about what the cache would have replayed. The golden gate is the pin: the emit tests compare rendered bytes against testdata goldens and report the first divergent line pair, so a prose edit, a config change, or an engine tweak that alters output fails loudly with the exact differing lines instead of shipping silently. Regeneration is a separate explicit walk, make emit-goldens runs the -update flag, so an intentional change and an accidental one can never look alike. The twin chapter teaches the same cache and golden discipline in the corpus itself.

floor: book 3, ch 33 (go 1.27, the offline engine and its js twin)

Each drill keeps the evidence honest, then pins the location:

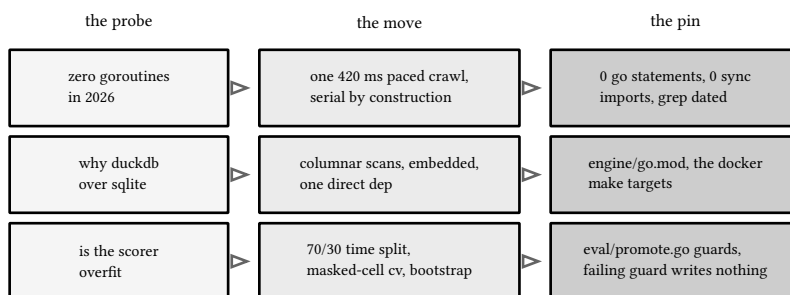


figure 100: hobby scale but real method, pinned to files

VERIFY: the five file locate drill

Repo tree for the package layout, stats/multiple.go for the family corrections, stats/bootstrap.go for the deterministic interval, emit/golden_test.go for the artifact pin, ingest/ratelimit.go for the crawl seam, order/order.go for the one guarded writer over the hub. The scale claims stay hobby scale, this chapter only has to prove the method is real. The engine's statistics are taught in the go book's three data chapters, this chapter defends the repo as evidence.